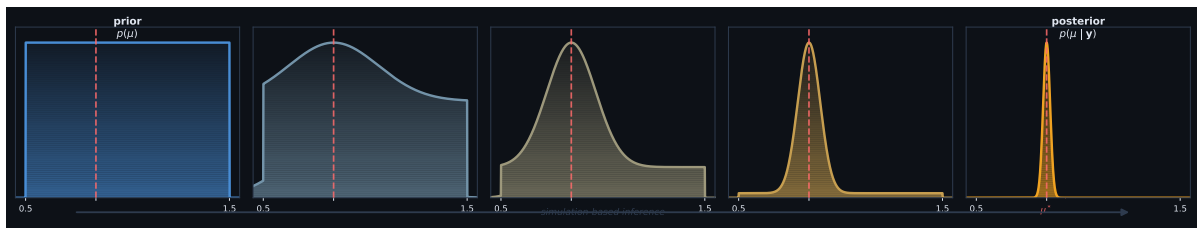


Simulation-based Inference for Parameter Density Estimation in Dynamical Systems



Master's Thesis M-04/2026-1391

Aritra Saha
Student ID 10065012

Hannover, 30 April 2026

First Examiner	Prof. Dr.-Ing. Marius Lindauer
Second Examiner	Prof. Dr.-Ing. Thomas Seel
Supervisor	Jan-Hendrik Ewering

Declaration of Authorship

Name: Aritra Saha
Student ID: 10065012

Thesis title: Simulation-based Inference for Parameter Density Estimation in Dynamical Systems

Type of thesis: Master's Thesis
Degree program: Computer Science
Submission date: 30 April 2026

First examiner: Prof. Dr.-Ing. Marius Lindauer
Second examiner: Prof. Dr.-Ing. Thomas Seel
Supervisor: Jan-Hendrik Ewering

I, *Aritra Saha*, hereby declare that the Master's Thesis entitled *Simulation-based Inference for Parameter Density Estimation in Dynamical Systems* was written independently, that no sources or aids other than those indicated were used, that all passages taken literally or analogously from other sources are marked as such, and that the thesis has not previously been submitted to any examination board in the same or a similar form.

I also consent to the transmission of my work to external plagiarism-checking services for review by plagiarism-detection software.

Hannover, 30 April 2026

(Aritra Saha)

Master's Thesis

Aritra Saha, Matr. No. 10065012

Simulation-based Inference for Parameter Density Estimation in Dynamical Systems

Introduction:

Simulation-based inference (SBI) is a well-established approach for estimating the probability density functions of simulator parameters, given a real-world dataset. Various simulation-based inference approaches rely on neural density estimators and follow a so-called amortized inference strategy. The main idea is to perform a—potentially computationally costly—offline training procedure and to amortize this “invest” by fast and computationally cheap online evaluations of the neural density estimator [1]. While neural amortized inference approaches are starting to mature in settings with black-box simulations and independent, identically distributed data points, there is still significant potential to leverage their advantages in other inference tasks. In particular, time-series data generated from dynamical systems exhibits a conditionally dependent structure, and the underlying system can usually be considered Markovian. These (and other) properties are exploited in some initial works to tailor amortized SBI methods in dynamical systems [2,3].

Task description:

The goal of this thesis is to analyze the potential of recent amortized SBI methods for online parameter density estimation in real-world systems, and to extend these methods, for instance, to cope with incomplete state observations. To this end, a baseline neural density estimator, such as neural posterior estimation (NPE) [1], and a state-of-the-art neural density estimator for time-series data, such as neural posterior score estimation (NPSE) [2,3], should be implemented and compared against each other in a case study. Moreover, an extension of the state-of-the-art amortized SBI method for hidden Markov models should be investigated. A promising strategy, formulated in [3, Sec. 5.2], is to use data assimilation strategies [4] within an NPSE framework.

In particular, the following tasks arise:

- Literature research regarding NPE, NPSE, and related amortized SBI approaches for time-series data.
- Implementation of a baseline and a state-of-the-art amortized SBI method.
- Comparison and analysis of their potential for parameter inference in representative case studies with dynamical systems that exhibit a Markovian property. Relevant evaluation criteria are the convergence speed, computational efficiency, and accuracy of the parameter posterior.
 - First, simulative evaluation with an existing simulator (robot or vehicle).
 - Second, real-world evaluation with an existing dataset (robot or vehicle).
- Investigation of extensions of the state-of-the-art method for hidden Markov models.
- Ablation studies with the above methods regarding algorithmic hyperparameters.
- Documentation of the work.

The number of working hours for the thesis is 900h.

Starting date: 01.11.2025

End date: 30.04.2026

Supervisor: Jan-Hendrik Ewering

.....
(First examiner)

.....
(Aritra Saha)

.....
(Second examiner)

References

- [1] A. Zammit-Mangion, M. Sainsbury-Dale, and R. Huser, "Neural Methods for Amortized Inference," Annual Review of Statistics and Its Application, vol. 12, no. 1, pp. 311–335, 2025, doi: 10.1146/annurev-statistics-112723-034123.
- [2] L. Sharrock, J. Simons, S. Liu, and M. Beaumont, "Sequential Neural Score Estimation: Likelihood-Free Inference with Conditional Score Based Diffusion Models," in Forty-first International Conference on Machine Learning, 2024.
- [3] M. Glöckler, S. Toyota, K. Fukumizu, and J. H. Macke, "Compositional simulation-based inference for time series," in The Thirteenth International Conference on Learning Representations, 2025.
- [4] F. Rozet and G. Louppe, "Score-based Data Assimilation," in Thirty-seventh Conference on Neural Information Processing Systems, 2023.

.....
Titel der Arbeit (Druckbuchstaben)

.....
Nr. der Arbeit

.....
Betreuer

Vereinbarungen zur Durchführung der Arbeit:

- Die Arbeit soll in enger Absprache mit dem Betreuer durchgeführt werden. Dazu ist mindestens einmal wöchentlich Rücksprache zu halten.
- Der schriftliche Teil der Arbeit ist in drei gebundenen Exemplaren sowie auf einem elektronischen Datenträger abzugeben. Im Rahmen der Arbeit entstandene elektronische Daten sind vollständig auf einem Datenträger mit entsprechender Dokumentation abzugeben.
- Bei der Erstellung von Quelltexten sind die Instituts-Konventionen einzuhalten. Quelltexte sind angemessen zu kommentieren.
- Die Arbeit sollte dem Betreuer 2-3 Wochen vor dem Abgabetermin zur Durchsicht vorgelegt werden.
- Geräte und Maschinen dürfen nur nach Rücksprache mit dem Betreuer und ggf. einer Sicherheitseinweisung benutzt werden. Die Laborordnung ist durch Aushang bekannt und ist einzuhalten.
- Die Arbeit darf ohne Genehmigung der Leibniz Universität Hannover nicht veröffentlicht oder an Dritte weitergegeben werden.
- Ausgehändigte Schlüssel und Transponder sind nach Beendigung der Arbeit zurückzugeben.

Der(die) Unterzeichnende stimmt den oben genannten Vereinbarungen zu:

.....
Name, Vorname

.....
Anschrift

.....
E-Mail-Adresse

.....
Telefonnummer

.....
Ort, Datum

.....
Unterschrift

Abstract

Identifying physical parameters of vehicle dynamics models from sensor measurements is challenging because the simulator likelihood is intractable. Simulation-based inference (SBI) addresses this by training a neural network offline on simulated data to approximate the Bayesian posterior, enabling amortised inference without repeated likelihood evaluations. This thesis presents a controlled comparison of four amortised SBI methods—NPE, NPSE, FNPSE, and Simformer—for static parameter estimation in a two-track vehicle model. Up to three physical parameters are inferred (tire–road friction μ , drag coefficient c_d , vehicle mass m) from nine observation channels. A benchmark of 432 experiments varies methods, parameter configurations, trajectory lengths, and simulation budgets, evaluated with parameter-recovery, calibration, posterior predictive, and runtime metrics. NPE achieves the lowest parameter-recovery error with near-instantaneous sampling. Simformer provides the highest empirical coverage, often through wider, less informative posteriors, and has substantially longer sampling times. NPSE is competitive with NPE for friction-only and friction–drag inference, but degrades when mass is included and is systematically overconfident. FNPSE does not achieve competitive posterior quality as global parameters require full-trajectory information. The results are complemented by a real-data evaluation on seven brake-test recordings.

Contents

1	Introduction	1
1.1	Motivation	2
1.2	Problem Setting and Research Questions	3
1.3	Contributions and Scope	4
1.4	Structure of the Thesis	4
2	Related Work	5
2.1	Simulation-Based Inference for Dynamical Systems	5
2.2	Neural Posterior Estimation	6
2.2.1	Direct Posterior Approximation	6
2.2.2	Density Approximators	6
2.2.3	Normalising Flows	7
2.3	Score-Based Diffusion Models for Simulation-Based Inference	8
2.3.1	Score Matching and Diffusion Models	8
2.3.2	Neural Posterior Score Estimation	8
2.3.3	Factorized Neural Posterior Score Estimation	9
2.3.4	Simformer	10
2.4	Scope and Positioning of This Thesis	13
3	Problem Setting and Vehicle Case Study	14
3.1	Problem Formulation	14
3.2	Two-Track Vehicle Model	15
3.3	State, Observation, and Control Vectors	19
3.4	Driving Scenario Generation	20
3.5	Noise Model	21
3.6	Real Measurement Data	22
3.7	Parameters of Interest and Prior Distributions	23
3.7.1	Parameter Selection	23
3.7.2	Prior Distributions	24
3.7.3	Normalisation	25

4	Evaluated Methods	26
4.1	NPE Baseline	27
4.1.1	Embedding Network	27
4.1.2	Normalising Flow Architecture	29
4.1.3	Training Procedure	29
4.2	NPSE	30
4.2.1	Score Network Architecture	30
4.2.2	Diffusion Process and Training	30
4.2.3	Sampling by Reverse Diffusion	31
4.3	FNPSE	31
4.3.1	Score Network Architecture	32
4.3.2	Training Data Generation	33
4.3.3	Training Procedure	34
4.3.4	Score Composition at Inference	34
4.3.5	Sampling	35
4.4	Simformer	35
4.4.1	Scalar Node Representation	35
4.4.2	Transformer Architecture	36
4.4.3	All-Conditional Training	36
4.4.4	SDE and Sampling	37
4.4.5	Differences from NPE and NPSE	37
5	Comparative Evaluation	39
5.1	Experimental Setup	39
5.1.1	Benchmark Grid	39
5.1.2	Evaluation Protocol	40
5.2	Simulative Comparison	42
5.2.1	Single-Parameter Estimation (μ only)	43
5.2.2	Two-Parameter Estimation (μ, c_d)	52
5.2.3	Two-Parameter Estimation (μ, m)	54
5.2.4	Three-Parameter Estimation (μ, c_d, m)	56
5.2.5	Cross-Configuration Analysis	59
5.3	Real-World Comparison	65
5.3.1	Simulator Calibration via PSO	65
5.3.2	SBI-Based Real-Data Inference	69
5.4	Discussion of Comparative Results	73
5.4.1	Method Strengths and Weaknesses	74

5.4.2	The Calibration–Accuracy Trade-Off	75
5.4.3	Limitations	76
5.4.4	Connection to Chapter 6	79
6	Conclusion and Outlook	80
6.1	Summary of Contributions	80
6.2	Scope, Limitations, and Choice of Focus	82
6.3	Outlook and Future Directions	84
6.3.1	Score-Based Data Assimilation for Hidden Markov Models	84
6.3.2	Improved Calibration for Flow-Based Methods	85
6.3.3	Higher-Dimensional Parameter Spaces	85
6.3.4	Alternative Embedding Architectures	85
6.3.5	Online and Sequential Inference	86
	Appendix	87
A.1	Data Assimilation Outlook	88
A.1.1	Motivation: From Full-State to Partial Observations	88
A.1.2	Score-Based Data Assimilation	89
A.1.3	Connection to FNPSE and Joint Inference	90
A.1.4	Application to Vehicle Dynamics	90
A.1.5	Expected Challenges	91
	Bibliography	92

List of Figures

2.1	Normalising flow transformation from Gaussian to target density.	7
2.2	Score-based SDE: forward corruption and reverse generation.	8
2.3	FNPSE score decomposition and accumulation overview.	10
2.4	Simformer: all-conditional joint inference overview.	11
2.5	Simformer token architecture and attention masking.	12
5.1	Illustrative diagnostic trajectories for three friction levels.	41
5.2	Parameter-recovery W_2 for μ -only estimation, all methods.	44
5.3	Calibration diagnostics for μ -only estimation.	45
5.4	Prior-posterior comparison for μ -only estimation, all methods.	47
5.5	PPC time-series for μ -only estimation (NPE and NPSE).	48
5.6	PPC time-series for μ -only estimation (FNPSE and Simformer).	49
5.7	W_2 versus trajectory length for μ -only estimation.	50
5.8	Per-parameter metrics for (μ, c_d) configuration.	53
5.9	Per-parameter metrics for (μ, m) configuration.	55
5.10	Calibration diagnostics for (μ, m) estimation.	56
5.11	Per-parameter metrics for (μ, c_d, m) configuration.	58
5.12	Joint W_2 for (μ, c_d, m) estimation, all budgets.	59
5.13	Training time versus simulation budget, μ -only estimation.	61
5.14	Pareto front: W_2 versus training time, μ -only.	62
5.15	Pareto front: PPC RMSE versus sampling time, μ -only.	63
5.16	Metric heatmap for μ -only estimation at $B = 60$ M.	64
5.17	PSO convergence history over 3 000 iterations.	68
5.18	Simulated vs. real trajectory for one Asphalt recording.	70

List of Tables

3.1	Fixed geometric parameters of the two-track vehicle model.	16
3.2	Pacejka Magic Tire Formula parameters.	17
3.3	Fixed dynamic parameters of the vehicle model.	19
3.4	Prior design bounds and Gaussian parameters for inferred quantities.	24
4.1	Summary of implementation differences across the four inference methods.	38
5.1	Benchmark grid dimensions and parameter combinations.	40
5.2	Benchmark results for μ -only estimation at $T_{\text{seg}} = 1000$	43
5.3	Benchmark results for (μ, c_d) estimation at $T_{\text{seg}} = 1000$	52
5.4	Benchmark results for (μ, m) estimation at $T_{\text{seg}} = 1000$	54
5.5	Benchmark results for (μ, c_d, m) estimation at $T_{\text{seg}} = 1000$	57
5.6	Global simulator parameters: default vs. PSO-optimised.	67
5.7	Per-trajectory friction coefficients from PSO calibration.	67
5.8	Real-data evaluation grid (best checkpoints at $T_{\text{seg}} = 1000$).	71
5.9	Simulative diagnostics of selected real-data checkpoints.	71
5.10	Per-recording PPC RMSE for μ -only inference.	72

Nomenclature

General Conventions

Scalar	Lowercase or uppercase italic letter: a, A, μ
Vector	Lowercase symbol; boldface used where typographically convenient: $\mathbf{x}, \boldsymbol{\theta}$
Matrix	Bold uppercase letter: $\mathbf{A}, \mathbf{J}$
Distribution	Calligraphic letter: $\mathcal{N}, \mathcal{U}$
Subscript	Descriptive subscripts in roman: T_{seg}, c_d

Latin Letters

a	Diffusion time (noise level) in the score-based SDE
A	Maximum diffusion time; upper limit of the noise schedule
B	Total simulation-step budget NPE/NPSE: $B = N_{\text{sim}} \cdot T_{\text{seg}}$ FNPSE: $B \approx N_{\text{sim}} \cdot (W + f_{\text{pilot}} \cdot T_{\text{pilot}})$, $W = 2$, $f_{\text{pilot}} = 0.02$, $T_{\text{pilot}} = 1500$
c_d	Aerodynamic drag coefficient
d_e	Embedding dimension
$d_{\boldsymbol{\theta}}$	Dimension of the parameter vector $\boldsymbol{\theta}$
d_x	Dimension of the state vector $\mathbf{x}$
d_y	Dimension of the observation vector $\mathbf{y}$
$\mathbf{e}$	Embedding vector produced by the embedding network
f_{pilot}	Fraction of simulations used as pilot data (FNPSE)
H	Hidden dimension of the GRU encoder
h_{ψ}	Embedding network with parameters ψ
I_z	Vehicle yaw moment of inertia [kg m^2]
K	Number of real-data recordings
l_f, l_r	Distance from centre of gravity to front/rear axle [m]
m	Vehicle mass [kg]
N_{sim}	Number of simulated training trajectories
N_{test}	Number of held-out test observations
p_0	Base distribution of the normalising flow (standard Gaussian)
q_{ϕ}	Learned posterior approximation (normalising flow)

r_w	Effective tire radius [m]
s_ψ	Conditional score network for NPSE
s_ϕ	Local score network for FNPSE
S	Number of posterior samples drawn per test observation
T	Number of transition windows / observation steps in score composition
T_{pilot}	Length of FNPSE pilot trajectories [time steps]
T_{seg}	Trajectory segment length [time steps]
$\mathbf{u}_{0:T}$	Control input trajectory
W	Local window width in FNPSE ($W = 2$ for one-step transitions)
W_2	Wasserstein-2 / parameter-recovery distance
W_f, W_r	Front/rear track width [m]
$\mathbf{x}_{0:T}$	Latent state trajectory
$\mathbf{y}_{0:T}$	Observation trajectory

Greek Letters

α_t	Attention weight at time step t (BiGRU attention pooling)
δ_{θ^*}	Dirac delta at the generating parameter θ^*
$\Lambda(\theta_a)$	Total precision matrix in the Gaussian-corrected score composition
λ_a	Noise-level weighting function in the DSM objective
μ	Tire-road friction coefficient
Σ_a	Prior denoising covariance at noise level a
$\Sigma_{a,t,t+1}$	Local denoising covariance for transition $(t, t + 1)$ at noise level a
σ	Noise standard deviation / diffusion scale
θ	Parameter vector (inference target)
θ_a	Parameter vector corrupted by diffusion noise at level a
θ^*	True (generating) parameter value
$\tilde{p}(\mathbf{x}_t)$	Proposal distribution over states, independent of θ (FNPSE)
ϕ	Parameters of the score network or normalising flow
ψ	Parameters of the embedding network

Abbreviations

ABS	Anti-lock Braking System
APT	Automatic Posterior Transformation
BiGRU	Bidirectional Gated Recurrent Unit

C2ST	Classifier Two-Sample Test
CNN	Convolutional Neural Network
CoG	Centre of Gravity
DSM	Denoising Score Matching
FNPSE	Factorized Neural Posterior Score Estimation
GFE	Gaussian Fourier Embedding
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
MAF	Masked Autoregressive Flow
MCMC	Markov Chain Monte Carlo
MTF	Magic Tire Formula
MLP	Multi-Layer Perceptron
NPE	Neural Posterior Estimation
NPSE	Neural Posterior Score Estimation
NRMSE	Normalized Root Mean Squared Error
ODE	Ordinary Differential Equation
PPC	Posterior Predictive Check
PSO	Particle Swarm Optimization
RK4	Fourth-Order Runge–Kutta
RMSE	Root Mean Squared Error
SBI	Simulation-Based Inference
SBC	Simulation-Based Calibration
SDE	Stochastic Differential Equation
VE-SDE	Variance Exploding SDE

1 Introduction

Dynamical systems appear in many engineering applications because they provide a structured way to describe how physical quantities evolve over time. In practice, however, even when the dynamics of a system are represented by a computational model—a simulator that takes parameters as input and generates sensor-observable outputs—determining the posterior probability of a given parameter set from observed data is often intractable. Internal states are only partially observed, measurements are corrupted by noise, and relevant model parameters are uncertain. In vehicle dynamics, for example, the same steering, braking, or acceleration manoeuvre can lead to different measured trajectories depending on quantities such as the tire–road friction level, the vehicle mass, or aerodynamic drag. Recovering such parameters from time-series measurements is therefore a system identification problem.

In many applications, a single point estimate is not sufficient. A parameter value may be plausible only within a range, and different parameter combinations may explain the same observed trajectory to different degrees. For this reason, a Bayesian perspective is attractive: instead of returning only one estimate, one aims to infer a posterior distribution over the unknown parameters, conditioned on the observed data. The central object of interest is thus the posterior over static vehicle parameters, given a multivariate observation trajectory.

For nonlinear dynamical systems with complex simulators, this inference problem is typically intractable: closed-form likelihoods are unavailable, and explicit probabilistic inference can be computationally prohibitive. Classical Bayesian methods such as particle MCMC can still be applied, but their cost grows with observation horizon length, state-space dimensionality, and posterior complexity. At the same time, many practical engineering problems are inherently sequential: the data arrive as time-series, the simulator may have a Markovian structure, and useful parameter information is distributed unevenly across the trajectory.

SBI addresses this setting by replacing explicit likelihood evaluation with repeated forward simulation. Rather than requiring a tractable formula for the likelihood, it assumes only that one can sample parameter values and generate corresponding trajectories from the simulator. Neural SBI methods developed since 2016 [PM16] are especially attractive because they support amortised inference: computational effort is invested offline during simulation and training, and the learned estimator can then be reused for fast posterior inference on new observations. In principle, this makes SBI well suited for settings in which repeated online parameter inference

is required.

However, the choice of inference method is not neutral when the data are long multivariate time-series from a dynamical system. Global posterior estimators such as NPE treat the full trajectory as one conditioning object. Score-based methods introduced in 2024, such as NPSE [SSLB24], replace normalised density estimation by posterior score estimation and diffusion-based sampling. Structure-aware approaches such as FNPSE go further by attempting to exploit the Markovian structure of the simulator explicitly. Simformer broadens the perspective again by modelling a joint distribution and using transformer-based conditioning. These approaches differ not only in their modelling assumptions, but also in their expected trade-offs with respect to simulation efficiency, calibration, scalability, and runtime.

This thesis focuses on that comparison in the context of vehicle dynamics. More specifically, it studies posterior inference for static physical parameters from simulated multivariate vehicle trajectories under a common experimental pipeline. The emphasis is on the simulation setting, where ground truth is available and the effect of modelling and training choices can be assessed rigorously. Real measurement data are included as an additional plausibility and robustness check, but not as the primary source of quantitative truth.

1.1 Motivation

From a practical perspective, vehicle dynamics are a case study with high-dimensional multi-channel observations and long temporal sequences. The unknown parameters have well-defined physical meaning, the observations are temporally structured, and the simulator often exhibits Markovian dynamics. At the same time, different parameters leave different signatures in the trajectory. A change in mass affects inertial response, a change in friction alters achievable tire forces and therefore transient behaviour, and a change in aerodynamic drag grows quadratically with speed (Eq. 3.12). These effects overlap, but they do not overlap in the same way across all parts of a manoeuvre.

This immediately creates two challenges. First, the inference problem is not only high-dimensional in the observation space, but also structured in time. Second, method quality cannot be judged only by whether the posterior mean is close to the true parameter. A practically useful method must also produce calibrated uncertainty and posterior samples that remain dynamically plausible under the simulator. For long sequential data, it is therefore an open question whether a normalising-flow-based global posterior estimator (NPE) is sufficient, or whether score-based or structure-aware methods provide measurable advantages in accuracy, calibration, or computational cost.

The motivation of this thesis is to answer that question in a controlled and well-documented way. Rather than proposing yet another inference algorithm, the thesis investigates how current amortised SBI approaches actually behave on a realistic engineering identification problem when the comparison is made fair. This includes matched simulation budgets, varying trajectory lengths, and a broader evaluation protocol than pointwise parameter error alone.

1.2 Problem Setting and Research Questions

Let θ denote the static model parameters, $\mathbf{x}_{0:T}$ the latent state trajectory, and $\mathbf{y}_{0:T}$ the corresponding observation trajectory. The inference target is the posterior

$$p(\theta \mid \mathbf{y}_{0:T}), \quad (1.1)$$

that is, the distribution over static vehicle parameters conditioned on a measured multivariate time-series. In the present thesis, the main parameters of interest are physically interpretable quantities such as friction, mass, and aerodynamic drag. The observations consist of simulated or measured vehicle trajectories, including quantities such as global-frame velocities, body-frame accelerations, yaw rate, and wheel-speed channels.

Against this background, the thesis addresses the following research questions:

1. How do NPE, NPSE, FNPSE, and Simformer compare for posterior inference over static vehicle parameters under matched simulation budgets?
2. How does trajectory length affect this comparison?
3. Do score-based or structure-aware methods improve calibration or posterior predictive consistency relative to a normalising-flow-based NPE baseline?
4. What runtime and efficiency trade-offs arise between these methods during training and inference?
5. How robust are the conclusions obtained in simulation when the trained methods are applied to real measurement trajectories?

These questions reflect the scope and positioning argued in Chapter 2: despite rapid progress in amortised SBI, there is not yet a focused, calibration-aware, and budget-matched comparison of these method families for posterior inference over static physical parameters in a Markovian vehicle dynamics model with multivariate time-series observations.

1.3 Contributions and Scope

The main contribution of this thesis is a controlled comparative study of amortised SBI methods for static parameter inference in vehicle dynamics. Concretely, the thesis makes the following contributions:

- It formulates a vehicle-dynamics parameter inference problem in a unified Bayesian and simulation-based framework.
- It implements and compares four relevant amortised SBI approaches-NPE, NPSE, FNPSE, and Simformer-within a common pipeline.
- It evaluates these methods under controlled simulation budgets and varying trajectory lengths.
- It uses a broader metric set that includes posterior quality, posterior predictive behaviour, calibration, and runtime rather than relying only on pointwise parameter error.
- It applies the resulting inference pipeline to real vehicle measurement data as a stress test of robustness and posterior predictive plausibility.

The thesis does not propose a fundamentally new SBI method. Its central contribution is empirical and methodological in the benchmarking sense: it provides a technically detailed and controlled analysis of how current amortised SBI approaches behave on a practically relevant vehicle-dynamics problem. Likewise, the real-data study is not presented as a full ground-truth benchmark, since the true posterior is unknown there.

1.4 Structure of the Thesis

The remainder of this thesis is organised as follows. Chapter 2 reviews the relevant background and related work, covering SBI for dynamical systems, NPE as a baseline, score-based approaches (NPSE, FNPSE), and Simformer. Chapter 3 defines the problem setting and vehicle case study, including the simulator, observations, real measurement data, and prior distributions. Chapter 4 describes how each method is instantiated within the common experimental pipeline. Chapter 5 presents the comparative evaluation under matched simulation budgets, varying trajectory lengths, and a broad metric set, including a real-data robustness check. Chapter 6 summarises the findings, discusses limitations, and outlines future directions.

2 Related Work

This chapter reviews the literature that underpins the comparative study presented in this thesis. Section 2.1 discusses the specific challenges of SBI for dynamical systems and the role of temporal structure. Sections 2.2 and 2.3 review the four methods compared in this thesis: NPE as the flow-based baseline, and score-based approaches (NPSE, FNPSE, Simformer). Section 2.4 identifies the specific research gap that this thesis addresses relative to the existing literature. Throughout this chapter, θ denotes the static model parameters, $\mathbf{x}_{0:T}$ the latent state trajectory, $\mathbf{y}_{0:T}$ the observation trajectory, and $\mathbf{u}_{0:T}$ the control inputs, following the notation introduced in Chapter 1. The inference target is the posterior $p(\theta \mid \mathbf{y}_{0:T})$.

2.1 Simulation-Based Inference for Dynamical Systems

Applying SBI to dynamical systems raises challenges that go beyond the static setting. Input dimensionality grows with trajectory length, parameter information is distributed unevenly across time rather than concentrated in a few summary statistics, and many simulators—including vehicle models—exhibit Markovian structure: the next state depends only on the current state, the inputs, and the parameters. This creates both a difficulty and an opportunity. A method that ignores Markovian structure processes the full sequence as one undifferentiated block, which can be wasteful. A method that exploits it can decompose inference into local transition-level contributions, potentially improving simulation efficiency and scalability for long sequences [ZMSDH25, GTFM24].

Reliability is a second major issue. Neural posterior approximations can appear visually plausible while being biased or miscalibrated. Comparative benchmarks show that posterior quality varies by up to an order of magnitude across tasks and methods [LBG⁺21], and targeted diagnostics reveal that even apparently good approximations can be misleading under closer scrutiny [HDR⁺22]. For vehicle dynamics, this matters: a posterior over friction or mass is useful only if both its location and its stated uncertainty are trustworthy.

The following sections focus on the specific SBI method families relevant to this comparison.

2.2 Neural Posterior Estimation

2.2.1 Direct Posterior Approximation

Among neural SBI methods, NPE is one of the most established approaches for direct posterior approximation. Its goal is to learn a conditional density model

$$q_\phi(\boldsymbol{\theta} \mid \mathbf{y}_{0:T}) \approx p(\boldsymbol{\theta} \mid \mathbf{y}_{0:T}) \quad (2.1)$$

from simulated parameter–observation pairs $(\boldsymbol{\theta}, \mathbf{y}_{0:T})$. Unlike approaches that first approximate a likelihood or a likelihood ratio, NPE targets the posterior distribution itself, which aligns it directly with the Bayesian inference objective [PM16, GNM19, CBL20].

Training data are obtained by repeated simulation:

$$\boldsymbol{\theta}^{(i)} \sim p(\boldsymbol{\theta}), \quad \mathbf{y}_{0:T}^{(i)} \sim p(\mathbf{y}_{0:T} \mid \boldsymbol{\theta}^{(i)}), \quad i = 1, \dots, N, \quad (2.2)$$

and the conditional density estimator is fitted by maximising the conditional log-likelihood

$$\mathcal{L}_{\text{NPE}}(\phi) = \frac{1}{N} \sum_{i=1}^N \log q_\phi(\boldsymbol{\theta}^{(i)} \mid \mathbf{y}_{0:T}^{(i)}). \quad (2.3)$$

Once trained, inference for a new observation requires only a single forward pass: evaluate or sample from $q_\phi(\boldsymbol{\theta} \mid \mathbf{y}_{0:T}^\circ)$. This is the core amortisation benefit.

2.2.2 Density Approximators

A key design choice in NPE is the family used to represent the conditional density $q_\phi(\boldsymbol{\theta} \mid \mathbf{y}_{0:T})$. For vehicle dynamics, the posterior can be non-Gaussian: parameters interact nonlinearly, different manoeuvre phases excite different parameters to different degrees, and the posterior may be skewed, correlated, or elongated along directions of partial non-identifiability. A restrictive density family (e.g., a Gaussian) would miss exactly this structure.

For this reason, NPE is commonly combined with *normalising flows* or other expressive conditional density models [RM16, PPM17, PNR⁺21]. The observation trajectory is first compressed through an embedding network $h_\psi(\mathbf{y}_{0:T})$, and the resulting embedding conditions the density model. This two-stage design—embedding plus density estimator—is the standard NPE architecture.

2.2.3 Normalising Flows

A normalising flow transforms a simple base variable $\mathbf{z} \sim p_0(\mathbf{z})$ into the target $\boldsymbol{\theta}$ through an invertible differentiable map conditioned on the observation embedding:

$$\boldsymbol{\theta} = f_\phi(\mathbf{z}; \mathbf{y}_{0:T}). \quad (2.4)$$

By the change-of-variables formula, the induced log-density is

$$\log q_\phi(\boldsymbol{\theta} \mid \mathbf{y}_{0:T}) = \log p_0(f_\phi^{-1}(\boldsymbol{\theta}; \mathbf{y}_{0:T})) + \log \left| \det J_{f_\phi^{-1}} \right|. \quad (2.5)$$

To achieve sufficient expressiveness, flow models compose several invertible transformations,

$$f_\phi = f_K \circ \dots \circ f_1, \quad (2.6)$$

with the Jacobian determinant factorising additively over the steps. In autoregressive architectures such as the Masked Autoregressive Flow (MAF), the Jacobian is triangular, making the determinant cheap to compute while the transformation remains expressive [PPM17, PNR⁺21]. Substituting the flow density into the NPE objective yields a straightforward maximum-likelihood training procedure on simulator-generated data.

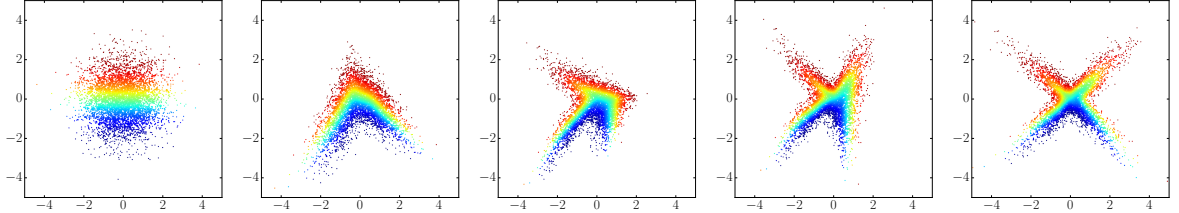


Figure 2.1: A normalising flow transforms a simple Gaussian base distribution (leftmost panel) into a complex target density (rightmost panel) through a chain of invertible differentiable maps. The colour of each point tracks its identity through the chain, making the bijective nature of the transformation visible. Figure reproduced from [PNR⁺21].

Sequential variants of NPE (e.g., APT [GNM19]) refine the simulation distribution across multiple rounds by using proposal distributions informed by earlier posteriors, improving simulation efficiency when the prior is broad relative to the posterior. In this thesis, a single-round amortised NPE is used throughout, since the focus is on comparing methods under matched offline simulation budgets rather than on sequential refinement strategies.

2.3 Score-Based Diffusion Models for Simulation-Based Inference

2.3.1 Score Matching and Diffusion Models

Score-based methods replace normalised density estimation by estimating $\nabla_{\mathbf{z}} \log p(\mathbf{z})$, the score [Hyv05]. Modern score-based generative modelling defines a forward SDE that gradually corrupts the target distribution toward a Gaussian reference, and trains a score network to reverse this process [SSDK⁺21, HJA20]. For SBI, the target is a posterior conditioned on observations, so score-based SBI methods train a *conditional* score network $s_{\psi}(\boldsymbol{\theta}_a, \mathbf{y}_{0:T}, a) \approx \nabla_{\boldsymbol{\theta}_a} \log p_a(\boldsymbol{\theta}_a \mid \mathbf{y}_{0:T})$ [SSLB24], where $\boldsymbol{\theta}_a$ is a noised version of the parameter. The advantage over normalising flows is that no normalised density family is specified explicitly; the trade-off is that inference requires solving a reverse-time SDE numerically rather than a single forward pass. Figure 2.2 illustrates the forward and reverse SDE.

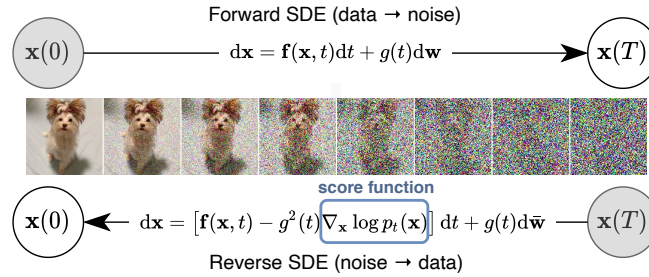


Figure 2.2: The forward SDE (top) gradually corrupts a data sample into Gaussian noise; the reverse SDE (bottom) uses the learned score function $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ to recover the original data distribution. Score-based generative modelling trains a neural network to approximate this score field and then integrates the reverse process to generate new samples. Figure reproduced from [SSDK⁺21].

2.3.2 Neural Posterior Score Estimation

NPSE [SSLB24] applies score-based diffusion to posterior inference. Rather than fitting a normalised conditional density, NPSE trains a score network

$$s_{\psi}(\boldsymbol{\theta}_a, \mathbf{y}_{0:T}, a) \approx \nabla_{\boldsymbol{\theta}_a} \log p_a(\boldsymbol{\theta}_a \mid \mathbf{y}_{0:T}), \quad (2.7)$$

where $p_a(\boldsymbol{\theta}_a \mid \mathbf{y}_{0:T})$ is the posterior after diffusion noise has been applied to the parameter. The true posterior score cannot be evaluated directly, but NPSE replaces the score-matching

objective with an equivalent denoising objective:

$$\mathcal{J}_{\text{post}}^{\text{DSM}}(\psi) = \frac{1}{2} \int_0^A \lambda_a \mathbb{E} \left[\|s_\psi(\boldsymbol{\theta}_a, \mathbf{y}_{0:T}, a) - \nabla_{\boldsymbol{\theta}_a} \log p_{a|0}(\boldsymbol{\theta}_a | \boldsymbol{\theta}_0)\|_2^2 \right] da, \quad (2.8)$$

where the expectation is over simulated pairs $(\boldsymbol{\theta}_0, \mathbf{y}_{0:T})$ and the forward transition $p_{a|0}(\boldsymbol{\theta}_a | \boldsymbol{\theta}_0)$. The denoising target is fully determined by the chosen forward diffusion and is therefore analytically available. At inference time, the learned score drives a reverse diffusion process to generate posterior samples $\boldsymbol{\theta} \sim p(\boldsymbol{\theta} | \mathbf{y}_{0:T}^o)$.

NPSE is the natural score-based counterpart to NPE: it conditions on the full trajectory globally but replaces the normalised density with a score field, which can represent more complex posterior geometry [ABK⁺25]. The trade-off is iterative sampling rather than a single forward pass.

2.3.3 Factorized Neural Posterior Score Estimation

FNPSE [GTFM24] is designed for Markovian simulators. Rather than conditioning a score network on the full trajectory globally, FNPSE learns local score contributions from individual state transitions and aggregates them into a global posterior score.

The key factorisation result follows from the Markovian structure of the simulator,

$$p(\mathbf{x}_{0:T} | \boldsymbol{\theta}) = p(\mathbf{x}_0) \prod_{t=0}^{T-1} p(\mathbf{x}_{t+1} | \mathbf{x}_t, \boldsymbol{\theta}). \quad (2.9)$$

Under an appropriate proposal distribution $\tilde{p}(\mathbf{x}_t)$ that is independent of $\boldsymbol{\theta}$, the global posterior score at diffusion time $a = 0$ decomposes as

$$\nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta} | \mathbf{x}_{0:T}^o) = \sum_{t=0}^{T-1} \nabla_{\boldsymbol{\theta}} \log \tilde{p}(\boldsymbol{\theta} | \mathbf{x}_t^o, \mathbf{x}_{t+1}^o) - (T-1) \nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{\theta}). \quad (2.10)$$

Each local term is trained with a denoising objective conditioned only on a transition pair $(\mathbf{x}_t, \mathbf{x}_{t+1})$, which means training requires only one-step simulations rather than full trajectories—the main simulation efficiency advantage.

For noisy diffusion states ($a > 0$), the exact decomposition no longer holds and FNPSE uses an

approximate score-composition rule [GPM23, LCG⁺25]:

$$\nabla_{\theta_a} \log p(\theta_a \mid \mathbf{x}_{0:T}) \approx \Lambda(\theta_a)^{-1} \left(\sum_{t=0}^{T-1} \Sigma_{a,t,t+1}^{-1} s_\phi(\theta_a, \mathbf{x}_t, \mathbf{x}_{t+1}, a) + (1-T) \Sigma_a^{-1} \nabla_{\theta_a} \log p(\theta_a) \right), \quad (2.11)$$

where Σ_a and $\Sigma_{a,t,t+1}$ are covariance terms that are approximated in practice. The practical performance therefore depends on both the quality of the local score network and the validity of the composition approximation.

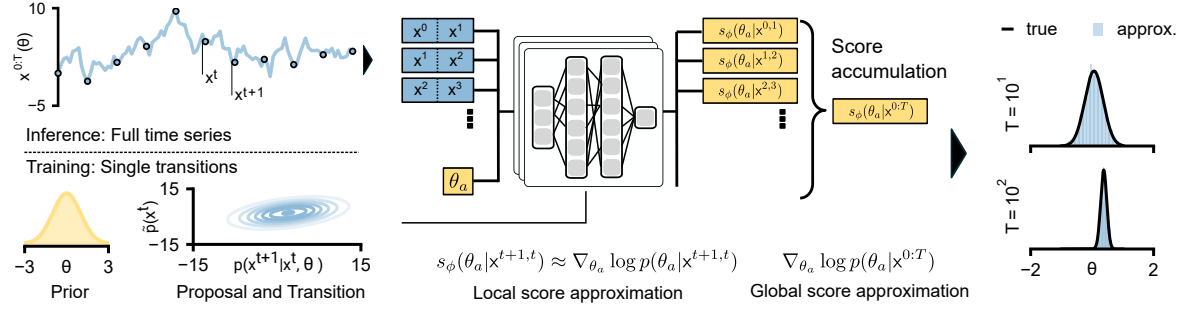


Figure 2.3: Overview of FNPSE. *Left:* a full time-series observation (top) is decomposed into overlapping transition pairs $(\mathbf{x}_t, \mathbf{x}_{t+1})$ for training (bottom), using a proposal and transition model. *Centre:* a shared score network s_ϕ processes each transition pair and a noised parameter θ_a to produce a local score approximation. *Right:* the local scores are accumulated to form the global posterior score, which concentrates as the trajectory length T increases. Figure reproduced from [GTFM24].

The strength of FNPSE is its simulation efficiency: training on short windows scales well to long sequences. The fundamental limitation is that some parameter information accumulates across multi-step temporal patterns, which a purely local decomposition cannot capture. This tension is central to the vehicle-dynamics benchmark in this thesis. Figure 2.3 illustrates this decomposition.

2.3.4 Simformer

Simformer [GDW⁺24] takes a more general approach. Rather than targeting only the posterior $p(\theta \mid \mathbf{y}_{0:T})$, it is trained on the full joint distribution

$$p(\theta, \mathbf{y}_{0:T}), \quad (2.12)$$

and can then evaluate different conditional distributions of that joint by conditioning on different variable subsets at inference time.

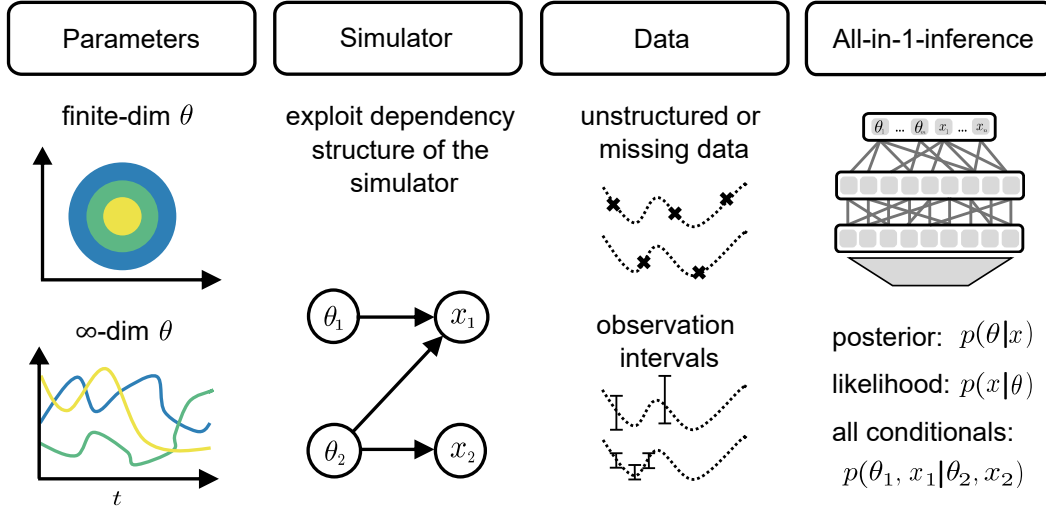


Figure 2.4: High-level overview of Simformer. A single trained network handles finite- or infinite-dimensional parameters, exploits the dependency structure of the simulator, and supports unstructured or missing observations-yielding posterior, likelihood, and arbitrary conditional queries without retraining. Figure reproduced from [GDW⁺24].

The method combines score-based diffusion with a transformer architecture. All variables-parameters and observations-are tokenized and passed to a score network that is trained under varying conditioning masks. This allows the same network to perform posterior inference, likelihood evaluation, or prior predictive sampling without retraining. Figures 2.4 and 2.5 illustrate the high-level concept and tokenization respectively.

Structural dependencies in the simulator can be encoded in the transformer’s attention pattern, enabling the model to handle multivariate trajectories where different channels carry different information about the parameters [GDW⁺24].

The trade-off is that modelling the full joint is a harder problem than targeting the posterior directly, and transformer inference on long sequences carries a higher computational cost. For this thesis, Simformer serves as a modern diffusion-based baseline that combines score-based modelling with explicit structural encoding, and its practical performance relative to the other three methods is an empirical question central to the benchmark.

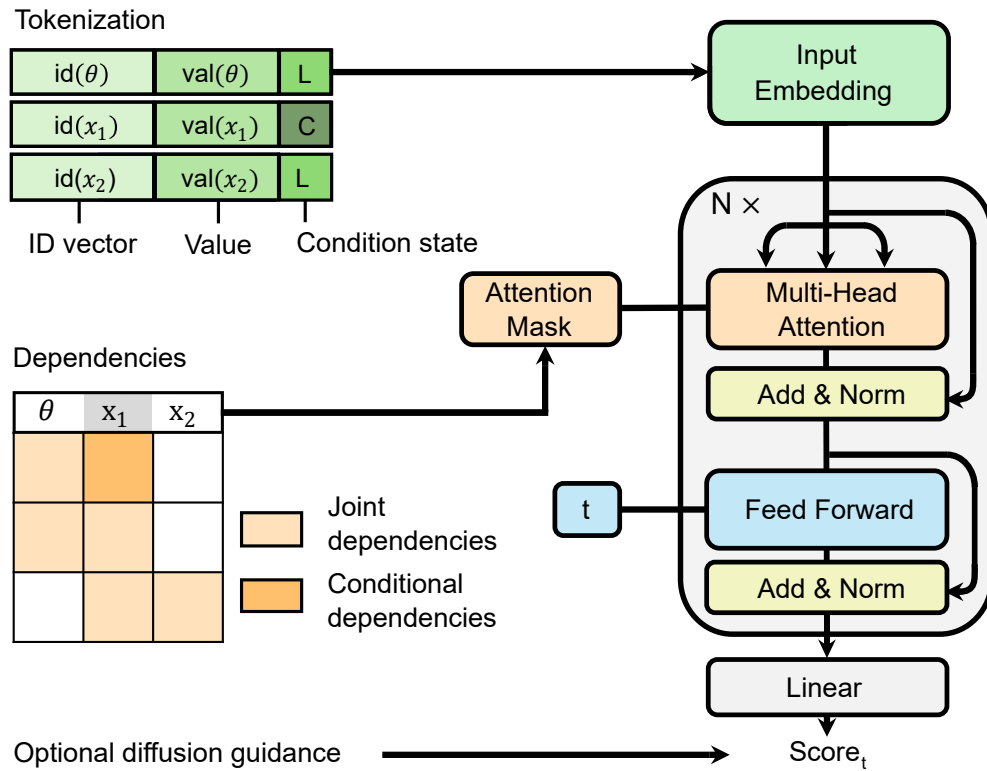


Figure 2.5: Simformer architecture. Each variable (parameter θ or observation x_i) is tokenized with an ID vector, its value, and a condition flag (L: latent to be inferred; C: conditioned on). An attention mask encodes which token pairs share joint dependencies (light) or are linked only by conditioning (dark). A diffusion-time-conditioned transformer then outputs the score at each latent token. Changing the condition flags and mask at inference time is sufficient to switch between different query types. Figure reproduced from [GDW⁺24].

2.4 Scope and Positioning of This Thesis

The neural SBI literature has expanded substantially since 2016. NPE and related methods provide flexible posterior approximations for simulator-based models [PM16, GNM19, CBL20]. Score-based approaches have more recently extended this landscape [SSLB24, ABK⁺25], and structure-aware methods such as FNPSE and Simformer exploit temporal and joint-distribution properties of the simulator more explicitly [GTFM24, GDW⁺24]. Each of these methods has been evaluated in its originating paper and on shared SBI benchmarks [LBG⁺21, ZMSDH25], but those benchmarks are deliberately abstract.

This thesis is positioned in the gap between method development and engineering deployment. It is not the case that no benchmarking has been done; it is the case that the existing benchmarking does not yet answer the practical question that motivates this work: *for static parameter inference in a Markovian engineering simulator with multivariate time-series observations and physically interpretable parameters, which of the recent amortised SBI methods is currently the most useful, and where do their failure modes lie?* Two specific gaps in the existing literature inform the contribution.

First, an applied benchmark for vehicle dynamics is missing. The original FNPSE and Simformer evaluations were conducted on abstract benchmarks where structural assumptions can be assessed in isolation. Whether the structural advantages claimed in those settings transfer to a realistic engineering simulator-where parameter information is distributed unevenly across time and channels, observations are high-dimensional and partially redundant, and the simulator carries domain-specific nonlinearities (Pacejka tire forces, wheel-level dynamics)-is an empirical question that has not been answered.

Second, evaluations to date are evaluation-incomplete: posterior point accuracy is reported alongside coverage in some papers but rarely all four of (i) posterior-recovery error, (ii) calibration as measured by coverage and coverage-curve fidelity, (iii) prior-relative informativeness, and (iv) posterior-predictive consistency under matched simulation budgets and matched architectures. As [HDR⁺22] argues, a posterior approximation can look accurate on standard visual checks while being poorly calibrated; a comparison that addresses this requires the full multi-metric protocol.

The contribution of this thesis is therefore an applied, multi-metric, budget-matched comparison of NPE, NPSE, FNPSE, and Simformer on a vehicle-dynamics parameter-estimation problem with an additional real-data robustness check. The thesis does not propose a new SBI method (Section 1.3); its contribution is empirical and methodological in the benchmarking sense. The findings are intended to inform method selection for similar engineering applications and to surface the failure modes that abstract benchmarks may obscure.

3 Problem Setting and Vehicle Case Study

This chapter defines the concrete inference problem studied in this thesis. After introducing the general problem formulation in Section 3.1, Section 3.2 describes the two-track vehicle model that serves as the simulator, including the tire force model, the equations of motion, and the numerical integration scheme. Section 3.3 specifies the state, observation, and control representations used throughout the experimental pipeline. Section 3.4 describes how driving scenarios are generated from randomised manoeuvre blocks. Section 3.5 introduces the calibrated noise model that bridges the gap between clean simulation and real sensor data. Section 3.6 presents the real measurement data used for robustness evaluation. Finally, Section 3.7 identifies the static parameters of interest and defines the prior distributions used for inference.

3.1 Problem Formulation

The inference problem studied in this thesis concerns the recovery of static physical parameters from multivariate vehicle trajectories. Let $\theta \in \mathbb{R}^{d_\theta}$ denote the unknown static model parameters, $\mathbf{x}_t \in \mathbb{R}^{d_x}$ the latent state of the vehicle at discrete time step t , and $\mathbf{y}_t \in \mathbb{R}^{d_y}$ the corresponding observation vector. The vehicle is driven by known control inputs $\mathbf{u}_t \in \mathbb{R}^{d_u}$ at each time step.

The simulator defines a Markovian transition model

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t, \theta), \quad (3.1)$$

where f is the deterministic vehicle dynamics model (RK4 integration of the two-track equations; see Section 3.2). Process noise is not injected into the state during numerical integration; instead, slowly varying perturbations are added to the simulated output trajectory as described in Section 3.5. The observation model maps the latent state and parameters to a measurement vector:

$$\mathbf{y}_t = h(\mathbf{x}_t, \mathbf{u}_t, \theta) + \mathbf{w}_t + \varepsilon_t, \quad (3.2)$$

where h is the measurement equation, $\varepsilon_t \sim \mathcal{N}(\mathbf{0}, \text{diag}(\sigma^2))$ is per-channel i.i.d. Gaussian observation noise, and $\mathbf{w}_t$ represents the slowly varying output perturbations (process noise on the trajectory, not the state) calibrated in Section 3.5. The inference target is the posterior $p(\theta \mid \mathbf{y}_{0:T})$ introduced in Eq. 1.1, where $p(\theta)$ is a prior over the unknown parameters. Because

the likelihood $p(\mathbf{y}_{0:T} \mid \boldsymbol{\theta})$ is induced by a nonlinear simulator with stochastic initial states and noise injection, it is intractable in closed form. SBI circumvents this by generating simulated pairs $(\boldsymbol{\theta}^{(i)}, \mathbf{y}_{0:T}^{(i)})$ from the prior and the simulator, and training a neural estimator on these pairs, as described in Chapter 2.

In the present case study, the simulator is a two-track vehicle dynamics model, the observations are multichannel time series at a fixed sampling interval $\Delta t = 0.01$ s, and the parameters of interest are physically interpretable quantities: the tire–road friction coefficient, the aerodynamic drag coefficient, and the vehicle mass.

3.2 Two-Track Vehicle Model

The simulator used throughout this thesis is a planar two-track vehicle dynamics model. Unlike single-track (bicycle) models, a two-track model treats the four wheels individually, which allows it to capture individual wheel dynamics, left–right tire-force differences, and the coupling between longitudinal and lateral dynamics during combined steering and braking manoeuvres. Dynamic load transfer due to longitudinal or lateral acceleration is not modelled in the primary configuration used for inference; this simplification is a known source of residual sim-to-real mismatch and is discussed further in Section 5.3.1. This level of fidelity is relevant for the present inference problem: the parameters of interest—friction, mass, and drag—manifest their influence most clearly through individual wheel dynamics, differential tire forces, and longitudinal–lateral force coupling, all of which are resolved at the wheel level in the two-track model.

Geometry and coordinate conventions. The model uses a body-fixed coordinate frame centred at the CoG of the vehicle, with the x -axis pointing forward and the y -axis pointing to the left. The four wheels are located at offsets determined by the track widths w_f and w_r (front and rear) and the axle distances l_f and l_r (from the CoG to the front and rear axles, respectively). The wheelbase is $L = l_f + l_r$. The numerical values used in this thesis are listed in Table 3.1.

Magic Tire Formula. Tire forces are computed using the Pacejka MTF [Pac12]. This semi-empirical model expresses the longitudinal and lateral tire forces as functions of the combined slip, the normal load, and the friction coefficient. For each tire i , the longitudinal and lateral force components are given by

$$F_{\{x,y\},i} = D_{\{x,y\}} \sin\left(C_{\{x,y\}} \arctan\left((1 - E_{\{x,y\}}) B_{\{x,y\}} s_i + E_{\{x,y\}} \arctan(B_{\{x,y\}} s_i)\right)\right), \quad (3.3)$$

Table 3.1: Fixed geometric parameters of the two-track vehicle model.

Symbol	Description	Value	Unit
l_f	Distance CoG to front axle	1.16	m
l_r	Distance CoG to rear axle	1.47	m
w_f	Front track width	1.543	m
w_r	Rear track width	1.513	m
r_w	Effective tire radius	0.3116	m
h_{CoG}	Height of center of gravity	0.2684	m
A_x	Frontal body surface area	2.19	m ²
g	Gravitational acceleration	9.81	m/s ²
ρ_{air}	Air density	1.225	kg/m ³

where $s_i = \sqrt{s_{x,i}^2 + s_{y,i}^2}$ is the combined slip magnitude and the direction of the force components is determined by the slip ratio:

$$F_{x,i}^{\text{tire}} = F_{\{x\}}(s_i, F_{z,i}, \mu) \cdot \frac{s_{x,i}}{s_i}, \quad F_{y,i}^{\text{tire}} = F_{\{y\}}(s_i, F_{z,i}, \mu) \cdot \frac{s_{y,i}}{s_i}. \quad (3.4)$$

The stiffness factor B , peak factor D , shape factor C , and curvature factor E are determined from the tire parameters. The peak factor is

$$D_{\{x,y\}} = \mu \cdot F_{z,i}, \quad (3.5)$$

so the friction coefficient μ directly scales the maximum achievable tire force. The cornering stiffness at the origin is

$$C_{F,\{x,y\}} = c_{1,\{x,y\}} \cdot \sin\left(2 \arctan\left(\frac{F_{z,i}}{c_{2,\{x,y\}}}\right)\right), \quad (3.6)$$

and the stiffness factor is

$$B_{\{x,y\}} = \frac{C_{F,\{x,y\}}}{C_{\{x,y\}} \cdot D_{\{x,y\}}}. \quad (3.7)$$

The MTF parameters used in this thesis are listed in Table 3.2.

Tire velocities and slip. To evaluate the MTF, the longitudinal and lateral velocity of each tire must be expressed in the respective tire coordinate frame. For each wheel, the velocity at the contact patch is computed from the body velocities (v_x, v_y) and the yaw rate $\dot{\psi}$ by accounting

Table 3.2: Pacejka Magic Tire Formula parameters.

Symbol	Description	Value
$c_{1,x}$	Longitudinal stiffness coefficient 1	2.5×10^7
$c_{2,x}$	Longitudinal stiffness coefficient 2	3.6×10^6
C_x	Longitudinal shape factor	1.42
E_x	Longitudinal curvature factor	-9.75
$c_{1,y}$	Lateral stiffness coefficient 1	1.9×10^6
$c_{2,y}$	Lateral stiffness coefficient 2	1.35×10^5
C_y	Lateral shape factor	1.9
E_y	Lateral curvature factor	0.52

for the geometric offsets and the steering angle. For the front-left tire, for example,

$$\begin{pmatrix} v_{x,FL}^{\text{tire}} \\ v_{y,FL}^{\text{tire}} \end{pmatrix} = R(-\delta) \begin{pmatrix} v_x - \dot{\psi} w_f/2 \\ v_y + \dot{\psi} l_f \end{pmatrix}, \quad (3.8)$$

where $R(\cdot)$ is a two-dimensional rotation matrix and δ is the front steering angle. The rear wheels are unsteered ($\delta = 0$).

The longitudinal and lateral slip components for each tire are then

$$s_{x,i} = \frac{\omega_i \cdot r_w - v_{x,i}^{\text{tire}}}{\max(|v_{x,i}^{\text{tire}}|, v_{\min})}, \quad s_{y,i} = \frac{-v_{y,i}^{\text{tire}}}{\max(|v_{x,i}^{\text{tire}}|, v_{\min})}, \quad (3.9)$$

where ω_i is the angular velocity of wheel i and $v_{\min} = 1$ m/s is a regularisation threshold that prevents division by zero at near-standstill conditions.

Static tire loads. The normal force on each tire is computed from the static weight distribution assuming a flat road:

$$F_{z,FL} = F_{z,FR} = \frac{g}{2} \cdot \frac{l_r}{L} \cdot m, \quad F_{z,RL} = F_{z,RR} = \frac{g}{2} \cdot \frac{l_f}{L} \cdot m, \quad (3.10)$$

where m is the vehicle mass. Dynamic load transfer effects due to longitudinal or lateral acceleration are not modelled in the primary configuration used for inference, keeping the model tractable while retaining the dominant mass-dependent static load effect.

Equations of motion. The translational dynamics in the body frame are governed by

$$\dot{v}_x = \frac{1}{m} \left(\sum_{i=1}^4 F_{x,i} - F_{\text{air}} \right) + \dot{\psi} v_y, \quad \dot{v}_y = \frac{1}{m} \sum_{i=1}^4 F_{y,i} - \dot{\psi} v_x, \quad (3.11)$$

where $F_{x,i}$ and $F_{y,i}$ are the tire forces rotated into the body frame, and the aerodynamic drag force is

$$F_{\text{air}} = \frac{1}{2} c_d \rho_{\text{air}} A_x v_x^2 \text{sgn}(v_x). \quad (3.12)$$

The yaw dynamics follow from the moment balance about the vertical axis through the CoG:

$$\ddot{\psi} = \frac{1}{I_z} \left((F_{y,\text{FL}} + F_{y,\text{FR}}) l_f - (F_{y,\text{RL}} + F_{y,\text{RR}}) l_r + (F_{x,\text{FL}} - F_{x,\text{FR}}) \frac{w_f}{2} + (F_{x,\text{RL}} - F_{x,\text{RR}}) \frac{w_r}{2} \right), \quad (3.13)$$

where I_z is the yaw moment of inertia. The position in the global frame is obtained by rotating the body velocities:

$$\begin{pmatrix} \dot{X} \\ \dot{Y} \end{pmatrix} = R(\psi)^\top \begin{pmatrix} v_x \\ v_y \end{pmatrix}. \quad (3.14)$$

Each wheel also has rotational dynamics. The angular acceleration of wheel i is

$$\dot{\omega}_i = \frac{1}{J_i} (T_{\text{drive},i} - F_{x,i}^{\text{tire}} r_w - T_{\text{roll},i} - T_{\text{brake},i} \text{sgn}(\omega_i)), \quad (3.15)$$

where $T_{\text{drive},i}$ is the drive torque (including gear transmission), $T_{\text{brake},i}$ is the braking torque, and $T_{\text{roll},i}$ is the rolling resistance torque modelled as

$$T_{\text{roll},i} = F_{z,i} (C_{\text{roll},1} + C_{\text{roll},2} |\omega_i|) \text{sgn}(\omega_i). \quad (3.16)$$

The effective inertia J_i accounts for both the tire inertia and the engine inertia reflected through the gear ratio:

$$J_i = I_{\text{tire}} + \frac{1}{2} I_{\text{engine}} \cdot \gamma_i^2, \quad (3.17)$$

where γ_i is the gear transmission ratio (nonzero only for driven wheels). The remaining model parameters are listed in Table 3.3.

Numerical integration. The continuous-time equations of motion are integrated using an RK4 (fourth-order Runge–Kutta) scheme. Because the tire force model is stiff-small changes in slip can produce large changes in force-the nominal simulation time step $\Delta t = 0.01$ s is subdivided into $n_{\text{sub}} = 15$ sub-steps. Each macro time step therefore involves 15 RK4 evaluations, each with an effective sub-step size of $\Delta t/15 \approx 0.667$ ms. This substep count

Table 3.3: Fixed dynamic parameters of the vehicle model.

Symbol	Description	Value	Unit
I_z	Yaw moment of inertia	2066	kg m^2
I_{tire}	Tire rotational inertia	28.6	kg m^2
I_{engine}	Engine rotational inertia	0.197	kg m^2
$C_{\text{roll},1}$	Rolling resistance coeff. 1	0.0083	–
$C_{\text{roll},2}$	Rolling resistance coeff. 2	0.0005	s/rad

was chosen to maintain numerical stability across the full operating range of the simulator, including emergency braking and high-slip conditions.

3.3 State, Observation, and Control Vectors

State vector. The full state of the vehicle model at each time step is a 10-dimensional vector:

$$\mathbf{x}_t = \left(X \ Y \ \psi \ \dot{\psi} \ v_x \ v_y \ \omega_{\text{FL}} \ \omega_{\text{FR}} \ \omega_{\text{RL}} \ \omega_{\text{RR}} \right)^\top \in \mathbb{R}^{10}, \quad (3.18)$$

where (X, Y) is the global position of the CoG, ψ is the yaw angle, $\dot{\psi}$ is the yaw rate, (v_x, v_y) are the longitudinal and lateral body velocities, and $\omega_{\text{FL}}, \dots, \omega_{\text{RR}}$ are the angular velocities of the four wheels in the order front-left, front-right, rear-left, rear-right.

Observation vector. The observation (measurement) equation produces a 9-dimensional output at each time step:

$$\mathbf{y}_t = \left(\dot{\psi} \ \dot{X} \ \dot{Y} \ \dot{v}_x \ \dot{v}_y \ \omega_{\text{FL}} \ \omega_{\text{FR}} \ \omega_{\text{RL}} \ \omega_{\text{RR}} \right)^\top \in \mathbb{R}^9. \quad (3.19)$$

The first channel is the yaw rate, directly available from the state. Channels two and three are the global-frame velocities, obtained by rotating the body-frame velocities by the yaw angle. Channels four and five are the body-frame accelerations $(\dot{v}_x, \dot{v}_y)$, which are derived quantities computed from the equations of motion (Eq. 3.11) and thus depend on the tire forces, aerodynamic drag, and vehicle mass. The last four channels are the wheel angular velocities.

This observation vector is designed to resemble the sensor suite available on a real test vehicle: an inertial measurement unit provides yaw rate and body accelerations, a GPS/INS combination provides velocity information, and wheel speed sensors provide tire angular velocities. Notably,

the global position (X, Y) and the yaw angle ψ are *not* included in the observation vector. Including absolute positions would make the observation space grow with the trajectory without adding useful information for static parameter inference, while the yaw angle is replaced by the yaw rate, which is the directly measured quantity on real vehicles.

Control vector. The control input at each time step is a 4-dimensional vector:

$$\mathbf{u}_t = \begin{pmatrix} \delta_v & T_{\text{engine}} & T_{\text{brake}} & \gamma \end{pmatrix}^\top \in \mathbb{R}^4, \quad (3.20)$$

where δ_v is the front steering angle (in radians), T_{engine} is the total engine drive torque (split equally between the two front wheels), T_{brake} is the braking torque (applied equally to all four wheels), and γ is the gear transmission ratio. These controls are treated as known inputs to the simulator and are concatenated to the observation channels when forming the neural network input, yielding a combined input dimension of $d_y + d_u = 9 + 4 = 13$ channels per time step.

3.4 Driving Scenario Generation

A key design choice in any SBI pipeline is how the simulator is exercised during data generation. In vehicle dynamics, the information content of a trajectory depends strongly on the driving scenario: a straight constant-speed segment reveals little about lateral dynamics or friction, while a combined braking-and-steering manoeuvre excites multiple parameters simultaneously.

Block-based manoeuvre construction. Each simulated trajectory is assembled from a randomised sequence of manoeuvre blocks. Each block has a randomly sampled duration and specifies the time profiles for all four control channels. The block types used in this thesis are:

- **Acceleration blocks:** Engine torque sampled from $[300, 500]$ Nm (scaled by a configurable factor) with a random duty cycle, combined with sinusoidal steering at amplitudes of 4–12 degrees and frequencies of 0.5–1.5 Hz. Duration: 1.5–4.0 s.
- **Braking blocks:** Brake torque sampled from $[100, 250]$ Nm with a duty cycle of 0.2–0.8, no steering. Duration: 0.6–3.0 s.
- **Coast blocks:** No drive or brake torque, with a small constant steering offset sampled from $[-5, 5]$ degrees. Duration: 0.8–2.5 s.
- **Emergency braking blocks:** High brake torque sampled from $[350, 600]$ Nm with duty cycles of 0.7–1.0. Duration: 1.0–4.0 s.

- **Brake-and-steer blocks:** Combined braking ($[200, 500]$ Nm) with sinusoidal evasive steering at amplitudes of 8–25 degrees. Duration: 1.5–3.5 s.
- **Coast-with-disturbance blocks:** Coasting with high-frequency steering input (1–3 Hz, 0.5–3 degrees amplitude), simulating road disturbances. Duration: 1.0–3.0 s.

A typical trajectory consists of 12 acceleration–coast pairs interleaved with braking blocks. The blocks are assembled in random order, and optional ramp transitions smooth the control signals at block boundaries. The full generated control sequence has a maximum length of $T = 3000$ time steps (30 s at $\Delta t = 0.01$ s). The benchmark evaluates segment lengths $T_{\text{seg}} \in \{1000, 2000, 3000\}$; shorter segments use proportionally fewer manoeuvre blocks.

The high-excitation block types (emergency braking, brake-and-steer, coast-with-disturbance) are included with a configurable fraction to push the simulator toward more informative operating regimes. In the default configuration, approximately 40% of braking blocks are replaced by high-deceleration variants (emergency braking at 350–600 Nm and brake-and-steer combinations at 200–500 Nm). This design ensures that the training data cover a range of dynamic conditions, from steady-state driving to high-slip transient manoeuvres, which is important for learning posteriors that remain informative across different parts of the parameter space.

Initial state sampling. The initial longitudinal velocity $v_{x,0}$ is sampled uniformly from a configurable range centred at 11 m/s (≈ 40 km/h) with a half-width of 10 m/s. The initial lateral velocity $v_{y,0}$ is drawn from $[-1, 1]$ m/s, the initial yaw angle from $[-0.05, 0.05]$ rad, and the initial yaw rate from $[-0.2, 0.2]$ rad/s. The initial wheel angular velocities are set to be kinematically consistent with the longitudinal speed, with 5% Gaussian noise added to each wheel independently. The global position is initialised at the origin.

This sampling regime ensures that each simulated trajectory starts from a different but dynamically consistent vehicle state, so the trained posterior estimator learns to handle a range of initial conditions rather than overfitting to a single operating point.

3.5 Noise Model

A purely deterministic simulator produces trajectories that are smoother and more regular than real measurement data. To make the simulated training data more representative of the conditions under which the inference methods will eventually be applied, a two-layer noise model is injected into the simulation pipeline. The noise standard deviations were calibrated from seven real vehicle measurement recordings (Section 3.6) using Savitzky–Golay residual

analysis.

Observation noise. The first layer adds independent, identically distributed Gaussian noise to each observation channel at each time step:

$$\tilde{y}_{t,j} = y_{t,j} + \varepsilon_{t,j}, \quad \varepsilon_{t,j} \sim \mathcal{N}(0, \sigma_j^2), \quad (3.21)$$

where σ_j is the calibrated standard deviation for channel j . This mimics sensor measurement noise. The calibrated values reflect the characteristics of typical automotive-grade sensors: low noise on the yaw rate channel ($\sigma \approx 4.6 \times 10^{-5}$ rad/s), moderate noise on global-frame velocities ($\sigma \approx 0.005$ – 0.009 m/s), higher noise on body-frame accelerations ($\sigma \approx 0.43$ – 1.05 m/s²), and moderate noise on wheel speed channels ($\sigma \approx 0.82$ rad/s).

Process noise. The second layer adds slowly varying perturbations to each dynamic channel, modelling unobserved disturbances such as road surface variations, tire temperature drift, or aerodynamic turbulence that are not captured by the deterministic vehicle model. These are implemented as random walks with per-time-step increments:

$$w_{t,j} = w_{t-1,j} + \eta_{t,j}, \quad \eta_{t,j} \sim \mathcal{N}(0, \sigma_{\text{proc},j}^2), \quad (3.22)$$

where the cumulative walk $w_{t,j}$ is smoothed with a moving-average kernel of width 5 before being added to the trajectory. This produces low-frequency bias-like drifts rather than white noise, which better matches the character of real model mismatch.

Both noise layers are scaled by global multipliers that default to 1.0 (calibrated level) but can be adjusted to study the sensitivity of inference methods to different noise regimes.

3.6 Real Measurement Data

In addition to simulated trajectories, the experimental pipeline includes real vehicle measurement data as a robustness and plausibility check. The measurements come from a test campaign conducted at the Jeversen proving ground in December 2021 and October 2022. Seven recordings are available, covering three road surface types:

- **Asphalt** (2 recordings): Full braking on dry asphalt, expected to correspond to high friction ($\mu \approx 1.0$ – 1.2).
- **Concrete** (2 recordings): Full braking on concrete, expected to have slightly lower friction than asphalt.

- **Basalt** (2 recordings): Full braking on polished basalt tiles, expected to correspond to noticeably lower friction ($\mu \approx 0.5\text{--}0.7$).
- **Mixed** (1 recording): A longer manoeuvre from October 2022 with varied driving conditions.

The real data serve a different purpose than the simulated data. In simulation, the true parameter values are known, so posterior quality can be assessed directly. For real data, the true posterior is unknown. Instead, the real recordings are used to check whether the posteriors produced by trained inference methods are physically plausible—for example, whether the inferred friction level is consistent with the known road surface—and whether posterior predictive trajectories reproduce the qualitative character of the measured signals. This distinction is important: the real-data evaluation is a stress test of robustness, not a ground-truth benchmark.

The measurement channels are mapped to the observation channels of the simulator as closely as possible, including yaw rate, global-frame velocities, body-frame accelerations, and wheel speeds. Where necessary, coordinate transformations and unit conversions are applied to align the data with the simulator conventions. The same control channels (steering angle, engine torque, braking torque, gear ratio) are extracted from the measurement recordings and treated as known inputs during inference.

3.7 Parameters of Interest and Prior Distributions

3.7.1 Parameter Selection

The inference target in this thesis comprises three static physical parameters:

1. **Tire–road friction coefficient μ** : Scales the peak force in the Magic Tire Formula (Eq. 3.3). Changes in friction directly affect the maximum achievable tire forces and thereby the transient lateral and longitudinal dynamics. This parameter is the most influential for vehicle behaviour under demanding driving conditions and cannot be measured directly without dedicated tire test equipment.
2. **Aerodynamic drag coefficient c_d** : Enters the aerodynamic drag force (Eq. 3.12). Its effect grows quadratically with speed and thus becomes most visible during high-speed segments. At lower speeds, the drag force is small relative to tire forces, making c_d harder to identify.
3. **Vehicle mass m** : Appears in the translational equations of motion (Eq. 3.11) and in the static tire loads. Changes in mass affect the inertial response to all forces and also shift the normal load distribution, which in turn influences the tire force magnitudes.

All remaining model parameters (tire model coefficients, inertia values, rolling resistance, geometric quantities) are held fixed at the values listed in Tables 3.1–3.3. This reflects the practical assumption that these quantities are specified by manufacturer data sheets or calibrated separately, and that the main uncertainties in a real application lie in friction (which changes with road conditions), mass (which changes with payload), and drag (which may vary with vehicle configuration or wind conditions).

The parameter vector for inference is therefore

$$\boldsymbol{\theta} = \begin{pmatrix} \mu & c_d & m \end{pmatrix}^\top \in \mathbb{R}^3. \quad (3.23)$$

The experimental pipeline also supports inference on subsets of this vector (for example, friction only, or friction and mass), with the remaining parameters fixed at their default values.

3.7.2 Prior Distributions

The prior distribution over $\boldsymbol{\theta}$ is a factorised Gaussian, with each parameter treated as independent:

$$p(\boldsymbol{\theta}) = \prod_{j=1}^{d_\theta} \mathcal{N}\left(\theta_j \mid \mu_j^{\text{prior}}, (\sigma_j^{\text{prior}})^2\right), \quad (3.24)$$

where for each parameter the mean is set to the midpoint of the physically plausible range, and the standard deviation is chosen as one-sixth of the range width. This ensures approximate three-sigma coverage of the specified bounds. The prior bounds and resulting Gaussian parameters are listed in Table 3.4.

Table 3.4: Design bounds and derived Gaussian prior parameters for each inferred quantity. The prior is a factorised Gaussian (unbounded); the lower/upper bounds define the approximate $\pm 3\sigma$ design range used to set the prior mean and standard deviation. Values outside these bounds can in principle be sampled but have very low prior probability.

Parameter	Symbol	Lower	Upper	μ^{prior}	σ^{prior}	Unit
Friction	μ	0.50	1.50	1.00	0.167	–
Drag coeff.	c_d	0.05	0.60	0.325	0.092	–
Mass	m	1700	2200	1950	83.3	kg

The choice of a Gaussian prior rather than a uniform prior is motivated by two considerations. First, a Gaussian prior is differentiable everywhere, which is advantageous for score-based

methods that require the prior score $\nabla_{\theta} \log p(\theta)$. Second, the Gaussian shape assigns lower prior probability to extreme parameter combinations, which reflects the engineering expectation that tail values (e.g., friction below 0.5 on ice or mass above 2200 kg) occur less frequently than values near the prior mean.

3.7.3 Normalisation

All parameters and observations are normalised to zero mean and unit variance before being passed to the neural networks. This normalisation uses a custom z-scoring scheme based on the empirical statistics of the training data, ensuring consistent treatment across all four inference methods.

4 Evaluated Methods

This chapter describes how each of the four inference methods introduced in Chapter 2 is instantiated within the common experimental pipeline defined in Chapter 3. All four methods share the same simulator, the same prior, the same normalisation scheme, and the same dataset, so that performance differences in Chapter 5 can be attributed to the inference approach itself rather than to inconsistent experimental configurations.

The following paragraphs describe the common components shared across methods. Sections 4.1–4.4 then present the implementation of each method in turn.

Data generation. All methods are trained on simulated parameter–observation pairs $(\boldsymbol{\theta}^{(i)}, \mathbf{y}_{0:T}^{(i)})$ generated from the prior and the vehicle simulator described in Chapter 3. Each experiment uses a trajectory segment length $T_{\text{seg}} \in \{1000, 2000, 3000\}$ as defined by the benchmark grid (Section 5.1.2). The maximum segment length is 3000 time steps, corresponding to 30 s at $\Delta t = 0.01$ s. The simulation-step budget B is held fixed across methods; the resulting number of training trajectories is $N_{\text{sim}} = B/T_{\text{seg}}$, so shorter segments yield more trajectories at the same budget. Each trajectory has 9 observation channels and 4 control channels concatenated into a 13-dimensional input per time step. FNPSE is an exception: it generates its own training data internally using short-window transitions rather than full trajectories (Section 4.3).

Normalisation. All inputs and outputs are z-score normalised as described in Section 3.7. The built-in z-scoring of the `sbi` library is disabled to ensure identical treatment across methods.

Held-out test set. A separate test dataset is generated with parameters drawn from the tail region of the prior (beyond the central quantiles) combined with high-excitation driving conditions. This creates an out-of-distribution evaluation regime that tests generalisation to tail regions of the prior rather than interpolation within the training distribution.

Evaluation. All methods produce posterior samples $\boldsymbol{\theta}^{(s)} \sim q(\boldsymbol{\theta} \mid \mathbf{y}_{0:T}^o)$, either by direct sampling (NPE) or by solving a reverse diffusion process (NPSE, FNPSE, Simformer). The evaluation metrics—including posterior predictive checks, calibration diagnostics, and Wasserstein distances—are applied identically to all methods.

4.1 NPE Baseline

NPE serves as the baseline method. It learns a conditional normalising flow $q_\phi(\boldsymbol{\theta} \mid \mathbf{y}_{0:T})$ that directly approximates the posterior density, as described in Section 2.2. The implementation uses the `sbi` library [TCBD⁺20, BDG⁺25] and is structured into three components: an embedding network that compresses the trajectory into a fixed-length summary, a MAF that models the posterior density conditioned on that summary, and a training procedure that maximises the conditional log-likelihood.

4.1.1 Embedding Network

Because the observation trajectory $\mathbf{y}_{0:T} \in \mathbb{R}^{T \times 13}$ is a long multivariate sequence (up to $T_{\text{seg}} = 3000$ time steps, 13 channels), it must be compressed into a fixed-dimensional summary vector before it can condition the normalising flow. This compression is performed by an embedding network $h_\psi : \mathbb{R}^{T \times 13} \rightarrow \mathbb{R}^{d_e}$ that is trained jointly with the flow.

The default embedding network is a BiGRU with attention pooling [CMG⁺14]. Selecting this architecture required evaluating several alternatives. The `sbi` library provides two built-in options that were considered: a causal convolutional network (`CausalCNNEmbedding`), which applies dilated causal convolutions along the time axis, and a Transformer encoder (`TransformerEmbedding`), which uses multi-head self-attention across all time steps. In addition, a custom bidirectional GRU with attention pooling was implemented from scratch as an independent PyTorch module, outside of and not relying on the `sbi` library.

The Transformer encoder was initially appealing due to its ability to model arbitrary long-range temporal dependencies. However, the full self-attention matrix scales quadratically with sequence length, and at the sequence lengths used in this work (up to $T_{\text{seg}} = 3000$ time steps with 13 input channels) this proved prohibitively memory-intensive, leading to out-of-memory conditions during training on the available GPU hardware. The causal CNN was a viable candidate and was retained as a configurable alternative; however, comparative experiments showed that the custom BiGRU encoder consistently achieved lower posterior error at comparable or lower memory usage. An earlier, heavier configuration of the BiGRU (hidden dimension $H = 128$, giving $d_e = 256$) was also tested but was progressively refined toward the current lighter design, which provides a better trade-off between expressiveness and computational cost.

The bidirectional GRU is a particularly well-suited choice for vehicle trajectory embedding. Parameter inference from driving trajectories benefits from non-causal temporal context: the friction coefficient μ , for instance, most strongly manifests itself during combined longitudinal–

lateral loading, where the onset and recovery of wheel slip unfold over many consecutive time steps. A bidirectional encoder processes the sequence in both temporal directions simultaneously, allowing it to leverage causal context (the history of control inputs and dynamic states leading up to a moment) alongside anti-causal context (how the vehicle evolves in the steps that follow). This gives the encoder access to informative patterns that would be invisible to a unidirectional network constrained to the forward direction alone. The attention pooling layer further focuses the resulting summary on the most parameter-sensitive time windows-such as the onset of wheel lockup or peak lateral loading-rather than averaging uniformly over the full trajectory.

The selected architecture (`BiGRUAttnEncoder`) is a custom implementation not provided by the `sbi` library. The architecture consists of:

1. A two-layer bidirectional GRU with hidden dimension $H = 32$, processing the input sequence in both temporal directions. At each time step t , the forward hidden state $\vec{\mathbf{h}}_t$ and backward hidden state $\overleftarrow{\mathbf{h}}_t$ are concatenated to produce $\mathbf{h}_t = [\vec{\mathbf{h}}_t \| \overleftarrow{\mathbf{h}}_t] \in \mathbb{R}^{2H}$.
2. An attention-weighted temporal pooling layer. Rather than averaging all hidden states uniformly or retaining only the final state, an additive attention mechanism learns to up-weight time steps that are most informative for parameter estimation. A learned query vector $\mathbf{w} \in \mathbb{R}^{2H}$ (trained jointly with the rest of the network) projects each hidden state to a scalar unnormalised alignment score $e_t = \mathbf{w}^\top \mathbf{h}_t \in \mathbb{R}$. These scores are normalised across all time steps by a softmax operation, yielding attention weights $\alpha_t = \exp(e_t) / \sum_{t'} \exp(e_{t'})$ that satisfy $\alpha_t \geq 0$ and $\sum_t \alpha_t = 1$. The final embedding is the attention-weighted sum of all hidden states:

$$\mathbf{e} = \sum_{t=1}^T \alpha_t \mathbf{h}_t \in \mathbb{R}^{d_e}, \quad (4.1)$$

where $d_e = 2H = 64$. Because $\mathbf{w}$ is learned end-to-end through the normalising-flow training objective, the attention weights converge toward emphasizing the time windows at which the observations are most diagnostic of the parameters-for example, the onset of wheel slip or the peak of a lateral load transfer event.

Dropout of 0.1 is applied within the GRU layers, and the fixed-length output $\mathbf{e} \in \mathbb{R}^{d_e}$ is independent of input sequence length, making the architecture compatible with any observation window duration.

4.1.2 Normalising Flow Architecture

The posterior density is modelled by a MAF [PPM17], a normalising flow architecture in which each transformation layer is an autoregressive neural network with a triangular Jacobian. The flow defines an invertible mapping

$$\boldsymbol{\theta} = f_\phi(\mathbf{z}; \mathbf{e}), \quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (4.2)$$

where $\mathbf{e} = h_\psi(\mathbf{y}_{0:T})$ is the embedding of the trajectory and $\mathbf{z} \in \mathbb{R}^{d_\theta}$ is drawn from a standard Gaussian base distribution. The flow is composed of $K = 8$ autoregressive transformations with hidden dimension 128 per layer. Each affine autoregressive layer computes element-wise shift and scale parameters conditioned on previous elements and the embedding:

$$\theta_j = \mu_j(\boldsymbol{\theta}_{<j}, \mathbf{e}) + \sigma_j(\boldsymbol{\theta}_{<j}, \mathbf{e}) \cdot z_j, \quad j = 1, \dots, d_\theta. \quad (4.3)$$

The Jacobian of each layer is lower-triangular by construction, so the log-determinant is simply the sum of $\log \sigma_j$:

$$\log |\det J_{f_k}| = \sum_{j=1}^{d_\theta} \log \sigma_j^{(k)}. \quad (4.4)$$

The full conditional log-density follows from the change-of-variables formula (Section 2.2). Because the MAF Jacobian is triangular, the log-determinant simplifies to a sum of log-scale terms:

$$\log q_\phi(\boldsymbol{\theta} \mid \mathbf{y}_{0:T}) = \log \mathcal{N}(f_\phi^{-1}(\boldsymbol{\theta}; \mathbf{e}) \mid \mathbf{0}, \mathbf{I}) + \sum_{k=1}^K \sum_{j=1}^{d_\theta} \log \sigma_j^{(k)}. \quad (4.5)$$

A key practical property of MAF is that density evaluation (and therefore training) requires only the inverse pass, which is autoregressive and requires $\mathcal{O}(d_\theta)$ sequential operations, while sampling requires the forward pass, which is sequential over dimensions but feasible for a 3-dimensional parameter space.

4.1.3 Training Procedure

The flow and embedding network are trained jointly by maximising the conditional log-likelihood on simulated data, following the NPE objective introduced in Section 2.2, with the trajectory embedding $\mathbf{e} = h_\psi(\mathbf{y}_{0:T})$ substituted for the raw observation. Training uses the Adam optimiser [KB14] with learning rate 5×10^{-4} , batch size 512, and gradient clipping at norm 20. The model is trained for a fixed number of 300 epochs without early stopping. A

held-out validation fraction of 15% is reserved for monitoring but not used for model selection or early stopping.

After training, inference for a new observation $\mathbf{y}_{0:T}^o$ requires a single forward pass through the embedding network and sampling from the flow:

$$\mathbf{z}^{(s)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad \boldsymbol{\theta}^{(s)} = f_\phi(\mathbf{z}^{(s)}; h_\psi(\mathbf{y}_{0:T}^o)), \quad s = 1, \dots, S. \quad (4.6)$$

This is the amortisation advantage: once trained, producing posterior samples is a single cheap feed-forward operation.

4.2 NPSE

NPSE replaces the normalised density estimation of NPE with posterior score estimation and diffusion-based sampling, as described in Section 2.3.2. The implementation uses the NPSE module of the `sbi` library.

4.2.1 Score Network Architecture

The score network $s_\psi(\boldsymbol{\theta}_a, \mathbf{y}_{0:T}, a)$ takes as input a noised parameter vector $\boldsymbol{\theta}_a \in \mathbb{R}^{d_\theta}$, an observation trajectory $\mathbf{y}_{0:T}$, and the diffusion time a , and outputs an estimate of the posterior score $\nabla_{\boldsymbol{\theta}_a} \log p_a(\boldsymbol{\theta}_a \mid \mathbf{y}_{0:T})$.

The trajectory is first compressed by the same BiGRU attention embedding network used in NPE (Section 4.1), producing a fixed-length summary $\mathbf{e} = h_\psi(\mathbf{y}_{0:T}) \in \mathbb{R}^{d_e}$. The score network then processes the concatenation of the noised parameter, the embedding, and a positional encoding of the diffusion time through a residual MLP. The score network architecture is provided by the `sbi` library’s internal score estimator.

Sharing the same embedding architecture between NPE and NPSE ensures that any performance difference is attributable to the posterior representation (flow versus score) rather than to a difference in how the trajectory is summarised.

4.2.2 Diffusion Process and Training

NPSE uses a VE-SDE as the forward noising process [SSDK⁺21]. The forward process progressively adds Gaussian noise to the parameter vector:

$$\boldsymbol{\theta}_a = \boldsymbol{\theta}_0 + \sigma(a) \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (4.7)$$

where $\sigma(a)$ is an increasing noise schedule and $\theta_0 \sim p(\theta \mid \mathbf{y}_{0:T})$ is a clean posterior sample. At the maximum diffusion time $a = A$, the distribution approaches a wide Gaussian.

The score network is trained via DSM on simulated data:

$$\mathcal{L}_{\text{NPSE}}(\psi) = \frac{1}{N} \sum_{i=1}^N \mathbb{E}_{a, \epsilon} \left[\lambda(a) \left\| s_\psi(\theta_a^{(i)}, \mathbf{y}_{0:T}^{(i)}, a) + \frac{\epsilon}{\sigma(a)} \right\|_2^2 \right], \quad (4.8)$$

where $\theta_a^{(i)} = \theta^{(i)} + \sigma(a)\epsilon$, the diffusion time a is uniformly sampled, and $\lambda(a)$ is a weighting function. The target $-\epsilon/\sigma(a)$ is the analytically known score of the Gaussian perturbation kernel, which avoids any need for the unknown true posterior score during training.

The training hyperparameters match NPE: Adam optimiser with learning rate 5×10^{-4} , batch size 512, gradient clipping at norm 20, and 300 fixed epochs.

4.2.3 Sampling by Reverse Diffusion

Posterior samples are obtained by numerically integrating the reverse-time SDE (Section 2.3.2) from $\theta_A \sim \mathcal{N}(\mathbf{0}, \sigma(A)^2 \mathbf{I})$ down to $a = 0$. Each sample requires iterating through the diffusion grid, making this more expensive per sample than NPE’s single-pass flow evaluation.

4.3 FNPSE

FNPSE exploits the Markovian structure of the vehicle simulator by decomposing the posterior score into local transition-level contributions. Unlike NPE and NPSE, which train on full trajectories, FNPSE trains a score network on short transition windows and then composes the local scores at inference time to approximate the full posterior, as described in Section 2.3.3.

The implementation uses the MarkovSBI framework [GTFM24] in JAX [BFH⁺18], with a custom vehicle dynamics task adapter.

Partial-observation approximation. The theoretical FNPSE factorisation is derived for full latent Markov states: the factorisation $p(\theta \mid \mathbf{x}_{0:T}) \propto p(\theta) \prod_t p(\mathbf{x}_{t+1} \mid \mathbf{x}_t, \theta)$ is exact when $\mathbf{x}_t$ is the full Markov state. In this implementation, the score network is conditioned on short windows of the *augmented observation* $\mathbf{z}_t = [\mathbf{y}_t, \mathbf{u}_t] \in \mathbb{R}^{13}$ (9 observation channels plus 4 control channels), rather than the 10-dimensional full simulator state $\mathbf{x}_t$. The state components excluded from $\mathbf{y}_t$ -global position (X, Y) and yaw angle ψ -are integrated quantities that are nearly deterministically recoverable from the observed velocities over short windows, so the partial-observation sequence is approximately Markov. Nevertheless, the factorisation is *strictly*

approximate, and this partial-observation mismatch is one plausible contributor to the observed FNPSE performance degradation, particularly for longer trajectories where the accumulated approximation error may grow.

4.3.1 Score Network Architecture

The FNPSE score network $s_\phi(\theta_a, \mathbf{z}_t, \mathbf{z}_{t+1}, a)$ takes as input a noised parameter vector, a short window of two consecutive augmented observations $\mathbf{z}_t = [\mathbf{y}_t, \mathbf{u}_t] \in \mathbb{R}^{13}$ (9 observation channels concatenated with 4 control channels), and the diffusion time. It outputs an estimate of the local posterior score $\nabla_{\theta_a} \log \tilde{p}_a(\theta_a | \mathbf{z}_t, \mathbf{z}_{t+1})$.

The architecture is an MLP with skip connections, layer normalisation, and GELU activations, implemented in Haiku [HCNB20]:

1. **Observation encoding.** The augmented observation window $(\mathbf{z}_t, \mathbf{z}_{t+1}) \in \mathbb{R}^{W \times 13}$ is processed by a GRU cell to produce a context vector $\mathbf{c} \in \mathbb{R}^{H/2}$, where $H = 128$ is the hidden dimension.
2. **Parameter input.** The noised parameter θ_a is projected to $\mathbb{R}^{H/2}$ via a linear layer.
3. **Diffusion time embedding.** The noise level $\sigma(a)$ is embedded using a Gaussian Fourier Embedding (GFE):

$$\text{GFE}(\sigma) = [\cos(2\pi \mathbf{B}\sigma), \sin(2\pi \mathbf{B}\sigma)] \in \mathbb{R}^{d_{\text{GFE}}}, \quad (4.9)$$

where $\mathbf{B} \in \mathbb{R}^{(d_{\text{GFE}}/2) \times 1}$ is a fixed random frequency matrix, followed by a linear projection to $\mathbb{R}^H$. The GFE allows the network to distinguish between different noise levels, which is critical because the score field changes qualitatively as diffusion noise is added or removed.

4. **Residual MLP.** The concatenated context and parameter vectors are summed with the time embedding and passed through $L = 5$ residual layers. Each layer applies a linear transformation, adds the time embedding, applies GELU activation, and adds a skip connection:

$$\mathbf{h}^{(l+1)} = \text{LayerNorm}\left(\text{GELU}(W^{(l)}\mathbf{h}^{(l)} + \mathbf{e}_a) + \mathbf{h}^{(l)}\right), \quad (4.10)$$

where $\mathbf{e}_a$ is the diffusion time embedding.

5. **Output.** A final linear layer projects to $\mathbb{R}^{d_\theta}$, and a preconditioning output transformation $c_{\text{out}}(a, \theta_a, \hat{\mathbf{s}})$ rescales the raw network output. This preconditioning normalises the score target across diffusion times so that the effective training target has mean-scaled unit variance per Karras et al. [KALA22], stabilising training.

The preconditioning follows the Karras [KALA22] style: three functions c_{in} , c_{noise} , c_{out} transform the inputs and outputs of the network so that the effective training target has approximately unit variance across diffusion times. Concretely, the preconditioned network output is

$$\hat{s}_\phi(\boldsymbol{\theta}_a, \mathbf{z}_t, \mathbf{z}_{t+1}, a) = c_{\text{out}}\left(a, \boldsymbol{\theta}_a, \tilde{s}_\phi\left(c_{\text{in}}(a, \boldsymbol{\theta}_a), \mathbf{z}_t, \mathbf{z}_{t+1}, c_{\text{noise}}(a)\right)\right), \quad (4.11)$$

where $\tilde{s}_\phi$ is the raw network.

4.3.2 Training Data Generation

A critical difference from NPE and NPSE is that FNPSE does not train on full-length trajectories. Instead, it generates training data as short transition windows. The training procedure involves:

1. **Pilot rollouts.** A small number of pilot trajectories (2% of the simulation budget, with a shorter horizon of $T_{\text{pilot}} = 1500$ steps) are simulated from parameters drawn from the prior. These pilots establish a pool of realistic state values.
2. **Proposal distribution.** A state-space proposal distribution $\tilde{p}(\mathbf{x}_t)$ is constructed from the pilot states. For each training sample, a state $\mathbf{x}_t$ is drawn from this proposal pool with small additive Gaussian noise ($\sigma_{\text{prop}} = 0.03 \times \text{std}(\text{pool})$), a parameter $\boldsymbol{\theta}$ is drawn from the prior, and the simulator is run for $W = 2$ steps to produce $\mathbf{x}_{t+1}$. The observation function h (Eq. 3.2) and control vector are then applied to obtain the augmented observation pair $(\mathbf{z}_t, \mathbf{z}_{t+1})$ where $\mathbf{z}_t = [\mathbf{y}_t, \mathbf{u}_t]$.
3. **Training pairs.** Each training example is a tuple $(\boldsymbol{\theta}, \mathbf{z}_t, \mathbf{z}_{t+1})$ consisting of a parameter value and a single augmented-observation transition pair. The proposal must not depend on $\boldsymbol{\theta}$ to ensure the validity of the score decomposition (Section 2.3.3).

This design has a major advantage in simulation efficiency: producing one transition window requires running the simulator for only 2 time steps. However, accounting for the pilot rollout overhead, the effective per-simulation-step cost is higher: each training pair requires $W + f_{\text{pilot}} \cdot T_{\text{pilot}} = 2 + 0.02 \times 1500 = 32$ simulation steps in expectation, where $f_{\text{pilot}} = 0.02$ is the pilot fraction and $T_{\text{pilot}} = 1500$ is the pilot trajectory length. A simulation-step budget of B therefore yields approximately

$$N_{\text{sim}} \approx \frac{B}{32} \quad (4.12)$$

transition pairs, each of which is an independent training example. This is substantially fewer than the naïve estimate $B/2$ that counts only the 2-step window cost.

4.3.3 Training Procedure

The score network is trained using the DSM objective, analogous to NPSE but conditioned on a single observation transition window rather than a full trajectory:

$$\mathcal{L}_{\text{FNPSE}}(\phi) = \mathbb{E}_{a, \theta, \epsilon, \mathbf{z}_t, \mathbf{z}_{t+1}} \left[\lambda(a) \left\| s_\phi(\theta_a, \mathbf{z}_t, \mathbf{z}_{t+1}, a) - \nabla_{\theta_a} \log p_{a|0}(\theta_a | \theta) \right\|_2^2 \right], \quad (4.13)$$

where θ_a is the noised parameter and $p_{a|0}$ is the known forward diffusion kernel.

Training uses the AdamW optimiser with a cosine one-cycle learning rate schedule peaking at 5×10^{-4} and adaptive gradient clipping at norm 20. The schedule consists of 100 outer epochs, each containing 50 inner epochs of gradient updates, with batch size 1000. A control variate is used in the loss to reduce gradient variance.

4.3.4 Score Composition at Inference

At inference time, a full observation trajectory is divided into $T - 1$ overlapping augmented-observation transition pairs $(\mathbf{z}_t, \mathbf{z}_{t+1})$. The local score network is evaluated on each pair, and the resulting scores are composed into a global posterior score approximation. The implementation provides three composition strategies:

FNPSE (original). The basic composition sums the local scores and subtracts redundant prior contributions:

$$\nabla_{\theta_a} \log p(\theta_a | \mathbf{y}_{0:T}) \approx \sum_{t=0}^{T-1} s_\phi(\theta_a, \mathbf{z}_t, \mathbf{z}_{t+1}, a) + (1 - T) \cdot w(a) \cdot \nabla_{\theta_a} \log p(\theta_a), \quad (4.14)$$

where $w(a) = (A - a)/A$ is a heuristic weight that interpolates the prior score from full weight at $a = 0$ to zero at $a = A$.

Uncorrected. Uses the exact marginalised prior score $\nabla_{\theta_a} \log p_a(\theta_a)$ instead of the heuristic weight:

$$\nabla_{\theta_a} \log p(\theta_a | \mathbf{y}_{0:T}) \approx \sum_{t=0}^{T-1} s_\phi(\theta_a, \mathbf{z}_t, \mathbf{z}_{t+1}, a) + (1 - T) \cdot \nabla_{\theta_a} \log p_a(\theta_a). \quad (4.15)$$

Gaussian-corrected. The most sophisticated variant applies precision-weighted averaging motivated by the Gaussian approximation to the local posteriors, as proposed in recent work on

diffusion posterior sampling [LCG⁺25]:

$$\nabla_{\theta_a} \log p(\theta_a \mid \mathbf{y}_{0:T}) \approx \Lambda(\theta_a)^{-1} \left(\sum_{t=0}^{T-1} \Sigma_{a,t,t+1}^{-1} s_\phi(\theta_a, \mathbf{z}_t, \mathbf{z}_{t+1}, a) + (1 - T) \Sigma_a^{-1} \nabla_{\theta_a} \log p_a(\theta_a) \right), \quad (4.16)$$

where $\Sigma_{a,t,t+1}^{-1}$ is the estimated precision of the local denoising posterior at transition $(t, t + 1)$, Σ_a^{-1} is the precision of the denoising prior, and $\Lambda = \sum_t \Sigma_{a,t,t+1}^{-1} + (1 - T) \Sigma_a^{-1}$ is the total precision. This weighting accounts for the fact that different transitions may carry different amounts of information about θ . The precision estimates are computed once per conditioning input by running short preliminary diffusion sweeps through the score network.

In the default configuration, the Gaussian-corrected composition is used.

4.3.5 Sampling

Posterior samples are obtained by reverse-diffusion sampling using the Euler–Maruyama integrator over a time grid of 500 steps from $a = A$ down to $a_{\min} = 0.05$. The minimum diffusion time $a_{\min} > 0$ avoids the singularity of the VE noise schedule at $a = 0$. Optionally, samples are clipped to the normalised prior bounds during the diffusion to prevent divergence. Sampling is performed in chunks of 64 to manage GPU memory.

4.4 Simformer

Simformer is an all-conditional score-based method that models the full joint distribution $p(\theta, \mathbf{y}_{0:T})$ using a transformer architecture, as described in Section 2.3.4. At inference time, it conditions on the observed trajectory and generates posterior samples via reverse diffusion. The implementation follows the reference codebase of Glöckler et al. [GDW⁺24], adapted to handle long vehicle trajectories through temporal subsampling.

4.4.1 Scalar Node Representation

Simformer represents each variable as a scalar token (Section 2.3.4). With the augmented observation vector $\mathbf{z}_t = [\mathbf{y}_t, \mathbf{u}_t]^\top \in \mathbb{R}^{d_z}$ ($d_z = d_y + d_u = 9 + 4 = 13$), each time point contributes d_z scalar nodes. For the vehicle dynamics problem, the node types are:

- $d_\theta = 3$ **parameter nodes**: one each for μ , c_d , and m .
- $d_z \times N_{\text{tp}}$ **augmented observation nodes**: one for each channel of $\mathbf{z}_t$ at each of N_{tp} sampled time points; $d_z = d_y + d_u = 13$.

Because processing all $T_{\text{seg}} = 3000$ time steps would produce $3000 \times d_z = 39,000$ augmented observation nodes, which is computationally prohibitive for a transformer, Simformer subsamples the trajectory. In the default configuration, $N_{\text{tp}} = 64$ time points are sampled, yielding $d_z \times 64 = 13 \times 64 = 832$ augmented observation nodes plus 3 parameter nodes, for a total of $N_{\text{nodes}} = 835$ nodes.

During training, the N_{tp} time points are randomly sampled per batch element, and the observation values at these time points are obtained by linear interpolation from the dense trajectory. During inference, the time points are fixed to a uniform grid. Each node carries two pieces of metadata: a **node identity** indicating which variable it represents (encoded as an integer ID), and a **temporal metadata** value indicating the normalised time position $\tau \in [0, 1]$ of the observation. Parameter nodes have no temporal metadata (set to NaN).

The joint data vector at each training step is therefore

$$\hat{\mathbf{z}} = \left(\theta_1 \quad \theta_2 \quad \theta_3 \quad z_{1,\tau_1} \quad \cdots \quad z_{d_z,\tau_{N_{\text{tp}}}} \right)^\top \in \mathbb{R}^{N_{\text{nodes}} \times 1}. \quad (4.17)$$

4.4.2 Transformer Architecture

The score network is a scalar transformer [GDW⁺24] that processes the node sequence. Each node is first projected into a token embedding of dimension $d_{\text{tok}} = 40$ through a per-node-type embedding layer. The diffusion time a is embedded via a learned sinusoidal embedding into $d_{\text{time}} = 128$ dimensions and broadcast to all tokens.

Conditioned nodes (observations during posterior inference) receive a separate learned condition token of dimension $d_{\text{cond}} = 10$, which is concatenated with the main token embedding. The model then applies $L = 8$ transformer blocks, each consisting of:

1. Multi-head self-attention with $n_h = 4$ heads and per-head dimension $d_h = 10$, followed by a skip connection.
2. A widening MLP with expansion factor 4, followed by a skip connection.
3. Layer normalisation after each sub-layer.

The output is projected back to scalar dimension to produce the node-level score estimate.

4.4.3 All-Conditional Training

At each training step, the conditioning mask is drawn from the following mixture, enabling the model to learn all possible conditional distributions of the joint:

- 20% **joint** (p_{joint}): No nodes conditioned; the model learns the unconditional joint score.
- 20% **posterior** (p_{post}): All observations conditioned, parameters latent; directly trains the posterior score.
- 20% **likelihood** (p_{lik}): All parameters conditioned, observations latent; trains the likelihood score.
- 20% **random 1** (p_{rnd1}): Each node independently conditioned with probability 0.3.
- 20% **random 2** (p_{rnd2}): Each node independently conditioned with probability 0.7.

The training loss is the DSM objective applied only to unconditioned nodes:

$$\mathcal{L}_{\text{Sim}}(\phi) = \mathbb{E}_{a, \hat{\mathbf{z}}, \epsilon, \mathbf{m}} \left[\lambda(a) \sum_{j \notin \mathbf{m}} \left\| s_{\phi}^{(j)}(\hat{\mathbf{z}}_a, a, \mathbf{m}) + \frac{\epsilon_j}{\sigma(a)} \right\|^2 \right], \quad (4.18)$$

where $\mathbf{m}$ is the set of conditioned node indices and $\hat{\mathbf{z}}_a$ is the noised joint vector.

4.4.4 SDE and Sampling

Simformer uses a VE-SDE with noise parameters $\sigma_{\min} = 0.01$ and $\sigma_{\max} = 15.0$, and diffusion time bounds $a_{\min} = 0.02$, $a_{\max} = 1.0$.

The total number of training steps is scaled linearly with the dataset size, clipped to [5,000, 100,000], with a scaling factor of 3. The optimiser is Adam with a linear learning rate schedule from 10^{-3} to 10^{-6} , and adaptive gradient clipping at norm 10. No validation split is used; the model is trained for the full number of steps without early stopping.

At inference time, all augmented observation nodes are conditioned and only the $d_{\theta} = 3$ parameter nodes are generated via reverse-SDE sampling over 500 diffusion steps. The generated parameter samples are optionally clipped to the normalised prior bounds. Sampling is performed in batches of 128 to manage transformer memory.

4.4.5 Differences from NPE and NPSE

Three structural differences from the PyTorch-based methods are worth noting:

1. **No external embedding network.** Simformer processes the raw observation values as scalar tokens within the transformer. The temporal and cross-channel relationships are learned through attention rather than through a separate pre-trained sequence encoder.
2. **Temporal subsampling.** Instead of processing all T_{seg} time steps, Simformer interpolates

observations at $N_{\text{tp}} = 64$ sampled time points. This trades temporal resolution for computational tractability. During training, the time points are randomised per batch, which acts as a data augmentation that regularises the model.

3. **Joint modelling.** The model is trained on the full joint rather than only the posterior. While this makes the training problem harder-requiring learning the full joint $p(\boldsymbol{\theta}, \mathbf{y}_{0:T})$ rather than only the conditional $p(\boldsymbol{\theta} \mid \mathbf{y}_{0:T})$ -it enables the model to learn the structure of the data-generating process more holistically. At inference time, only the posterior conditioning pattern is used.

The key differences are summarised in Table 4.1.

Table 4.1: Summary of implementation differences across the four inference methods.

	NPE	NPSE	FNPSE	Simformer
Framework	PyTorch	PyTorch	JAX/Haiku	JAX/Haiku
Posterior representation	Norm. flow	Score field	Factorised score	Joint score
Embedding network	BiGRU	BiGRU	GRU (local)	Attention (internal)
Trained on	Full traj.	Full traj.	Windows ($W=2$)	Subsampled nodes
Training data	Shared	Shared	Self-generated	Shared
Sampling	Direct	Rev. diffusion	Rev. diffusion	Rev. diffusion
Diffusion steps	–	500	500	500
Structural assumption	None	None	Markov	Learned

5 Comparative Evaluation

This chapter presents the comparative evaluation of the four inference methods introduced in Chapter 4. Section 5.1 defines the experimental protocol, including the benchmark grid, evaluation metrics, and computational environment. Section 5.2 reports the simulative comparison across all combinations. Section 5.3 evaluates the trained posteriors on real vehicle measurement data from the Jeversen proving ground. Section 5.4 synthesizes the findings and formulates method recommendations.

5.1 Experimental Setup

5.1.1 Benchmark Grid

To evaluate the four methods under controlled and comparable conditions, a full-factorial benchmark grid is constructed over the following dimensions:

- **Inference methods** (4): NPE, NPSE, FNPSE, and Simformer, as described in Chapter 4.
- **Parameter sets** (4): The inference target is varied across four combinations of the vehicle parameters defined in Section 3.7:
 1. $\theta = (\mu)$ - friction coefficient only ($d_\theta = 1$),
 2. $\theta = (\mu, c_d)$ - friction and aerodynamic drag ($d_\theta = 2$),
 3. $\theta = (\mu, m)$ - friction and mass ($d_\theta = 2$),
 4. $\theta = (\mu, c_d, m)$ - all three parameters ($d_\theta = 3$).
- **Observation window lengths** (3): $T_{\text{seg}} \in \{1000, 2000, 3000\}$ time steps, corresponding to trajectory durations of 10 s, 20 s, and 30 s at $\Delta t = 0.01$ s.
- **Simulation budgets** (3): The total number of simulation steps is set to $B \in \{20\text{M}, 40\text{M}, 60\text{M}\}$ (where M denotes millions of simulation steps). The number of training trajectories is $N_{\text{sim}} = B/T_{\text{seg}}$, so longer observation windows yield fewer training samples for the same budget.

The full grid contains $4 \times 4 \times 3 \times 3 = 144$ unique configurations. Each configuration is repeated with 2–5 independent random seeds: 5 seeds at $T_{\text{seg}} = 1000$ and 2 seeds at $T_{\text{seg}} \in \{2000, 3000\}$.

In total, 432 experiments were completed. Table 5.1 summarises the grid.

Table 5.1: Benchmark grid dimensions. The total simulation budget is held constant; increasing T_{seg} reduces the number of available training trajectories proportionally.

Dimension	Values
Methods	NPE, NPSE, FNPSE, Simformer
Parameter sets	(μ) , (μ, c_d) , (μ, m) , (μ, c_d, m)
T_{seg}	1000, 2000, 3000 time steps
Budget B	20 M, 40 M, 60 M simulation steps
Seeds per config	2–5
Unique configurations	144
Total completed runs	432

5.1.2 Evaluation Protocol

The evaluation is performed on a held-out test set of $N_{\text{test}} = 50$ simulated observations, each generated with a different parameter vector drawn from the tail region of the prior (beyond the central quantiles) under high-excitation driving conditions. For every test observation $\mathbf{y}^{(i)}$, the method produces $S = 400$ posterior samples $\{\boldsymbol{\theta}^{(i,s)}\}_{s=1}^S$, from which aggregate posterior quality and predictive performance metrics are computed. Figure 5.1 shows representative simulator trajectories at three friction levels for visual orientation.

Implication for coverage interpretation. Drawing test parameters from the tails of the prior is a deliberate stress-test choice: a method that has merely memorised the prior should not score well, and a method that genuinely extracts information from the observation should still recover the parameter even when it lies in low-prior-density regions. This design is different from the standard SBC convention, in which test parameters are drawn from the prior and a calibrated posterior is expected to cover the truth at the nominal credible level. The empirical coverage values reported in this thesis should therefore be interpreted as relative comparisons across methods under a common, demanding test regime, not as absolute calibration scores against a nominal target.

Posterior quality metrics. The following metrics quantify posterior concentration around the generating parameter, calibration, and prior-relative informativeness. Because the likelihood is intractable, no reference posterior is computed: parameter-recovery metrics compare posterior samples directly to the known generating parameter $\boldsymbol{\theta}^*$.

Simulated vehicle observations for three friction levels ($\mu \in \{0.6, 0.9, 1.4\}$, $T_{\text{seg}} = 1000$ steps, 10 s at 100 Hz).
Control: 0.4 Hz sine-steer ($\pm 6^\circ$); engine torque (0–5 s); braking (5–10 s).

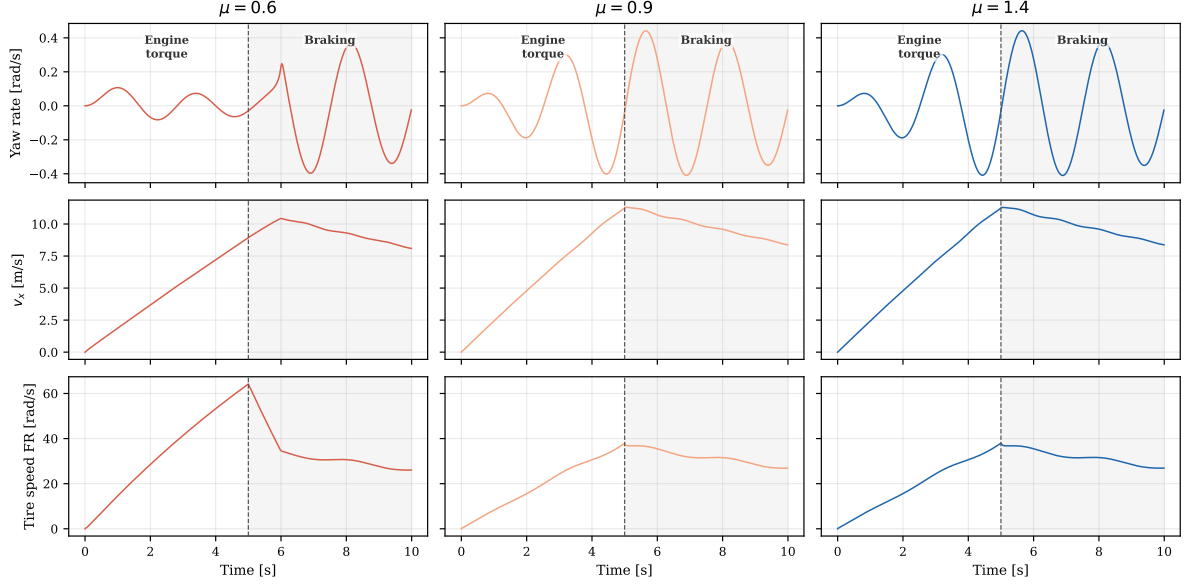


Figure 5.1: Illustrative diagnostic trajectories for three friction levels ($\mu \in \{0.6, 0.9, 1.4\}$, all within the inference prior range $[0.50, 1.50]$; $T_{\text{seg}} = 1000$ time steps, 10 s at 100 Hz). The control input is a *fixed* synthetic manoeuvre—0.4 Hz sinusoidal steering ($\pm 6^\circ$) with constant engine torque for the first 5 s (unshaded), followed by braking (shaded; dashed line marks the phase transition)—selected for visual clarity; actual training trajectories are generated from randomised block-based manoeuvres (Section 3.4). Each column corresponds to one μ value; each row shows one observation channel (rows shared-scale within each channel). Higher friction amplifies the yaw response and delays braking deceleration. At $\mu = 0.6$, the sharp drop in tire speed at $t = 5$ s reflects wheel lock under low friction; at higher μ the tire decelerates more gradually.

- **Parameter-recovery W_2 :** The L^2 -Wasserstein distance between the empirical posterior distribution and the Dirac delta at the generating parameter θ^* , computed as $W_2(\hat{p}, \delta_{\theta^*}) = \sqrt{\mathbb{E}_{\hat{p}}[\|\theta - \theta^*\|^2]}$. Since θ^* is known exactly in simulation, this equals the root mean squared distance from posterior samples to the generating parameter—a *posterior concentration* metric that rewards both correct mode location and appropriate spread. Lower is better.
- **L^2 error:** The Euclidean distance between the posterior mean and the true parameter value, averaged over all test observations.

- **Coverage at α (Cov@ α):** The fraction of test observations for which the true parameter lies within the α -credible region of the posterior. Evaluated at $\alpha \in \{50\%, 90\%\}$. Well-calibrated methods should achieve coverage close to the nominal level.
- **Coverage curve MAE:** The mean absolute error of the empirical coverage curve with respect to the ideal diagonal, aggregated over all credible levels. A single scalar summary of calibration quality.
- **Prior-Separation C2ST (C2ST-prior):** A neural classifier is trained to distinguish posterior samples ($S = 400$ per test observation) from prior samples $\theta^{\text{ref}} \sim p(\theta)$. Values near 0.5 indicate the posterior is indistinguishable from the prior (no information extracted); values near 1.0 indicate a posterior that has departed substantially from the prior. Higher is more informative (table headings use $\uparrow$), but informativeness does not imply correctness—a posterior may be highly concentrated in the wrong place. This use of C2ST differs from the standard SBI calibration diagnostic, which compares against a reference posterior.

Predictive quality metrics. To assess whether the posterior translates into accurate predictions, each posterior sample is used to simulate a forward trajectory under the same controls as the test observation:

- **PPC RMSE:** The RMSE between the posterior predictive trajectories and the held-out test trajectory, computed per observation channel and averaged.

Per-parameter metrics. For multi-parameter configurations, per-parameter versions of RMSE, bias, and coverage are computed in both physical units and in the normalised $[0, 1]$ space of the prior. This allows diagnosing which parameters are well-identified and which are not.

Computational metrics. Training wall-clock time (in seconds) and average sampling time per posterior (in seconds) are recorded. All experiments were run on a GPU compute cluster using NVIDIA A100 and H100 GPUs, with 8 CPU cores and 64 GB RAM per job.

5.2 Simulative Comparison

This section presents the benchmark results, organised by parameter set. For each parameter set, the key metrics are reported across all methods, budgets, and trajectory lengths. Budget

scaling and trajectory length effects are discussed within each subsection rather than as separate ablation studies, since they are integral to understanding method behaviour.

5.2.1 Single-Parameter Estimation (μ only)

The simplest estimation task considers only the tire–road friction coefficient μ as the inference target ($d_\theta = 1$). This configuration serves as a baseline: because there is only one parameter, the posterior is a univariate distribution that all methods can approximate with fewer capacity constraints.

Posterior accuracy. Table 5.2 reports the main metrics for the μ -only configuration at $T_{\text{seg}} = 1000$. NPE consistently achieves the lowest W_2 and L^2 across all budgets, followed by NPSE. FNPSE and Simformer produce substantially wider posteriors, with W_2 values 4–20 $\times$ higher than NPE.

Table 5.2: Benchmark results for single-parameter estimation (μ only) at $T_{\text{seg}} = 1000$. Values are mean $\pm$ s.d. across 5 independent seeds at $T_{\text{seg}} = 1000$. Results at $T_{\text{seg}} = 2000$ and $T_{\text{seg}} = 3000$ (2 seeds each) are reported in figures only and are interpreted as directional evidence; see Section 5.4.3 (Limitation 7). For error and coverage metrics, best mean values are highlighted in **bold**. C2ST-prior is an informativeness diagnostic, not a calibration metric; this column is not bolded.

Method	Budget	$W_2 \downarrow$	$L^2 \downarrow$	Cov@90 $\uparrow$	Cov@50 $\uparrow$	C2ST-prior $\uparrow$	PPC RMSE $\downarrow$
NPE	20 M	0.026 $\pm$ 0.004	0.019 $\pm$ 0.005	0.678 $\pm$ 0.190	0.350 $\pm$ 0.145	0.927 $\pm$ 0.008	1.287
NPE	40 M	0.022 $\pm$ 0.005	0.017 $\pm$ 0.007	0.680 $\pm$ 0.273	0.372 $\pm$ 0.186	0.935 $\pm$ 0.005	1.270
NPE	60 M	0.017 $\pm$ 0.001	0.011 $\pm$ 0.002	0.848 $\pm$ 0.057	0.430 $\pm$ 0.096	0.939 $\pm$ 0.006	1.280
NPSE	20 M	0.062 $\pm$ 0.013	0.054 $\pm$ 0.013	0.394 $\pm$ 0.113	0.154 $\pm$ 0.064	0.882 $\pm$ 0.023	1.289
NPSE	40 M	0.042 $\pm$ 0.004	0.034 $\pm$ 0.004	0.566 $\pm$ 0.091	0.226 $\pm$ 0.080	0.900 $\pm$ 0.011	1.244
NPSE	60 M	0.030 $\pm$ 0.004	0.021 $\pm$ 0.003	0.776 $\pm$ 0.119	0.350 $\pm$ 0.053	0.907 $\pm$ 0.008	1.269
FNPSE	20 M	0.443 $\pm$ 0.191	0.436 $\pm$ 0.196	0.028 $\pm$ 0.047	0.000 $\pm$ 0.000	0.838 $\pm$ 0.186	1.711
FNPSE	40 M	0.255 $\pm$ 0.144	0.250 $\pm$ 0.148	0.136 $\pm$ 0.155	0.062 $\pm$ 0.068	0.891 $\pm$ 0.131	2.030
FNPSE	60 M	0.235 $\pm$ 0.050	0.233 $\pm$ 0.052	0.028 $\pm$ 0.028	0.004 $\pm$ 0.008	0.939 $\pm$ 0.100	2.793
Simformer	20 M	0.389 $\pm$ 0.033	0.259 $\pm$ 0.077	0.810 $\pm$ 0.319	0.260 $\pm$ 0.377	0.593 $\pm$ 0.038	1.507
Simformer	40 M	0.344 $\pm$ 0.029	0.252 $\pm$ 0.023	0.900 $\pm$ 0.104	0.006 $\pm$ 0.005	0.542 $\pm$ 0.016	1.388
Simformer	60 M	0.327 $\pm$ 0.048	0.203 $\pm$ 0.063	0.968 $\pm$ 0.044	0.364 $\pm$ 0.368	0.560 $\pm$ 0.029	1.486

Empirical coverage. While NPE achieves the best accuracy, it does not achieve the highest empirical coverage. Simformer achieves the highest 90%-coverage, driven by posteriors that remain close to the prior-wide, diffuse posteriors naturally cover the true value, but without reducing parameter uncertainty. NPE and NPSE produce much more concentrated posteriors

(C2ST near 1.0), which improves W_2 and L^2 but at the cost of empirical coverage. FNPSE coverage is near zero, indicating severely overconfident posteriors. Figure 5.2 shows W_2 across all budgets and trajectory lengths; Figure 5.3 shows the multi-metric calibration panel.

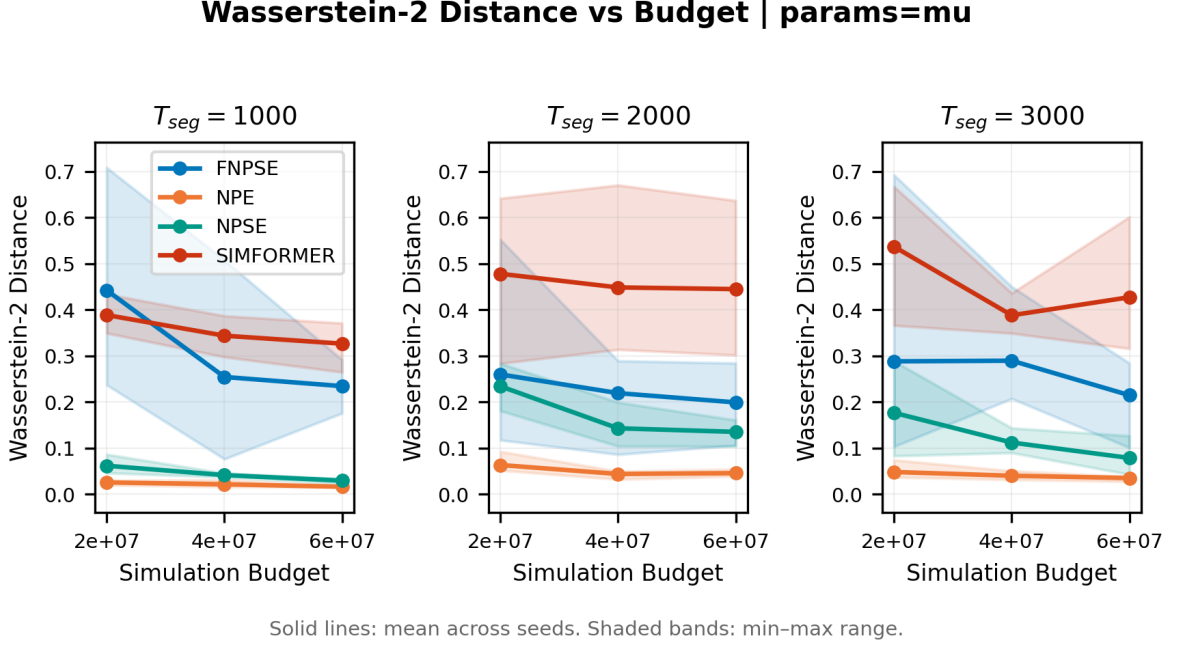


Figure 5.2: Parameter-recovery W_2 (root mean squared distance from posterior samples to generating parameter μ^*) for single-parameter estimation across all methods, trajectory lengths, and simulation budgets. Lower is better. NPE consistently achieves the lowest parameter-recovery W_2 , while Simformer and FNPSE produce wider or misplaced posteriors with higher values.

Qualitative posterior comparison. Figure 5.4 shows representative posterior distributions from all four methods for a single held-out test case ($\theta^* = \mu = 0.845$, $B = 60$ M, $T_{seg} = 3000$, seed 42), plotted against the prior. The qualitative differences are more revealing than the aggregate metrics alone.

NPE produces a sharp, unimodal posterior tightly concentrated at the true value, with the prior shown for scale. NPSE achieves a similar result with marginally wider spread. These two methods correctly identify μ from the observation. FNPSE exhibits a qualitatively different failure: the posterior collapses to a mode near $\mu \approx 0.5$, far from the true value $\mu = 0.845$, and the credible interval does not cover the ground truth at any standard level. This is not a calibration artifact but a posterior correctness failure-the Markov-factorised score model

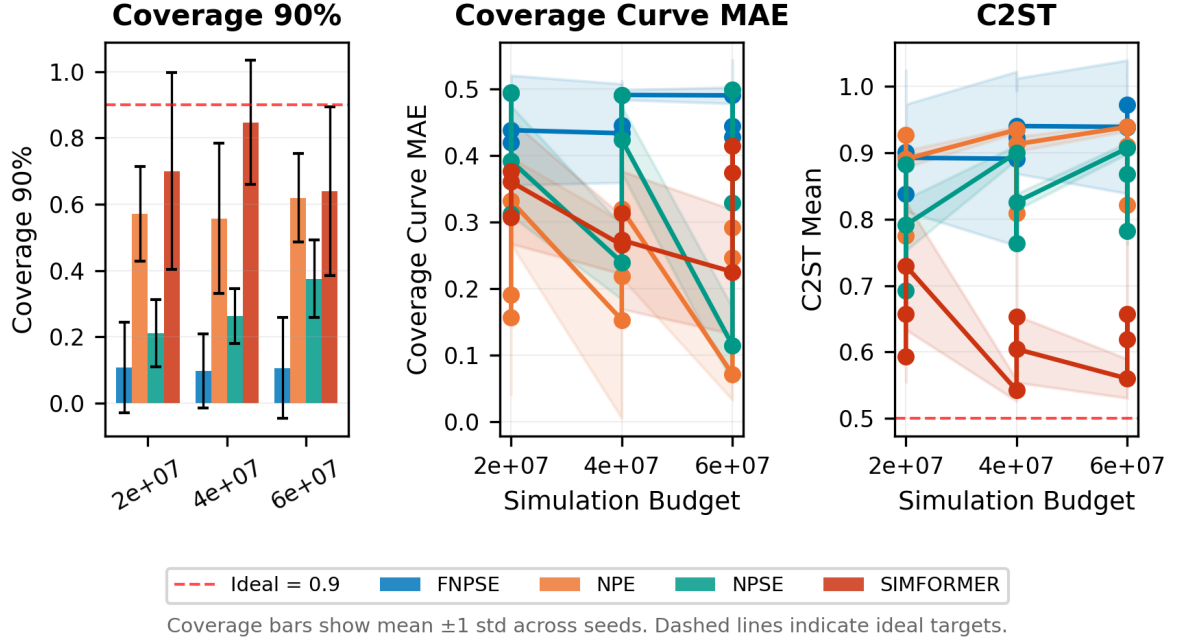
Calibration & Posterior Quality Diagnostics | params= μ 

Figure 5.3: Calibration diagnostics for μ -only estimation at three simulation budgets (2×10^7 , 4×10^7 , 6×10^7 simulation steps). **Left:** 90% empirical coverage; each bar shows mean ± 1 standard deviation across seeds (aggregated over trajectory lengths); the dashed red line marks the nominal 90% target. **Middle:** Coverage Curve MAE (lower is better); each method shows three connected points per budget level, one per $T_{\text{seg}} \in \{1000, 2000, 3000\}$, with shaded bands spanning one standard deviation across seeds. **Right:** Prior-Separation C2ST (higher values indicate posteriors more informative - i.e. further from the prior; $\uparrow$ direction matches the tables); same structure as the middle panel; the dashed red line marks the prior-equivalent value of 0.5. Simformer (red) achieves the highest empirical coverage and the lowest C2ST-prior score (closest to 0.5, the prior-equivalent value), consistent with its diffuse, near-prior posteriors; this indicates low informativeness rather than posterior correctness. FNPSE (blue) severely undercovers across all budgets.

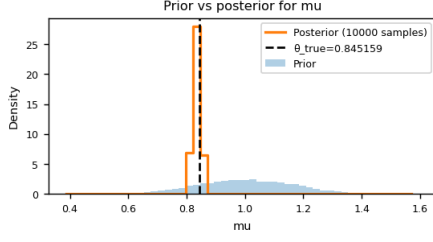
converges to the wrong region of parameter space on this test case. The aggregate near-zero coverage reported in Table 5.2 is a direct consequence of this mode-collapse behaviour. Simformer, in contrast, produces a wide posterior that remains close to the prior and does cover the true value, but provides little concentrated information about μ . Its empirically “good” calibration reflects diffuseness rather than informativeness.

These qualitative observations are important context for interpreting the aggregate metrics. A method with low W_2 and high C2ST (NPE, NPSE) is genuinely identifying μ from the data. A method with near-zero coverage (FNPSE) may be producing a narrow posterior, but it is pointing at the wrong location. A method with high coverage (Simformer) achieves this by remaining uncertain.

Qualitative PPC comparison. Figures 5.5 and 5.6 show the posterior predictive checks for the same test case: the held-out simulated trajectory is overlaid with the PPC median and 10th–90th percentile band obtained by propagating 400 posterior samples through the simulator. NPE and NPSE produce PPC ensembles whose median closely tracks the true signal across all 9 channels, with credible bands that are narrow but still cover the trajectory. FNPSE’s predictions diverge visibly in the wheel-speed channels (tire_FL through tire_RR), which is the direct consequence of drawing simulator runs with $\mu \approx 0.5$ instead of the true $\mu = 0.845$: the modelled tyres saturate at lower slip and the simulated wheel speed profiles differ from the observed ones. Simformer’s PPC median is reasonable but the uncertainty band is wide, consistent with the near-prior posterior that samples across the full μ range.

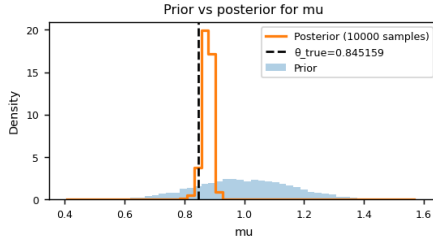
Trajectory length effects. The effect of the observation window length T_{seg} appears non-monotonic for some methods, but this analysis rests on uneven seed counts: $T_{\text{seg}} = 1000$ was run with 5 seeds whereas $T_{\text{seg}} = 2000$ and 3000 were run with only 2 seeds (Section 5.4.3, Limitation 7). The trajectory-length comparisons in this subsection should therefore be read as directional indications rather than statistically robust effects. With this caveat, the patterns observed are as follows. For NPE, W_2 appears best at $T_{\text{seg}} = 1000$ (0.017–0.026), worse at $T_{\text{seg}} = 2000$ (0.044–0.064), and partially recovered at $T_{\text{seg}} = 3000$ (0.035–0.049); given the seed count, this non-monotonicity is suggestive rather than conclusive. One plausible explanation, rooted in the fixed budget constraint $B = N_{\text{sim}} \cdot T_{\text{seg}}$, is that increasing T_{seg} simultaneously reduces the number of distinct training trajectories ($N_{\text{sim}} = B/T_{\text{seg}}$ halves when T_{seg} doubles) while increasing the per-trajectory information content; at $T_{\text{seg}} = 2000$, the cost of halving the training-set diversity may outweigh the benefit of longer context. At $T_{\text{seg}} = 3000$, the very long trajectories begin to span qualitatively richer dynamics-covering more complete braking events and sustained combined-slip phases-whose added informativeness could partially compensate for the further reduction in training-set size. This explanation is consistent with the observed pattern but cannot be confirmed from two seeds alone. For NPSE, the directional evidence suggests a sharper degradation at $T_{\text{seg}} = 2000$ ($W_2 = 0.14$ – 0.23 , with coverage moving toward zero) that is less severe at $T_{\text{seg}} = 3000$ ($W_2 = 0.08$ – 0.18); FNPSE shows the opposite directional trend (improvement from $W_2 = 0.23$ – 0.44 at $T_{\text{seg}} = 1000$ to $W_2 = 0.20$ –

Prior vs Posterior (ex0)
 $\theta_{\text{true}}: \mu=0.845159$



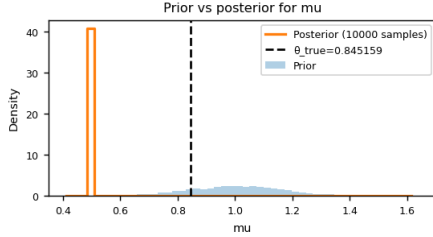
(a) NPE

Prior vs Posterior (ex0)
 $\theta_{\text{true}}: \mu=0.845159$



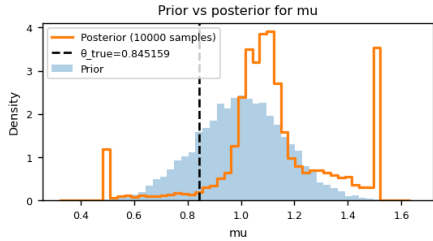
(b) NPSE

Prior vs Posterior (ex0)
 $\theta_{\text{true}}: \mu=0.845159$



(c) FNPSE

Prior vs Posterior (ex0)
 $\theta_{\text{true}}: \mu=0.845159$



(d) Simformer

Figure 5.4: Representative prior–posterior comparison for single-parameter μ estimation. All panels show the same held-out test case ($\mu^* = 0.845$, $B = 60\text{M}$, $T_{\text{seg}} = 3000$, seed 42). Blue: prior $p(\mu)$. Orange: estimated posterior $\hat{p}(\mu \mid \mathbf{y})$. Red dashed line: ground truth μ^* . **(a) NPE:** sharp posterior concentrating tightly at the true value. **(b) NPSE:** accurate, marginally wider than NPE. **(c) FNPSE:** mode collapse to $\mu \approx 0.5$; the posterior does not cover μ^* . **(d) Simformer:** near-prior, wide posterior that covers μ^* by remaining diffuse rather than by identifying it.

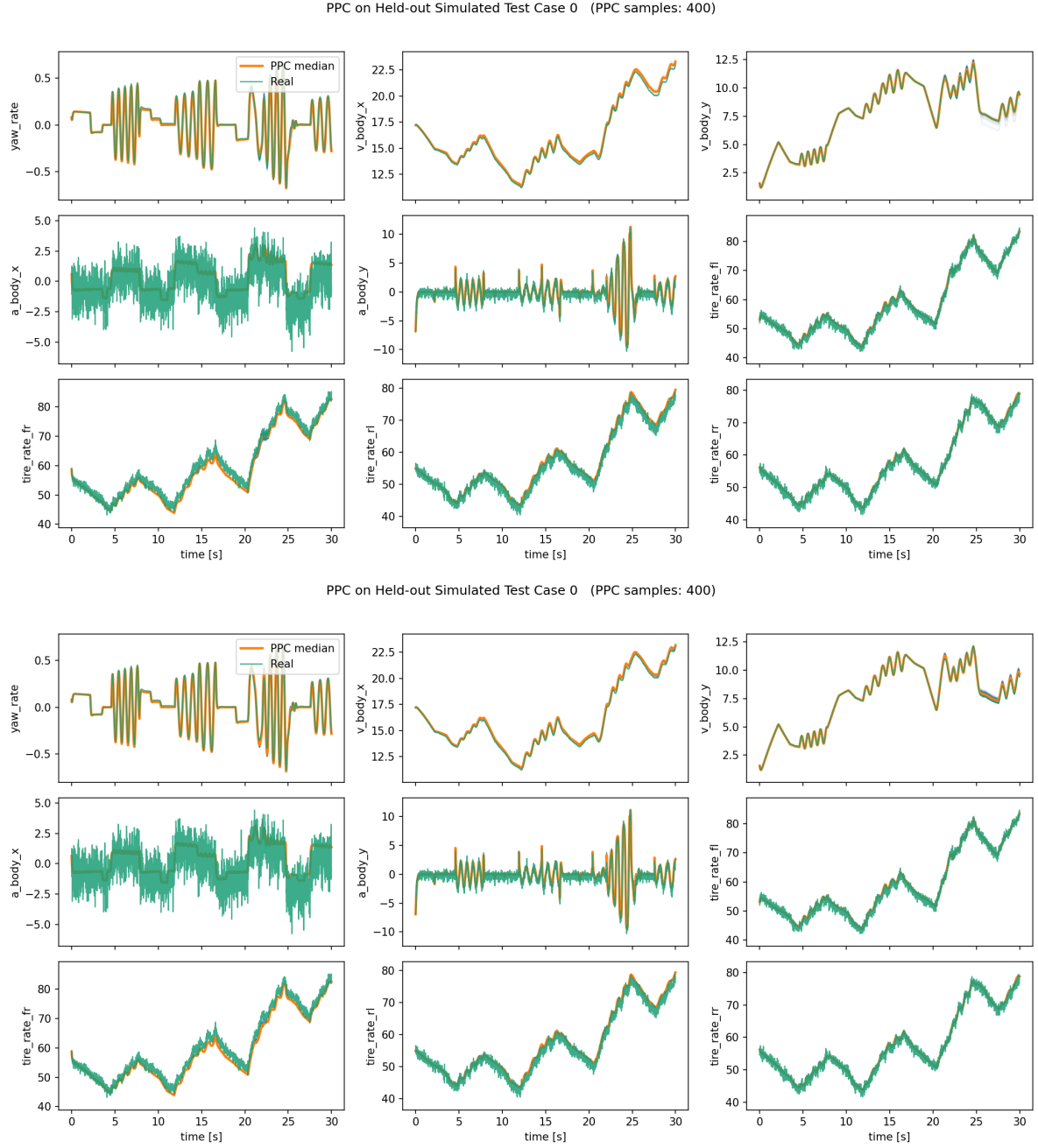


Figure 5.5: Posterior predictive check for single-parameter μ estimation (NPE and NPSE), same test case as Figure 5.4 ($\mu^* = 0.845$, $B = 60\text{ M}$, $T_{\text{seg}} = 3000$, seed 42). Each panel shows all 9 observation channels (yaw rate, global-frame velocities, body-frame accelerations, four wheel speeds). Teal line: held-out simulated test trajectory. Orange line: PPC median. Teal shading: 10th–90th percentile band across 400 posterior samples. Both NPE and NPSE produce tight bands that closely track the test signal across all channels.

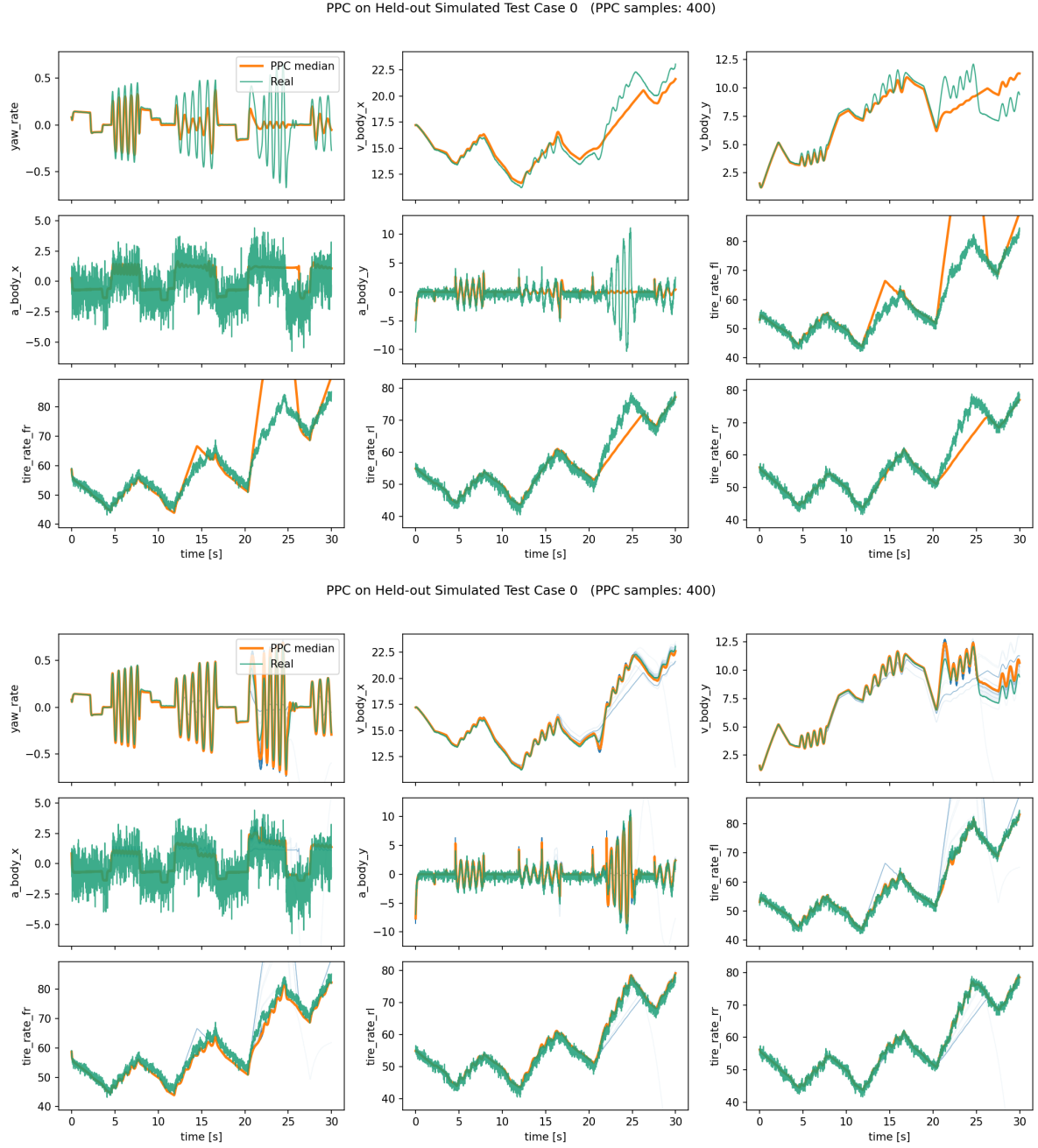


Figure 5.6: Posterior predictive check for single-parameter μ estimation (FNPSE and Simformer), same test case as Figures 5.4 and 5.5. Teal line: held-out test trajectory. Orange line: PPC median. Teal shading: 10th–90th percentile band across 400 posterior samples. FNPSE deviates systematically in the wheel-speed channels, a direct consequence of its posterior collapsing to $\mu \approx 0.5$ rather than the true $\mu^* = 0.845$; the median trajectory simulates a vehicle with lower friction, causing incorrect wheel-lock onset timing. Simformer’s median is plausible but the band is wide, consistent with its near-prior posterior that samples across the full μ range.

0.29 at $T_{\text{seg}} = 2000$); and Simformer is least sensitive, with modest increases at $T_{\text{seg}} \geq 2000$, consistent with its 64-point subsampling design.

Two mechanisms could plausibly contribute to the patterns above. First, a segment-completeness effect: under the block-based manoeuvre generation scheme (Section 3.4), longer segments are more likely to span complete braking-and-cornering cycles, which would partially compensate for the reduced number of training trajectories at high T_{seg} . Second, a fixed-capacity bottleneck: the BiGRU embedding produces a 64-dimensional summary regardless of sequence length, so doubling T_{seg} doubles the information that must be compressed into the same space, plausibly degrading score-network input quality for NPSE. Both mechanisms are stated as hypotheses for future work rather than conclusions of this thesis; verifying them would require additional seeds at $T_{\text{seg}} \geq 2000$, an architecture ablation across embedding capacities, or both.

Wasserstein-2 Distance vs T_{seg} | params= μ

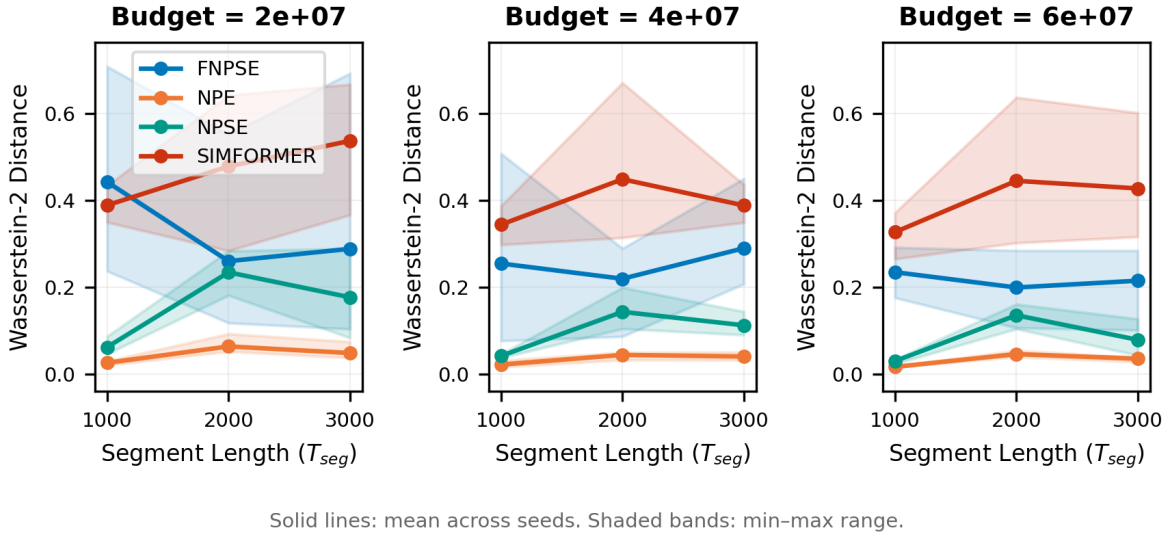


Figure 5.7: Parameter-recovery W_2 versus trajectory length for μ -only estimation. NPE exhibits non-monotonic behaviour; NPSE degrades sharply at $T_{\text{seg}} = 2000$; FNPSE improves slightly with longer windows; Simformer is relatively invariant.

Simformer: structural limitation from sequence subsampling. As described in Section 4.4, the Simformer subsamples all trajectories to a fixed 64-point context window before feeding them to the Transformer, regardless of the original length. At $T_{\text{seg}} = 1000$, this corresponds to one context point every ≈ 15.6 time steps, yielding an effective sampling rate of

≈ 6.4 Hz.

This is a fundamental bottleneck for friction estimation. The tire–road friction coefficient μ manifests most strongly in two high-frequency phenomena: the onset and recovery of wheel slip during ABS braking cycles (typically 10–15 Hz) and the transients of lateral load transfer during combined steering and braking, which unfold over 100–200 ms (10–20 simulator time steps). Both of these diagnostic features sit at or above the Nyquist limit of the 6.4 Hz effective rate, so the subsampling systematically discards the signal components that best identify μ . The Transformer then operates on a trajectory from which the most informative content has been removed, regardless of its ability to model temporal dependencies among the retained 64 points.

A natural remedy would be to train Simformer on shorter trajectories at native 100 Hz resolution—for example, 200-step (2 s) windows—which would preserve the high-frequency dynamics while keeping the sequence length computationally tractable for Transformer self-attention. The trade-off is that very short windows may not always capture a complete slip event, requiring careful segment selection. As implemented in this benchmark, the subsampling strategy is a plausible major contributor to Simformer underperforming the recurrent methods on posterior accuracy, despite its competitive empirical coverage.

Budget scaling. All methods improve with increasing simulation budget, but the marginal gains differ. NPE and NPSE at $T_{\text{seg}} = 1000$ show approximately log-linear improvement: doubling the budget from 20 M to 40 M reduces W_2 by 15–33%, and a further increase to 60 M yields another 10–23% improvement. FNPSE benefits more from additional data: W_2 drops by 47% from 0.44 (20 M) to 0.23 (60 M) at $T_{\text{seg}} = 1000$, indicating that the Markov-factorised training requires larger datasets even at high budgets. Simformer shows weak and inconsistent budget scaling, with standard deviation across seeds exceeding the mean improvement.

Predictive quality. PPC RMSE values are comparable across NPE, NPSE, and Simformer at $T_{\text{seg}} = 1000$ (1.27–1.49 at 60 M), indicating that even wide or miscalibrated posteriors can produce adequate trajectory predictions. FNPSE has the highest PPC RMSE values, increasing from 1.71 at 20 M to 2.79 at 60 M at $T_{\text{seg}} = 1000$ —a counterintuitive trend where more training leads to worse predictions, possibly due to mode collapse in the factorised score.

Summary. For μ -only estimation, NPE achieves the lowest W_2 and L^2 across all configurations, and Simformer achieves the highest empirical coverage (coverage nearest nominal, C2ST nearest 0.5), primarily through diffuse posteriors that remain close to the prior. NPSE occupies a competitive middle ground on accuracy but suffers from degraded empirical coverage at longer

trajectory lengths. FNPSE performs worst on all posterior quality metrics in this benchmark, indicating that the Markov factorisation does not benefit this single-parameter estimation task in this setting, where the full trajectory already contains sufficient information.

5.2.2 Two-Parameter Estimation (μ, c_d)

Adding the aerodynamic drag coefficient c_d to the inference target doubles the dimensionality and introduces a second parameter with different identifiability characteristics. While μ dominates the dynamics during braking and lateral manoeuvres, c_d becomes influential primarily at higher speeds through the quadratic drag force.

Posterior accuracy. Table 5.3 presents the results at $T_{\text{seg}} = 1000$. The accuracy gap between NPE and NPSE narrows compared to the μ -only case—both achieve nearly identical W_2 at 60 M budget, indicating that the added complexity of the two-parameter problem closes the gap between normalising flows and score-based diffusion. FNPSE shows a smaller relative gap to NPE and NPSE than in the μ -only case. Simformer has the highest W_2 but achieves the highest empirical coverage, through diffuse near-prior posteriors.

Table 5.3: Benchmark results for two-parameter estimation (μ, c_d) at $T_{\text{seg}} = 1000$. Values are mean $\pm$ s.d. across 5 independent seeds at $T_{\text{seg}} = 1000$. Results at $T_{\text{seg}} = 2000$ and $T_{\text{seg}} = 3000$ (2 seeds each) are reported in figures only and are interpreted as directional evidence; see Section 5.4.3 (Limitation 7). For error and coverage metrics, best mean values in **bold**; C2ST-prior is an informativeness diagnostic and is not bolded.

Method	Budget	$W_2 \downarrow$	$L^2 \downarrow$	Cov@90 $\uparrow$	Cov@50 $\uparrow$	C2ST-prior $\uparrow$	PPC RMSE $\downarrow$
NPE	20 M	0.157 $\pm$ 0.004	0.137 $\pm$ 0.004	0.450 $\pm$ 0.102	0.153 $\pm$ 0.067	0.962 $\pm$ 0.002	0.774
NPE	40 M	0.163 $\pm$ 0.009	0.145 $\pm$ 0.011	0.516 $\pm$ 0.054	0.226 $\pm$ 0.010	0.973 $\pm$ 0.002	0.776
NPE	60 M	0.163 $\pm$ 0.007	0.144 $\pm$ 0.007	0.465 $\pm$ 0.075	0.146 $\pm$ 0.074	0.974 $\pm$ 0.005	0.776
NPSE	20 M	0.162 $\pm$ 0.008	0.139 $\pm$ 0.010	0.384 $\pm$ 0.125	0.059 $\pm$ 0.036	0.951 $\pm$ 0.006	0.807
NPSE	40 M	0.158 $\pm$ 0.001	0.136 $\pm$ 0.003	0.511 $\pm$ 0.166	0.160 $\pm$ 0.083	0.958 $\pm$ 0.006	0.802
NPSE	60 M	0.160 $\pm$ 0.004	0.139 $\pm$ 0.004	0.486 $\pm$ 0.048	0.156 $\pm$ 0.035	0.959 $\pm$ 0.004	0.793
FNPSE	20 M	0.269 $\pm$ 0.095	0.213 $\pm$ 0.108	0.271 $\pm$ 0.357	0.108 $\pm$ 0.159	0.970 $\pm$ 0.043	2.128
FNPSE	40 M	0.318 $\pm$ 0.210	0.316 $\pm$ 0.211	0.135 $\pm$ 0.076	0.044 $\pm$ 0.024	0.982 $\pm$ 0.024	1.773
FNPSE	60 M	0.269 $\pm$ 0.049	0.267 $\pm$ 0.049	0.048 $\pm$ 0.070	0.016 $\pm$ 0.027	0.994 $\pm$ 0.007	2.932
Simformer	20 M	0.466 $\pm$ 0.116	0.350 $\pm$ 0.129	0.958 $\pm$ 0.053	0.321 $\pm$ 0.196	0.716 $\pm$ 0.095	1.060
Simformer	40 M	0.502 $\pm$ 0.120	0.370 $\pm$ 0.137	0.984 $\pm$ 0.020	0.423 $\pm$ 0.190	0.754 $\pm$ 0.128	1.059
Simformer	60 M	0.438 $\pm$ 0.040	0.270 $\pm$ 0.083	0.973 $\pm$ 0.052	0.535 $\pm$ 0.256	0.729 $\pm$ 0.072	1.313

Per-parameter analysis. The per-parameter breakdown at $T_{\text{seg}} = 1000$, $B = 60$ M reveals parameter-dependent identification quality. NPE achieves the best μ -RMSE, while Simformer has the widest posteriors but the best μ -coverage. Figure 5.8 shows the per-parameter breakdown.

For the drag coefficient c_d , all methods show a consistent negative bias, indicating that the inferred c_d is systematically lower than the true value. This bias likely reflects the weak identifiability of c_d at the moderate speeds present in the training scenarios. NPE and NPSE are overconfident on c_d (coverage far below nominal), while Simformer reaches near-perfect c_d -coverage by producing appropriately wide posteriors.

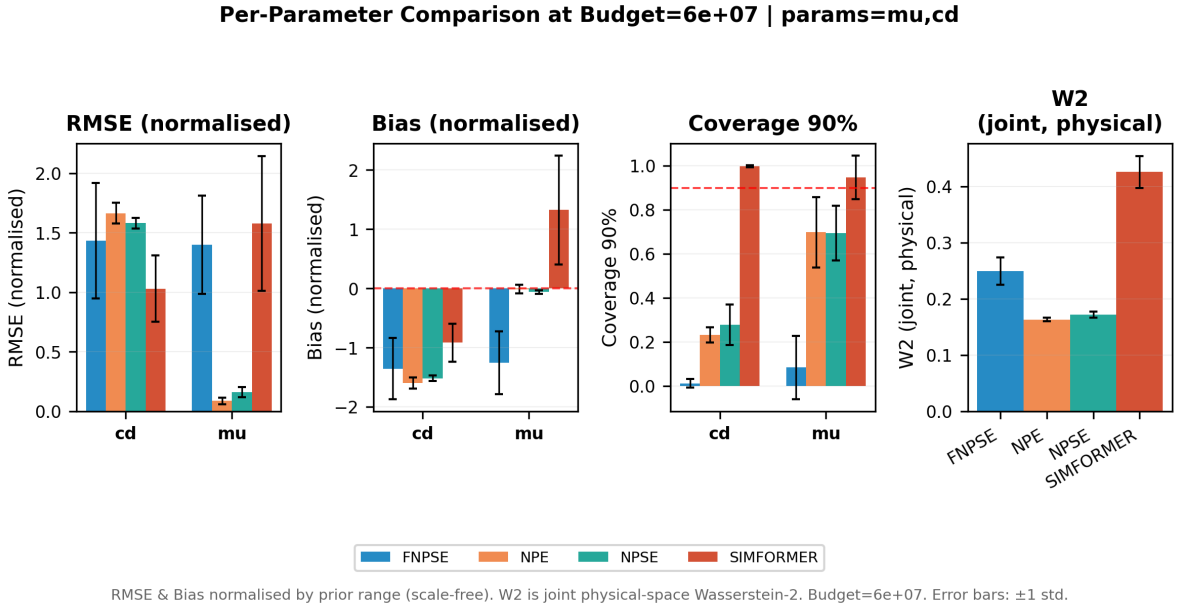


Figure 5.8: Per-parameter RMSE, bias, and coverage for the (μ, c_d) configuration. Left group: μ metrics; right group: c_d metrics. All methods exhibit negative c_d bias (-0.08 to -0.14). Simformer compensates with wide posteriors (high coverage), while NPE and NPSE are overconfident on c_d .

Budget scaling. A distinct pattern in the (μ, c_d) configuration is the saturation of NPE and NPSE performance: increasing the budget from 20 M to 60 M yields almost no improvement in W_2 . For NPE, W_2 remains at 0.157–0.163 across all three budget levels. This plateau indicates that the limiting factor is no longer the number of training samples but the identifiability of c_d given the driving scenario and noise model. FNPSE, in contrast, shows budget sensitivity (0.27–0.32), though this could reflect training instability rather than systematic improvement.

5.2.3 Two-Parameter Estimation (μ, m)

This subsection considers the (μ, m) configuration, which pairs the most identifiable parameter (friction) with vehicle mass while omitting the weakly-identified aerodynamic drag coefficient c_d . This combination tests whether the relative difficulty imposed by c_d in the two-parameter setting is a fundamental dimensionality effect or a c_d -specific identifiability problem.

Posterior accuracy. Table 5.4 presents aggregate results at $T_{\text{seg}} = 1000$. As in the three-parameter configuration, W_2 is in physical units and is dominated by the mass dimension. NPE achieves the best W_2 (68.3 at 20 M, improving to 29.9 at 60 M), substantially outperforming all other methods, and its PPC RMSE is the lowest (≈ 0.76 – 0.78). Coverage at 90% is moderate ($\text{Cov@90} = 0.61$ – 0.82) and improves with budget. Simformer achieves the highest Cov@90 (0.94 – 0.97) through its characteristic wide posteriors, at the cost of large W_2 (139–181). NPSE shows high W_2 (120–152) and variable Cov@90 (0.28 – 0.47), indicating imprecise posteriors. FNPSE remains the weakest method by Cov@50 (≈ 0.03 – 0.05) and shows high W_2 (118–157), consistent with its mode-collapse failure discussed in Section 5.4.3.

Table 5.4: Benchmark results for two-parameter estimation (μ, m) at $T_{\text{seg}} = 1000$. Values are mean $\pm$ s.d. across 5 independent seeds at $T_{\text{seg}} = 1000$. Results at $T_{\text{seg}} = 2000$ and $T_{\text{seg}} = 3000$ (2 seeds each) are reported in figures only and are interpreted as directional evidence; see Section 5.4.3 (Limitation 7). Joint W_2 is dominated by mass scale; per-parameter analysis in Figure 5.9. For error and coverage metrics, best mean values in **bold**; C2ST-prior is an informativeness diagnostic and is not bolded.

Method	Budget	$W_2 \downarrow$	$L^2 \downarrow$	$\text{Cov@90} \uparrow$	$\text{Cov@50} \uparrow$	C2ST-prior $\uparrow$	PPC RMSE $\downarrow$
NPE	20 M	68.3 $\pm$ 17.2	51.3 $\pm$ 17.0	0.606 $\pm$ 0.112	0.230 $\pm$ 0.081	0.955 $\pm$ 0.012	0.777
NPE	40 M	32.2 $\pm$ 4.6	19.6 $\pm$ 3.3	0.796 $\pm$ 0.098	0.386 $\pm$ 0.087	0.981 $\pm$ 0.006	0.759
NPE	60 M	29.9 $\pm$ 3.8	18.2 $\pm$ 4.8	0.822 $\pm$ 0.130	0.454 $\pm$ 0.195	0.984 $\pm$ 0.003	0.758
NPSE	20 M	152.5 $\pm$ 6.4	133.3 $\pm$ 7.6	0.437 $\pm$ 0.110	0.131 $\pm$ 0.076	0.933 $\pm$ 0.013	0.873
NPSE	40 M	147.3 $\pm$ 15.9	128.2 $\pm$ 18.3	0.284 $\pm$ 0.138	0.023 $\pm$ 0.034	0.915 $\pm$ 0.013	0.868
NPSE	60 M	120.3 $\pm$ 34.5	100.4 $\pm$ 37.4	0.470 $\pm$ 0.226	0.165 $\pm$ 0.126	0.934 $\pm$ 0.008	0.841
FNPSE	20 M	157.4 $\pm$ 49.6	156.7 $\pm$ 50.0	0.111 $\pm$ 0.093	0.036 $\pm$ 0.032	0.979 $\pm$ 0.015	1.498
FNPSE	40 M	118.1 $\pm$ 72.5	117.2 $\pm$ 73.1	0.125 $\pm$ 0.095	0.054 $\pm$ 0.038	0.990 $\pm$ 0.009	2.040
FNPSE	60 M	129.8 $\pm$ 45.0	129.4 $\pm$ 45.1	0.076 $\pm$ 0.130	0.028 $\pm$ 0.054	0.984 $\pm$ 0.026	2.859
Simformer	20 M	138.7 $\pm$ 27.1	65.6 $\pm$ 34.4	0.942 $\pm$ 0.061	0.356 $\pm$ 0.095	0.657 $\pm$ 0.077	1.029
Simformer	40 M	170.8 $\pm$ 25.4	91.1 $\pm$ 46.3	0.971 $\pm$ 0.035	0.376 $\pm$ 0.286	0.698 $\pm$ 0.097	1.203
Simformer	60 M	181.3 $\pm$ 23.9	107.1 $\pm$ 35.6	0.970 $\pm$ 0.027	0.475 $\pm$ 0.289	0.663 $\pm$ 0.073	1.373

Effect of removing c_d . Comparing the (μ, m) results with the (μ, c_d) results (Table 5.3) reveals a parameter-dependent pattern. Removing c_d does not benefit FNPSE: its Cov@90 remains near-zero (0.08–0.13) in the (μ, m) case, mirroring the near-zero coverage in the (μ, c_d) case. The mode-collapse failure is not driven by c_d identifiability but is a persistent failure mode. Notably, NPSE is most affected by the parameter swap: while NPSE matches NPE in W_2 for the (μ, c_d) setting (0.158–0.162 vs. 0.157–0.163), it shows a large W_2 gap in the (μ, m) setting (120–152 vs. 30–68), suggesting that score-based sequential methods struggle to integrate the trajectory-level mass information. The overall method ranking-NPE best accuracy, Simformer best coverage-is preserved.

Per-parameter analysis. Figure 5.9 shows the per-parameter breakdown at $B = 60$ M, $T_{\text{seg}} = 1000$. NPE achieves the lowest RMSE for both μ and m ; NPSE achieves better coverage on m at the cost of broader posteriors. FNPSE shows the characteristic pattern of low RMSE but near-zero Cov@50, reflecting its collapsed posteriors. Figure 5.10 shows the corresponding calibration diagnostics for the (μ, m) configuration.

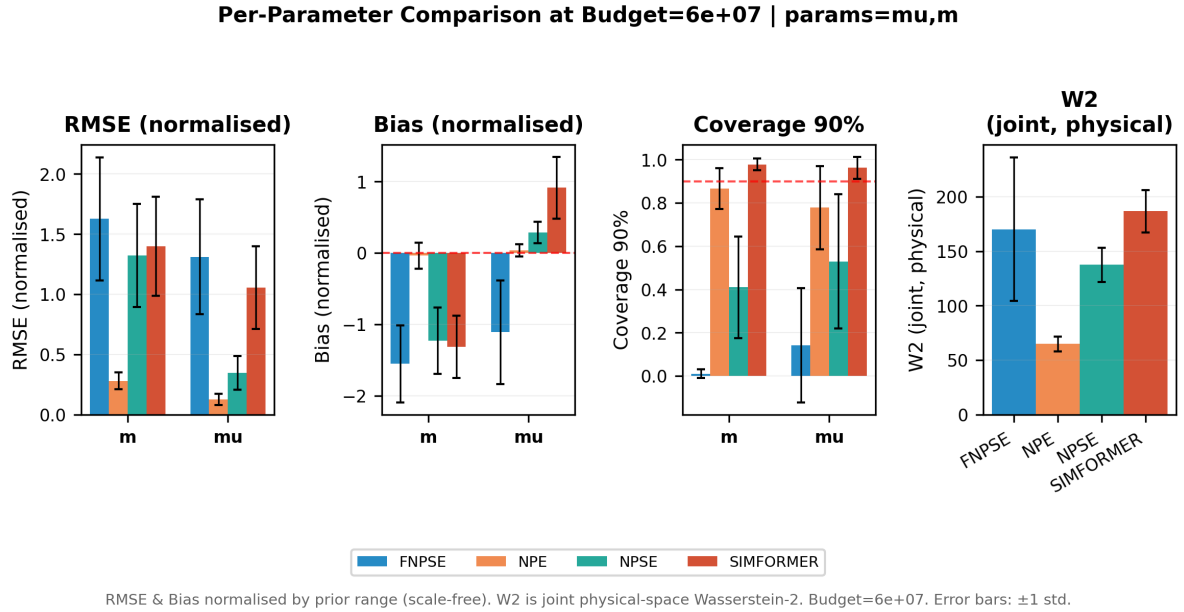


Figure 5.9: Per-parameter RMSE, bias, and coverage for the (μ, m) configuration at $B = 60$ M, $T_{\text{seg}} = 1000$. Left group: μ metrics; right group: m metrics. NPE achieves the lowest RMSE for both parameters. FNPSE shows low per-parameter RMSE but near-zero coverage, consistent with its mode-collapse failure.

Calibration & Posterior Quality Diagnostics | params= μ, m

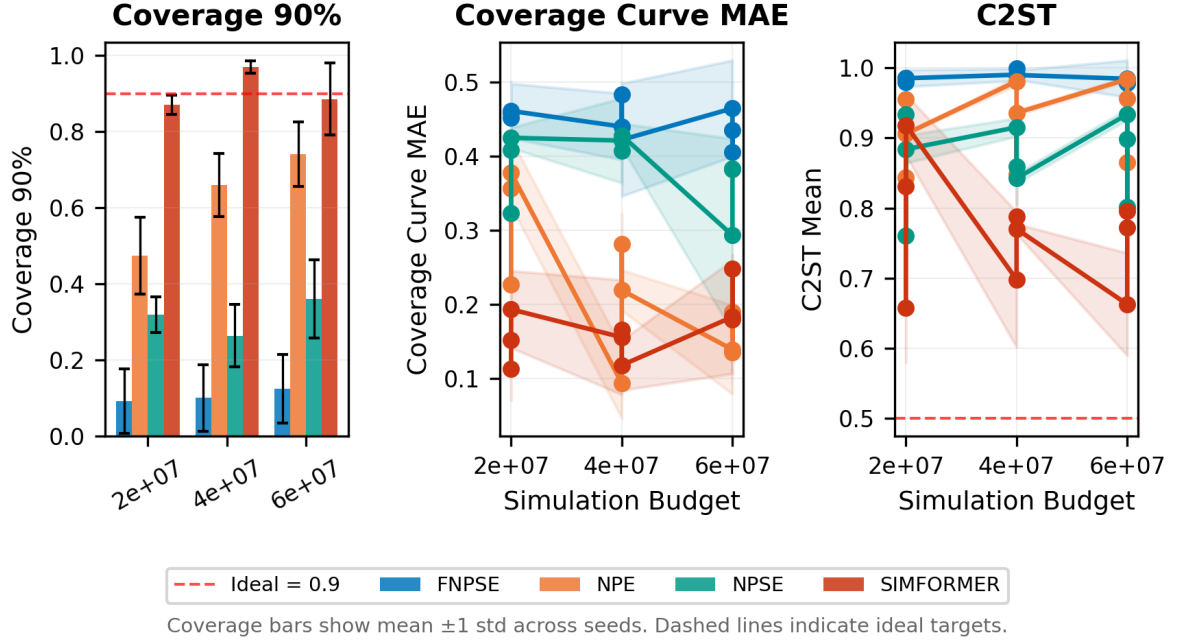


Figure 5.10: Calibration diagnostics for (μ, m) estimation at three simulation budgets. **Left:** 90% empirical coverage; each bar shows mean ± 1 standard deviation across seeds. **Middle:** Coverage Curve MAE (lower is better). **Right:** Prior-Separation C2ST (higher = more informative). Simformer achieves the highest empirical coverage; NPE and NPSE produce more concentrated posteriors at the cost of empirical coverage; FNPSE undercovers across all budgets.

5.2.4 Three-Parameter Estimation (μ, c_d, m)

The full three-parameter configuration adds the vehicle mass m to the inference target, the most challenging estimation problem in this benchmark, with the highest parameter dimensionality and the strongest inter-parameter correlations. Mass affects the translational dynamics through Newton’s second law and also shifts the static tire loads, creating interactions with friction and drag.

Scale separation in metrics. A practical issue in the three-parameter case is the large difference in physical scales: $\mu \in [0.50, 1.50]$, $c_d \in [0.05, 0.60]$, and $m \in [1700, 2200]$ kg (Table 3.4). Joint metrics such as W_2 are dominated by the mass dimension because the errors are measured in physical units. Therefore, a W_2 value of 86.7 (Table 5.5) does not indicate poor

inference—it reflects primarily the mass error in kilograms. Per-parameter metrics are essential for interpretation.

Table 5.5: Benchmark results for three-parameter estimation (μ , c_d , m) at $T_{\text{seg}} = 1000$. Values are mean $\pm$ s.d. across 5 independent seeds at $T_{\text{seg}} = 1000$. Results at $T_{\text{seg}} = 2000$ and $T_{\text{seg}} = 3000$ (2 seeds each) are reported in figures only and are interpreted as directional evidence; see Section 5.4.3 (Limitation 7). Joint W_2 is dominated by mass scale (kg); per-parameter analysis in text. For error and coverage metrics, best mean values in **bold**; C2ST-prior is an informativeness diagnostic and is not bolded.

Method	Budget	$W_2 \downarrow$	$L^2 \downarrow$	Cov@90 $\uparrow$	Cov@50 $\uparrow$	C2ST-prior $\uparrow$	PPC RMSE $\downarrow$
NPE	20 M	86.7 $\pm$ 27.2	66.3 $\pm$ 31.6	0.636 $\pm$ 0.179	0.176 $\pm$ 0.116	0.961 $\pm$ 0.010	1.086
NPE	40 M	42.5 $\pm$ 6.6	27.6 $\pm$ 7.8	0.745 $\pm$ 0.066	0.276 $\pm$ 0.059	0.985 $\pm$ 0.002	0.943
NPE	60 M	29.6 $\pm$ 4.1	17.1 $\pm$ 3.7	0.794 $\pm$ 0.041	0.323 $\pm$ 0.033	0.989 $\pm$ 0.002	0.936
NPSE	20 M	149.2 $\pm$ 5.3	124.3 $\pm$ 6.3	0.652 $\pm$ 0.077	0.083 $\pm$ 0.041	0.932 $\pm$ 0.017	1.205
NPSE	40 M	146.6 $\pm$ 17.6	122.2 $\pm$ 21.5	0.673 $\pm$ 0.094	0.142 $\pm$ 0.074	0.926 $\pm$ 0.007	1.160
NPSE	60 M	147.7 $\pm$ 15.9	124.5 $\pm$ 18.4	0.697 $\pm$ 0.115	0.138 $\pm$ 0.106	0.917 $\pm$ 0.023	1.141
FNPSE	20 M	184.1 $\pm$ 76.7	183.1 $\pm$ 76.5	0.093 $\pm$ 0.083	0.034 $\pm$ 0.034	0.995 $\pm$ 0.003	2.186
FNPSE	40 M	109.6 $\pm$ 34.9	108.8 $\pm$ 35.1	0.051 $\pm$ 0.062	0.018 $\pm$ 0.022	0.995 $\pm$ 0.004	2.533
FNPSE	60 M	125.0 $\pm$ 87.4	123.4 $\pm$ 88.6	0.130 $\pm$ 0.141	0.057 $\pm$ 0.055	0.996 $\pm$ 0.004	2.025
Simformer	20 M	179.7 $\pm$ 13.5	90.9 $\pm$ 34.7	0.981 $\pm$ 0.012	0.557 $\pm$ 0.238	0.788 $\pm$ 0.072	1.666
Simformer	40 M	183.9 $\pm$ 34.0	129.6 $\pm$ 34.5	0.980 $\pm$ 0.012	0.323 $\pm$ 0.169	0.748 $\pm$ 0.026	1.680
Simformer	60 M	178.3 $\pm$ 27.3	93.6 $\pm$ 50.7	0.972 $\pm$ 0.021	0.479 $\pm$ 0.197	0.825 $\pm$ 0.046	1.612

Per-parameter analysis. At $T_{\text{seg}} = 1000$, $B = 60$ M, the methods differ substantially in their ability to estimate each parameter. Figure 5.11 shows the per-parameter breakdown; Figure 5.12 shows the joint W_2 across all budget levels.

- **Friction μ :** NPE achieves the best accuracy–coverage trade-off. Simformer produces wider posteriors with the highest empirical coverage. FNPSE is severely overconfident.
- **Drag c_d :** All methods continue to show negative bias. FNPSE’s two-step Markov windows do not capture the drag coefficient in this setting. Simformer reaches the best c_d -coverage.
- **Mass m :** This is the most challenging parameter. NPE achieves the best trade-off between accuracy and calibration. NPSE and FNPSE systematically underestimate the mass with large negative bias, indicating that the score network and the Markov factorisation both struggle to integrate trajectory-level mass information. Simformer achieves the best m -coverage by producing appropriately wide posteriors.

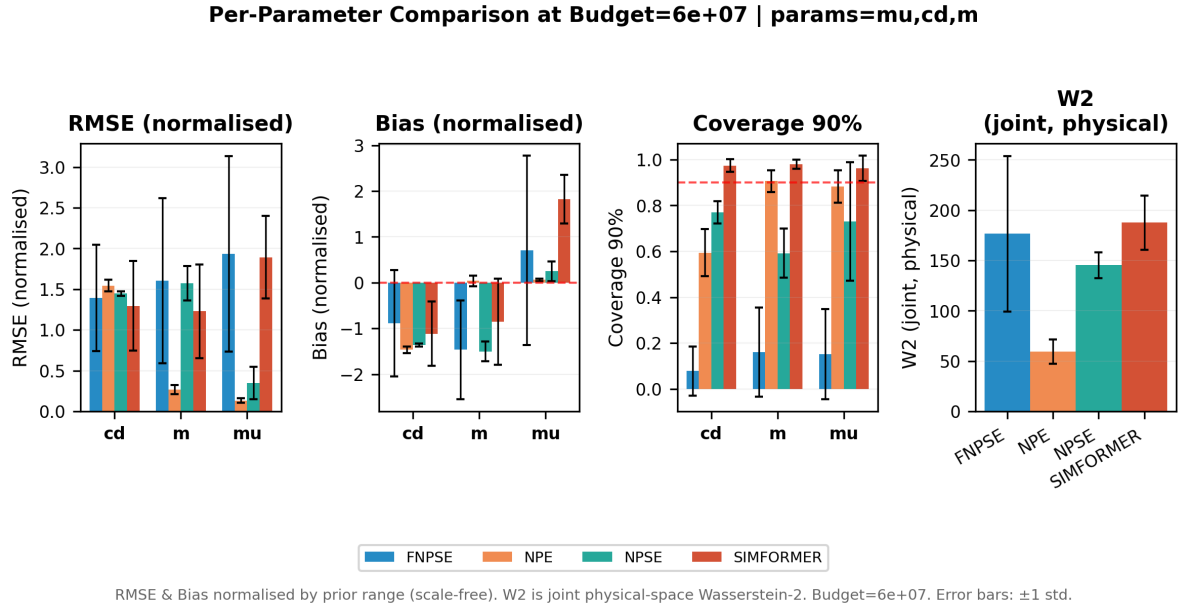


Figure 5.11: Per-parameter RMSE, bias, and coverage for the (μ, c_d, m) configuration. The mass parameter dominates the overall error and is poorly estimated by NPSE and FNPSE. Simformer achieves near-nominal mass coverage by producing appropriately wide posteriors.

Mass identifiability. The difficulty of estimating mass deserves special attention. Unlike friction, which directly affects the dominant tire forces during braking and cornering, mass enters the equations of motion as a scalar multiplier that is partially degenerate with friction: a higher mass (requiring more force to decelerate) can produce similar observations to a lower friction coefficient (producing less force at the tire). This parameter correlation makes the joint estimation substantially harder in this model.

FNPSE is most affected by this degeneracy because its Markov factorisation decomposes the trajectory into short overlapping windows. Within a single two-step window, the effect of mass on the trajectory is small compared to friction, making mass nearly unidentifiable from local dynamics alone. The global normalising flow or score models used by NPE, NPSE, and Simformer can, in principle, integrate information across the full trajectory to resolve the friction–mass ambiguity, though NPSE does not effectively integrate trajectory-level mass information in this setting, achieving m -RMSE of 130.3 kg compared to NPE’s 22.0 kg.

Budget scaling. NPE shows the strongest budget scaling in the three-parameter case: W_2 drops from 86.7 (20 M) to 29.6 (60 M), a 66% reduction. This demonstrates that with sufficient

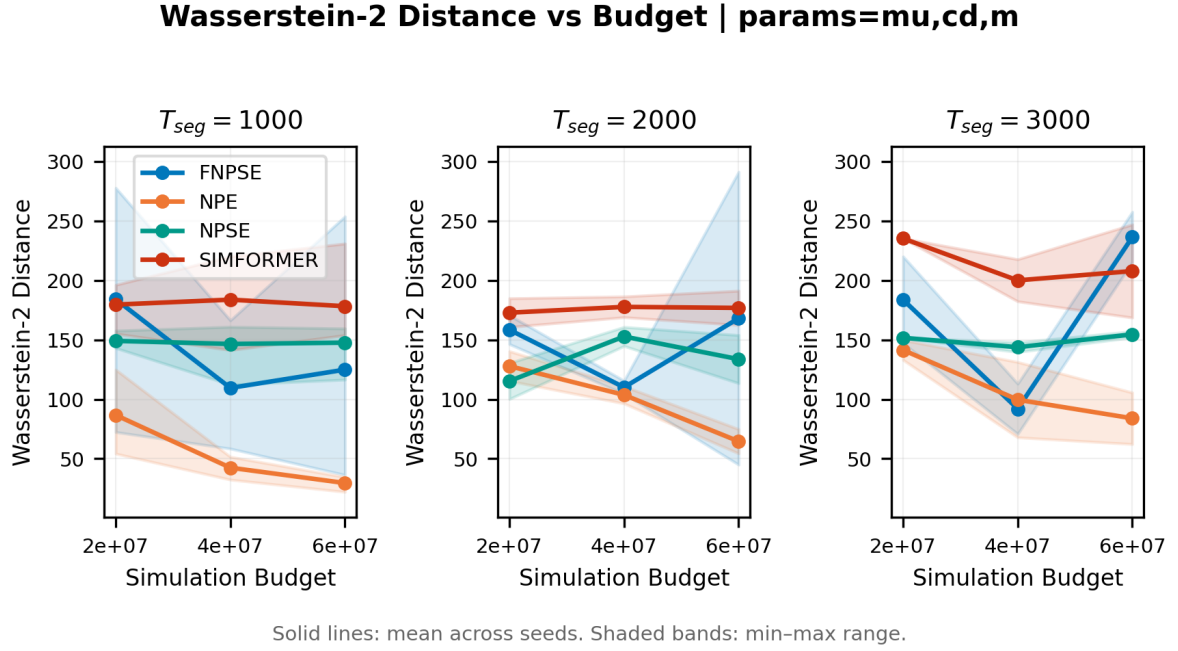


Figure 5.12: Joint parameter-recovery W_2 for three-parameter estimation (μ, c_d, m). The large absolute values are dominated by the mass dimension (errors in kg). NPE achieves the lowest parameter-recovery W_2 at all budget levels; budget scaling is most effective for NPE.

data, NPE can learn the complex three-parameter posterior. NPSE, in contrast, shows almost no improvement with budget (147–149 across all three levels), suggesting that its training saturates early on this problem. FNPSE shows some improvement from 20 M to 40 M (184 $\rightarrow$ 110) but worsens again at 60 M (125), indicating training instability. Simformer’s W_2 is stable around 178–184 regardless of budget.

5.2.5 Cross-Configuration Analysis

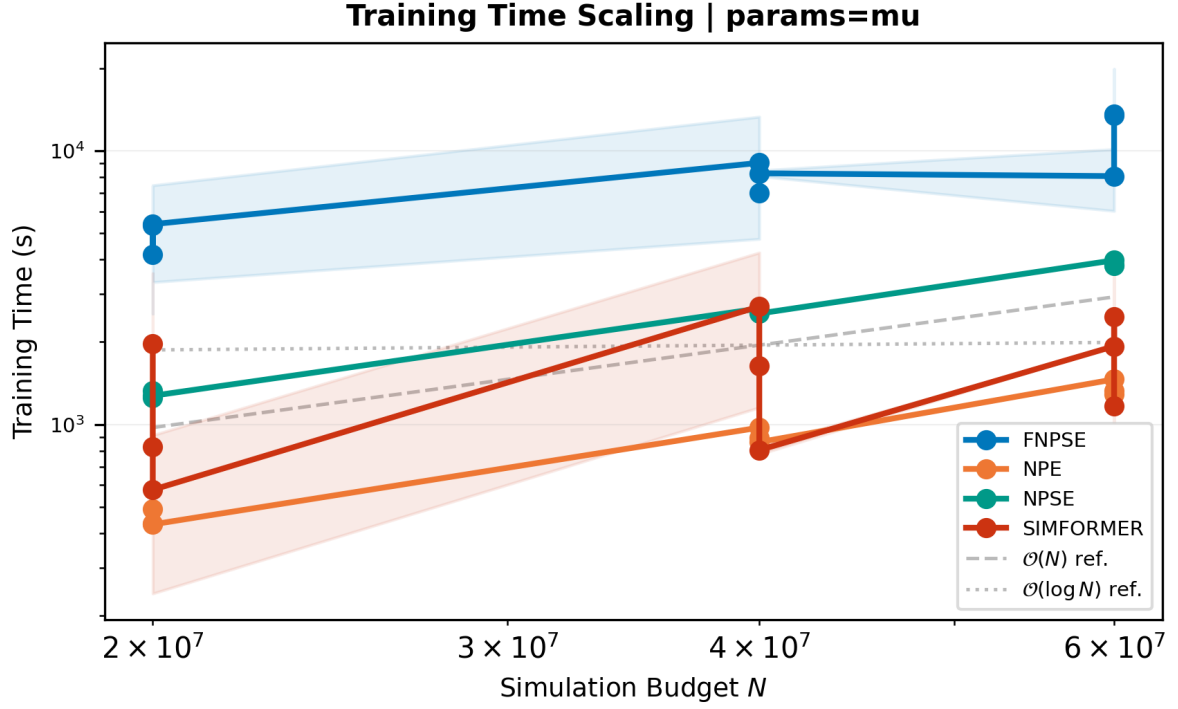
Effect of parameter count. Increasing the number of inferred parameters degrades posterior accuracy systematically. For NPE at $T_{seg} = 1000$, $B = 60$ M, the normalised W_2 in prior-scaled space increases from 0.017 (μ only) to 0.163 (μ, c_d), representing a tenfold degradation. Adding mass further increases the physical-space joint W_2 to 29.6, dominated by the mass dimension; per-parameter normalised metrics are needed for fair comparison. This degradation is not simply a dimension effect: the parameters differ in their identifiability, and each additional weakly-identified parameter degrades the estimation of the well-identified ones. For example, the μ -RMSE increases from 0.011 (μ only) to 0.014 (μ, c_d) to 0.022 (μ, c_d, m), even though μ

is always the best-identified parameter.

Computational cost and runtime. Training time scales approximately linearly with budget for all methods. NPE and NPSE are the fastest to train, while FNPSE requires $3\text{--}10\times$ more time due to per-window score model training with internal trajectory generation. Figure 5.13 plots training time versus budget.

Sampling time is where the methods differ most substantially. Figures 5.14 and 5.15 show the W_2 –training-time and PPC-RMSE–sampling-time Pareto fronts respectively. Figure 5.16 provides a heatmap summary of all key metrics at $B = 60$ M. NPE samples a full posterior in under 0.05 s by a single forward pass through the normalising flow (measured wall-clock times per checkpoint are given in Table 5.9; the forward-pass component alone is under 0.01 s per sample on GPU). The diffusion-based methods (NPSE, FNPSE, Simformer) are orders of magnitude slower due to the iterative reverse-SDE integration, with Simformer being the slowest because of transformer attention over all observation tokens at each diffusion step.

For deployment in real-time or near-real-time settings, the sampling time is the critical constraint. NPE’s fast sampling (single forward pass < 0.01 s per sample on GPU; Table 5.9 reports full posterior wall-clock times per checkpoint) makes it the only method suitable for online parameter estimation. NPSE at 1–5 s per posterior is feasible for offline batch processing. Simformer’s 163–175 s per posterior (see Table 5.9) and FNPSE’s 4–20 s make them impractical for high-throughput applications without substantial computational investment or architectural optimisation.



Dashed & dotted lines show $\mathcal{O}(N)$ and $\mathcal{O}(\log N)$ reference scaling from median values.

Figure 5.13: Training wall-clock time as a function of simulation budget for μ -only estimation. Each method contributes three data points per budget level (one per $T_{\text{seg}} \in \{1000, 2000, 3000\}$); lines connect method means and shaded bands span one standard deviation across seeds. Reference lines show $\mathcal{O}(N)$ (dashed) and $\mathcal{O}(\log N)$ (dotted) scaling anchored at the median. NPE and NPSE scale approximately linearly; FNPSE’s large overhead and high variance arise from per-window score model training with internal simulation; Simformer’s high variance reflects GPU batch-size interactions.



Figure 5.14: Held-out W_2 versus training time for μ -only estimation, shown as three subplots—one per simulation budget ($B \in \{2, 4, 6\} \times 10^7$). Colour encodes method; marker shape encodes trajectory length ($\bigcirc T_{\text{seg}} = 1000$; $\square T_{\text{seg}} = 2000$; $\triangle T_{\text{seg}} = 3000$). The dashed grey line marks the Pareto-optimal configurations (non-dominated set). At all budgets, the Pareto front consists exclusively of NPE configurations (orange), which achieve the lowest W_2 at the shortest training time; no other method is non-dominated.

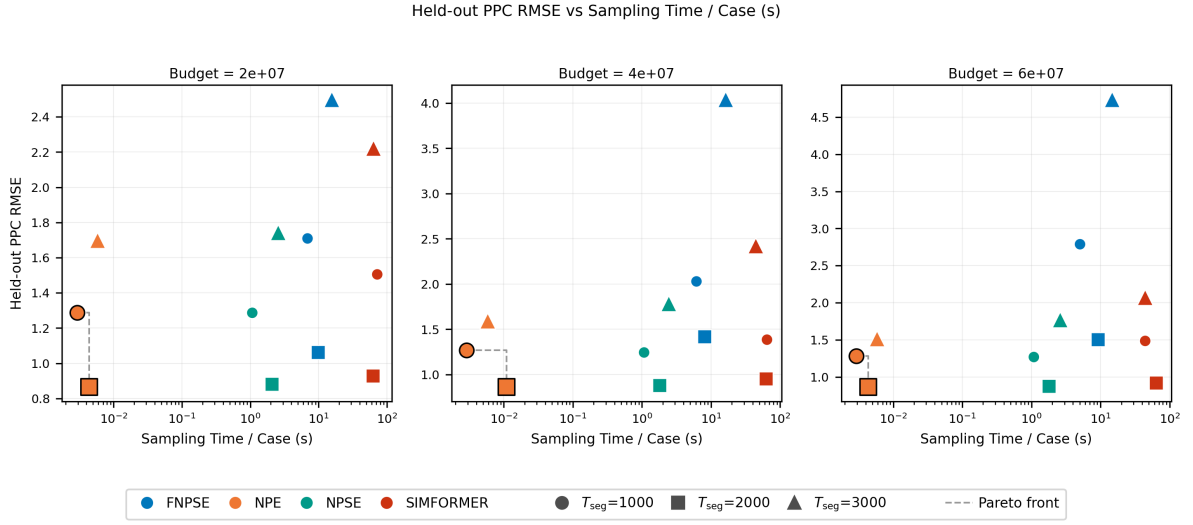
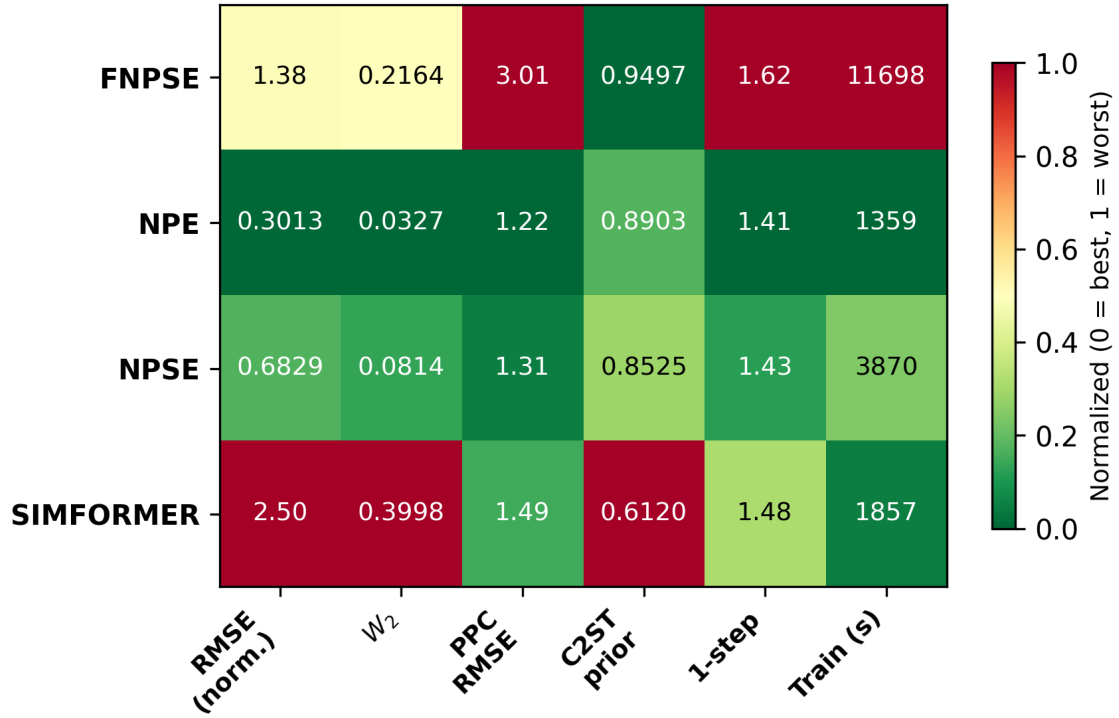


Figure 5.15: Held-out PPC RMSE versus posterior sampling time per case for μ -only estimation, shown as three subplots—one per simulation budget. Colour encodes method; marker shape encodes trajectory length ($\circ$ $T_{\text{seg}} = 1000$; $\square$ $T_{\text{seg}} = 2000$; $\triangle$ $T_{\text{seg}} = 3000$). The dashed grey line marks the Pareto-optimal configurations. At every budget level, the Pareto front consists exclusively of NPE configurations (orange): NPE achieves the lowest PPC RMSE with near-instantaneous sampling (single forward pass < 0.01 s per sample on GPU; full posterior wall-clock times are reported in Table 5.9). All diffusion-based methods (NPSE, FNPSE, Simformer) are 2–4 orders of magnitude slower due to iterative reverse-SDE integration, and none reaches NPE’s PPC RMSE at any trajectory length.

Metric Summary — Budget=6e+07 | params=mu | T_seg: averaged

RMSE(norm) = avg per-param RMSE $\div$ prior range. W_2 (joint) = Wasserstein-2 vs ground truth (physical space). Green = best.

Figure 5.16: Heatmap of key metrics at $B = 60$ M for μ -only estimation. Each cell shows the mean metric value for a method– T_{seg} combination. Green indicates better performance for error, PPC RMSE, and runtime metrics. For C2ST-prior, green indicates higher prior separation (more informative posterior, further from the prior). The heatmap confirms NPE’s strength in accuracy metrics and Simformer’s high empirical coverage, driven by diffuse near-prior posteriors.

5.3 Real-World Comparison

This section transitions from the controlled simulation benchmark to real vehicle measurement data. Before applying the trained SBI methods to real recordings, the simulator itself must be calibrated so that it can reproduce the measured signals with sufficient fidelity. Section 5.3.1 describes a global parameter optimisation that serves this purpose, while Section 5.3.2 presents the SBI-based inference results on real data.

5.3.1 Simulator Calibration via PSO

The benchmark experiments in Section 5.2 operate entirely within the simulator: the training data and the test observations are all generated by the same two-track model. When moving to real measurements, a sim-to-real gap is inevitable because the simulator’s default parameter values (Tables 3.1–3.3) are approximate. To assess and reduce this gap, a global parameter optimisation is performed in which 23 simulator parameters are fitted jointly to seven real brake-test recordings from the Jeversen proving ground (Section 3.6).

This calibration step serves two purposes. First, it provides a set of globally optimised model parameters under which the simulator reproduces the measured signals as closely as possible, providing a best-achieved in-sample calibration error under the chosen optimisation setup. Second, it yields per-trajectory friction estimates that can later be compared with the SBI posterior distributions to check whether the SBI methods recover plausible values.

Method

The optimisation uses PSO [KE95], a population-based metaheuristic that maintains a swarm of candidate solutions and iteratively updates them based on each particle’s personal best position and the swarm’s global best. PSO is well suited to this problem because the objective landscape is non-convex and high-dimensional, and gradient information is unavailable (the simulator is treated as a black box).

The parameter vector comprises 16 global parameters shared across all trajectories and one per-trajectory friction coefficient μ_k for each of the $K = 7$ recordings, yielding 23 parameters in total. The global parameters include the vehicle mass m , yaw inertia I_z , tire and engine inertias I_{tire} and I_{engine} , the aerodynamic drag coefficient, all eight Pacejka tire model coefficients ($c_{1,x}$, $c_{2,x}$, C_x , E_x , $c_{1,y}$, $c_{2,y}$, C_y , E_y), both rolling resistance coefficients, and the effective tire radius. The per-trajectory friction coefficient allows the optimiser to account for the different road surfaces without constraining all recordings to share a single friction value.

The swarm uses 200 particles for 3 000 iterations, with three standard PSO enhancements: an adaptive inertia weight ($w = 0.9 \rightarrow 0.4$ over the run), a switch from a fully-connected star to a local ring neighbourhood topology at 70% of iterations, and stagnation-based reinitialisation of 50% of particles after 30 stalled iterations.

The objective function evaluates the NRMSE between the simulated and measured signals, averaged across all seven trajectories. Nine observation channels are used: yaw rate, longitudinal and lateral global-frame velocities, longitudinal and lateral body-frame accelerations, and four wheel speeds. The body-level channels receive unit weight, while the wheel-speed channels are weighted by a factor of 0.5 to prevent the tire dynamics from dominating the objective.

Parameters spanning several orders of magnitude (mass, inertias, Pacejka stiffness coefficients) are optimised on a logarithmic scale; the rest on a linear scale. Each simulation segment uses $T_{\text{seg}} = 800$ steps (8 s at 100 Hz) starting at index 50. The optimisation required approximately 605 000 function evaluations and completed in 46.1 h on a single CPU.

Results

Figure 5.17 shows the PSO convergence history. Rapid initial progress (NRMSE = 0.232 at iteration 1, decreasing to 0.194 at iteration 750) is followed by gradual refinement (0.186 at iteration 1 500, 0.185 at iteration 3 000). The majority of the improvement is concentrated in the first quarter of the iteration budget, with strongly diminishing returns thereafter, which is consistent with the optimiser finding a good basin early and then refining within it. The final NRMSE of 0.185 is evaluated on the seven recordings used for optimisation (in-sample fit); no held-out data are available for this calibration step, so generalisation to unseen recordings cannot be assessed directly from this number.

Table 5.6 compares the optimised global parameters with the simulator defaults from Tables 3.1–3.3. Several parameters shift substantially. These shifts indicate that the optimiser compensates for modelling simplifications (e.g., the absence of dynamic load transfer, simplified tire–road contact) by reshaping the tire force characteristics globally. The default vehicle mass of 1 720 kg corresponds to the kerb weight of the test vehicle as specified in its technical documentation. It differs from the centre of the SBI prior (1 950 kg, midpoint of [1700, 2200] kg) because the prior was designed to span the physically plausible operating range-including payload, passenger weight, and configuration variations-rather than to encode knowledge of the specific vehicle used in the Jeversen recordings.

Table 5.7 lists the per-trajectory friction coefficients. The asphalt recordings yield $\mu = 1.40$ and 1.31, the concrete recordings yield $\mu = 1.42$ and 1.16, both basalt recordings saturate at the upper bound $\mu = 1.60$, and the mixed-surface recording yields $\mu = 0.41$.

Table 5.6: Global simulator parameters: default values vs. PSO-optimised values. Parameters with large relative changes are highlighted.

Parameter	Default	Optimised	Ratio
m [kg]	1 720	1 200	0.70
I_z [kg m ²]	2 066	1 440	0.70
I_{tire} [kg m ²]	28.6	5.0	0.17
I_{engine} [kg m ²]	0.197	0.05	0.25
c_d (air resistance)	0.270	0.179	0.66
r_w [m]	0.3116	0.3138	1.01
$c_{1,x}$	2.50×10^7	3.80×10^7	1.52
$c_{2,x}$	3.60×10^6	8.36×10^4	0.02
C_x	1.42	3.97	2.80
E_x	-9.75	-15.0	1.54
$c_{1,y}$	1.90×10^6	2.00×10^6	1.05
$c_{2,y}$	1.35×10^5	9.99×10^6	74.0
C_y	1.90	5.00	2.63
E_y	0.52	-2.05	-
$C_{\text{roll},1}$	0.0083	0.050	6.0
$C_{\text{roll},2}$	0.0005	1.19×10^{-4}	0.24

Table 5.7: Per-trajectory friction coefficients from PSO calibration.

Recording	Surface	μ
Asphalt recording 1	Asphalt	1.40
Asphalt recording 2	Asphalt	1.31
Concrete recording 1	Concrete	1.42
Concrete recording 2	Concrete	1.16
Basalt recording 1	Basalt	1.60
Basalt recording 2	Basalt	1.60
Mixed recording	Mixed	0.41

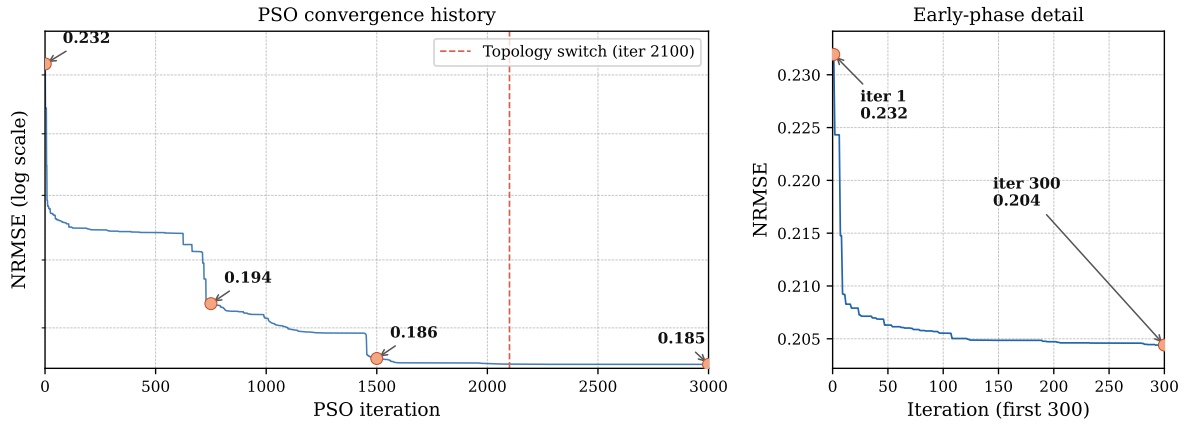


Figure 5.17: PSO convergence history over 3 000 iterations. Left: NRMSE on log scale for the full run; the red dashed line marks the topology switch from a fully-connected star to a local ring topology at iteration 2 100 (70% of the budget). Annotated values show the NRMSE at key milestones. Right: linear-scale detail of the first 300 iterations, showing the rapid initial improvement.

Interpretation of optimised parameters

Five of the 16 global parameters reach a search bound: vehicle mass ($m = 1\,200\text{ kg}$), tire inertia ($I_{\text{tire}} = 5.0\text{ kg m}^2$), engine inertia ($I_{\text{engine}} = 0.05\text{ kg m}^2$), and the lateral stiffness factor $C_y = 5.00$, all at bounds, with $c_{2,y}$ effectively at its upper bound of 10^7 . Among the per-trajectory parameters, both basalt recordings saturate at $\mu = 1.60$ (the extended PSO upper bound; the SBI prior’s design upper bound is 1.50). Two diagnostic patterns follow.

First, simultaneous boundary saturation indicates that the global optimum lies outside the feasible region, so the optimiser settles at a corner of the search space. The most physically suggestive case is the vehicle mass dropping 30% below the nominal kerb weight: the two-track model omits dynamic load transfer during braking, and lower mass increases the simulated deceleration for a given torque, partially compensating for the missing physics.

Second, large simultaneous changes in the Pacejka coefficients (C_x , C_y , $c_{2,y}$, E_x) globally reshape the tire force curves. Within this reshaped curve family, the per-trajectory μ no longer carries its nominal peak-friction interpretation. The basalt recordings yielding $\mu = 1.60$, above the asphalt and concrete recordings (1.16–1.42), is a concrete illustration: basalt is physically the lowest-friction surface in this set, so the ordering inversion reflects compensation by global shape changes rather than a plausible friction value. As a consequence, the PSO per-trajectory μ values cannot be used as ground truth for validating SBI-inferred friction.

The combination of these signals—five global parameters at their bounds, a friction inversion

across surfaces, and large global Pacejka reshaping—is the classical signature of a misspecified model being twisted to fit data within constraints. The substantive conclusion of the PSO calibration is therefore not that the optimised values are correct, but that the two-track model in its current form cannot reproduce the real measurements without compensating distortions of the tire-force characteristics. This finding should be interpreted as a structural limitation of the simulator, not as a limitation of the optimiser.

Implications. The PSO-calibrated global parameters provide a simulator configuration under which the in-sample NRMSE across all channels and trajectories reaches 0.185 (see the Results section above). This confirms that the two-track model is structurally capable of reproducing the gross features of the measured dynamics; 0.185 is the best achieved in-sample calibration error under this optimisation setup and does not constitute a theoretical lower bound. The calibrated simulator achieves substantially lower in-sample fitting error than the default parametrisation, but the boundary-saturation pattern (Section 5.3.1) and the surface-friction inversion (Table 5.7, basalt $\mu = 1.60$ above asphalt $\mu = 1.31$ – 1.40) indicate that the calibrated parameter values are not individually physically meaningful—they are joint compensation values for an under-specified model. The calibrated configuration is therefore best understood as a higher-fidelity *fitting object* for predictive purposes (e.g., generating SBI training data whose marginal distributions of observable signals are closer to the real measurements) rather than as a higher-fidelity *physical model*. Using it as a forward model for future SBI training would improve the marginal distribution of the simulated training signals but would not correct the structural mismatch revealed by the boundary saturations. In the present thesis, however, the real-data SBI evaluation uses the default simulator parameters to remain consistent with the simulative benchmark.

5.3.2 SBI-Based Real-Data Inference

All experiments below use the *default* simulator parameters, not the PSO-calibrated values, for consistency with the simulative benchmark. The sim-to-real gap is therefore larger than it would be under the calibrated model. Figure 5.18 illustrates this gap between default and PSO-calibrated simulation against one Asphalt recording.

Experimental Setup

For each method, the best-performing checkpoint from the simulative benchmark (Section 5.2) at $T_{\text{seg}} = 1000$ was selected based on the lowest W_2 distance across the available budget–seed combinations. Table 5.8 lists the 12 resulting experiments.

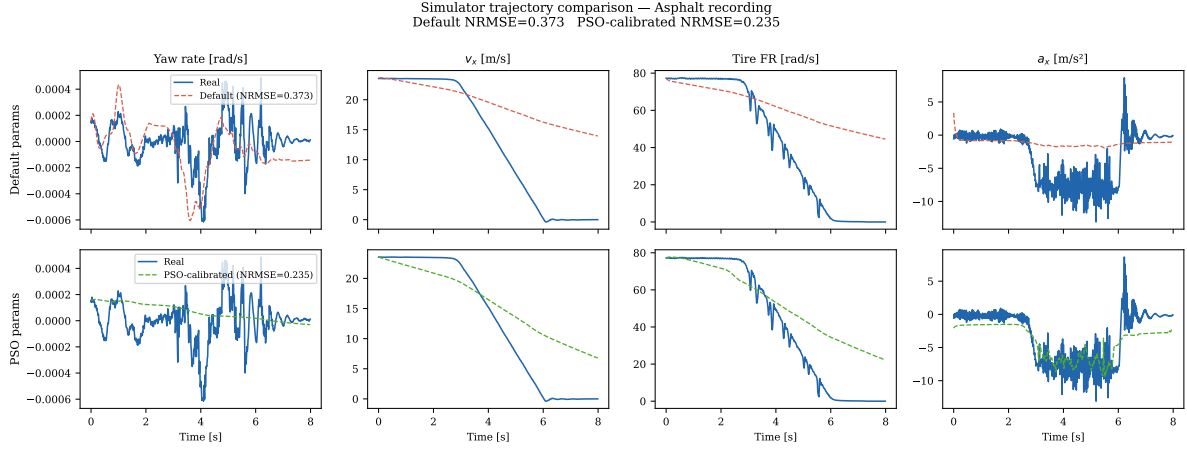


Figure 5.18: Simulated vs. real trajectory comparison for one representative Asphalt recording. Top row: default simulator parameters (NRMSE = 0.373). Bottom row: PSO-calibrated parameters (NRMSE = 0.235). Each column shows one observation channel (yaw rate, v_x , tire speed FR, a_x). Blue: real measurement. Orange dashed: default simulation. Green dashed: PSO-calibrated simulation. The calibrated model more closely tracks the real signal, particularly in wheel-speed dynamics during braking, though residual mismatches in the transient phases persist due to unmodelled load-transfer effects.

Each experiment follows a two-stage protocol. First, the loaded checkpoint is evaluated on a held-out set of 100 simulated test cases to obtain simulative diagnostics (W_2 , coverage, rank uniformity). Second, the posterior is applied to each of the seven real recordings, and PPC trajectories are generated by sampling parameters from the posterior and running the simulator forward.

Simulative Diagnostics of Selected Checkpoints

Table 5.9 reports the simulative quality of the checkpoints used for real-data inference. These numbers provide context for interpreting the real-data results: a method with poor simulative calibration is unlikely to produce reliable posteriors on real data.

The simulative diagnostics are consistent with the full benchmark results from Section 5.2. NPE achieves the lowest W_2 across all configurations. FNPSE exhibits substantial overconfidence: its posteriors are considerably narrower than those of NPE and achieve near-zero empirical coverage. Simformer shows the opposite pattern—overdispersed posteriors with near-perfect 90%-coverage but very low 50%-coverage, indicating that the posterior mass is spread wide rather than concentrated near the true value.

Table 5.8: Real-data evaluation grid. Each row corresponds to one experiment using the best benchmark checkpoint at $T_{\text{seg}} = 1000$ for that method–parameter combination.

Method	Parameters	Budget	N_{sim}
NPE	μ	20 M	20 000
NPE	$\{\mu, c_d\}$	40 M	40 000
NPE	$\{\mu, c_d, m\}$	20 M	20 000
NPSE	μ	60 M	60 000
NPSE	$\{\mu, c_d\}$	60 M	60 000
NPSE	$\{\mu, c_d, m\}$	40 M	40 000
FNPSE	μ	40 M	40 000
FNPSE	$\{\mu, c_d\}$	20 M	20 000
FNPSE	$\{\mu, c_d, m\}$	20 M	20 000
Simformer	μ	40 M	40 000
Simformer	$\{\mu, c_d\}$	60 M	60 000
Simformer	$\{\mu, c_d, m\}$	40 M	40 000

Table 5.9: Simulative diagnostics of the checkpoints selected for real-data inference. W_2 : mean Wasserstein-2 distance across 100 held-out test cases. Coverage: fraction of test cases where the true parameter falls within the 50% and 90% credible intervals.

Method	Params	$W_2 \downarrow$	Cov. 50%	Cov. 90%	Sampling [s]
NPE	μ	0.020	0.55	0.89	0.19
NPSE	μ	0.031	0.43	0.90	1.32
FNPSE	μ	0.075	0.11	0.29	3.98
Simformer	μ	0.354	0.01	0.99	164.5
NPE	$\{\mu, c_d\}$	0.160	0.23	0.61	0.19
NPSE	$\{\mu, c_d\}$	0.160	0.23	0.54	2.13
FNPSE	$\{\mu, c_d\}$	0.203	0.00	0.04	3.31
Simformer	$\{\mu, c_d\}$	0.382	0.87	1.00	162.7
NPE	$\{\mu, c_d, m\}$	54.1	0.18	0.69	0.22
NPSE	$\{\mu, c_d, m\}$	112.3	0.10	0.55	2.32
FNPSE	$\{\mu, c_d, m\}$	69.1	0.09	0.27	8.64
Simformer	$\{\mu, c_d, m\}$	141.9	0.55	0.99	174.7

In the three-parameter configuration, the W_2 distance increases by three orders of magnitude, driven almost entirely by the mass parameter, confirming that mass is the hardest parameter to recover from the observation channels.

Per-Recording Analysis

Table 5.10 reports PPC RMSE for each of the seven recordings using a *randomly selected* 1 000-step window per recording, which for full-braking (Vollbremsung) recordings typically falls within the ABS braking phase. The recordings span three surface types and one mixed-surface scenario.

Table 5.10: Per-recording PPC RMSE for μ -only inference across all four methods and all seven Jeveresen recordings. Recordings are grouped by surface type. Each entry uses a randomly-selected $T_{\text{seg}} = 1000$ step window; for Vollbremsung recordings this window typically includes the hard ABS braking phase, where the simulator is most stressed. Mean $\pm$ std across recordings reported in the bottom row.

Recording	Surface	NPE	NPSE	FNPSE	Simf.
Asphalt 1	Asphalt	24.3	24.1	22.9	23.5
Asphalt 2	Asphalt	23.6	27.8	19.2	23.3
Concrete 1	Concrete	22.2	25.0	22.9	25.7
Concrete 2	Concrete	18.4	23.0	22.6	18.5
Basalt 1	Basalt	15.2	23.1	15.3	17.8
Basalt 2	Basalt	24.6	24.8	18.3	17.6
Mixed	Mixed	3.1	2.9	2.9	3.2
Mean $\pm$ std		18.8 $\pm$ 7.1	21.5 $\pm$ 7.7	17.7 $\pm$ 6.6	18.5 $\pm$ 6.9

The per-recording breakdown reveals that the mixed-surface recording achieves markedly lower PPC RMSE (~ 3) compared with the full-braking recordings (15–28). This recording contains gentler manoeuvres that remain within the linear tire operating regime, where the default simulator parameters provide a reasonable approximation. The full-braking recordings, by contrast, excite the nonlinear tire region where the Pacejka model parameters deviate most from the physical ground truth under the default parametrisation.

Among the braking recordings, FNPSE shows the most consistent performance across surfaces (range 15.3–22.9), while NPSE exhibits the highest variability (range 23.0–27.8 for braking recordings). The asphalt and concrete recordings produce PPC RMSE values of 18–28, reflecting the sim-to-real gap that the PSO calibration in Section 5.3.1 was designed to close. These residual errors are dominated by the wheel-speed and body-acceleration channels, whose

per-dimension RMSE values reach 34–36 for the braking recordings compared with 4–5 for the mixed recording.

Interpretation

The real-data results show that simulative posterior quality and real-data PPC RMSE do not align perfectly under the default simulator. Methods with sharp posteriors in simulation do not necessarily produce the lowest real-data PPC error, because the dominant factor on real recordings is the sim-to-real gap.

Importantly, these results are obtained with default simulator parameters, not the PSO-calibrated model. The PSO calibration in Section 5.3.1 demonstrated that the two-track model can achieve low normalised error ($\text{NRMSE} = 0.185$) with optimised global parameters, indicating that the simulator is structurally capable of reproducing the measured dynamics. The substantially higher PPC RMSE values observed here—mean 17–22 for braking recordings, approximately 3 for the mixed-surface recording (Table 5.10)—reflect the sim-to-real mismatch introduced by the default parametrisation of the fixed global simulator parameters (tire model coefficients, vehicle inertias, etc.) that lie outside the SBI prior. Note that because NRMSE and PPC RMSE use different normalisation conventions, these numbers are not directly comparable in magnitude; the qualitative point is that the default-parameter simulator leaves a substantial gap that the PSO-calibrated model can close. Reducing this gap for SBI-based inference would require either (i) expanding the inference targets to include additional simulator parameters, (ii) training the SBI methods on data generated with the PSO-calibrated simulator, or (iii) applying domain adaptation techniques to bridge the distribution shift.

The per-recording analysis further supports this interpretation: the mixed-surface recording, which involves gentler manoeuvres within the linear tire regime, yields PPC RMSE ≈ 3 across all methods, approaching the accuracy achievable by the default simulator. The full-braking recordings, which exercise the nonlinear tire dynamics where the default Pacejka coefficients are least accurate, produce PPC RMSE values of 15–28.

5.4 Discussion of Comparative Results

The benchmark results across 432 experiments reveal a nuanced picture in which no single method dominates on all criteria. This section synthesizes the findings into practical recommendations.

5.4.1 Method Strengths and Weaknesses

The following paragraphs summarise the relative strengths and weaknesses of each method based on the benchmark results.

NPE. NPE with normalising flows consistently achieves the best posterior accuracy across all parameter configurations and scales well to three parameters. It is the fastest method to train and offers near-instantaneous sampling. Its main weakness is moderate calibration: the posteriors, while accurate in location, are systematically overconfident and consistently undercovering at nominal credible levels. On real data, its narrow posteriors do not adapt as effectively to the sim-to-real gap, resulting in higher PPC RMSE than FNPSE or NPSE. NPE emerges as the strongest default choice when simulative accuracy is the primary concern.

NPSE. NPSE achieves accuracy comparable to NPE in the two-parameter case and ranks second in the single-parameter case. However, its calibration is the weakest overall: coverage collapses at longer trajectory lengths and the method shows a systematic tendency to produce overconfident posteriors. Its training time scales identically to NPE and its sampling time is practical for offline applications. On real data, NPSE achieves the best or second-best PPC RMSE, indicating that its score-based posteriors transfer effectively across the sim-to-real gap. NPSE may be preferred when score-based diffusion offers modelling advantages not captured in this benchmark (e.g., complex multi-modal posteriors), but the calibration concerns warrant caution.

FNPSE. FNPSE, which exploits the Markov structure of the dynamical system, achieves the worst or second-worst results on all posterior quality metrics in this benchmark. With two-step local windows, the factorised approach does not recover parameters that require integrating information across the full trajectory (such as mass). Training is substantially slower than NPE/NPSE due to internal trajectory generation. The method does show meaningful budget scaling and improvement with longer trajectories, but in its current implementation it does not achieve competitive posterior quality for this application. Notably, FNPSE achieves the lowest PPC RMSE on the μ -only configuration in the per-recording analysis (Table 5.10), but as discussed in Section 5.4.3 this should not be interpreted as accurate inference.

Simformer. The Simformer achieves the highest empirical 90%-coverage of any tested method in this benchmark, and the lowest C2ST-prior score (closest to the prior-equivalent value of 0.5), indicating posteriors that depart least from the prior. This coverage advantage

reflects the method’s tendency to produce wide, diffuse posteriors rather than genuinely learning the informative posterior shape. Simformer is the most robust to trajectory length due to its temporal subsampling, and it scales well to multiple parameters. Its main limitation is sampling speed, which makes it impractical for high-throughput applications. On real data, Simformer’s overdispersed posteriors yield moderate PPC RMSE, comparable to NPE, though these rankings are provisional given the use of default simulator parameters (Section 5.3.2).

5.4.2 The Calibration–Accuracy Trade-Off

A recurring theme across all configurations is the tension between posterior accuracy and calibration. Methods that produce narrow, accurate posteriors (NPE, NPSE) tend to be overconfident, while methods that produce high-coverage posteriors (Simformer) achieve this by being wider, which inflates the W_2 distance. No method in this benchmark achieves both high accuracy and nominal calibration simultaneously.

This trade-off is not unique to this work. The SBI literature has documented similar calibration challenges for normalising-flow-based NPE methods [HDR⁺22]. The empirical coverage advantage of Simformer likely stems from its joint training objective, which models the full data-generating process rather than only the posterior, providing a form of implicit regularisation. However, as shown in Section 5.4.3, this “calibration” is achieved primarily by remaining close to the prior rather than learning an informative posterior distribution.

The real-data evaluation in Section 5.3.2 adds a new dimension to this trade-off, though the results should be interpreted with caution given the use of default (uncalibrated) simulator parameters. Under those conditions, the method with the worst simulative calibration (FNPSE) achieves the lowest real-data PPC RMSE, while Simformer’s high-coverage but overdispersed posteriors yield only moderate PPC RMSE. This provisional evidence suggests that calibration quality alone does not guarantee better predictive performance on real data, and these rankings could change under a better-matched simulator.

A second methodological caveat applies: because the simulative test set was drawn from the tails of the prior (Section 5.1.2), the empirical coverage advantage of Simformer reflects the combined effect of (i) producing wide posteriors and (ii) wide posteriors being well-suited to tail-shifted test truths. Disentangling these contributions would require a re-evaluation under prior-drawn test parameters, which is identified as future work.

5.4.3 Limitations

The following limitations apply to this benchmark and are relevant to interpreting the results reported above. Some are methodological constraints; others reflect qualitative failures in the evaluated methods that require explicit acknowledgement (see also the broader methodological limitations in Section 6.2).

1. **FNPSSE posterior correctness failure.** The aggregate metric tables show that FNPSSE achieves near-zero empirical coverage, and this is sometimes described as “overconfidence.” The qualitative posterior comparison in Figure 5.4 reveals the more precise failure mechanism: on a non-trivial fraction of test cases, FNPSSE does not merely produce a narrow posterior—it collapses to an incorrect mode far from the true parameter value. This is a correctness failure, not only a calibration failure. The implication is that the FNPSSE training procedure, as implemented here with two-step local Markov windows, does not reliably converge to the correct posterior in this setting. The relatively low mean PPC RMSE that FNPSSE achieves on the μ -only configuration in the per-recording analysis (Table 5.10) should therefore not be interpreted as evidence of accurate inference; it reflects predictive coincidence for those recordings rather than a valid posterior.

A mechanistic explanation of the specific collapse value ($\mu \approx 0.5$, the lower design bound of the SBI prior) can be given as follows. Within a single two-step observation window, the change in vehicle trajectory induced by a small friction perturbation is small relative to the measurement noise level. The local score network therefore has weak discriminative power per window, and each local score contribution is dominated by the prior gradient. In the Gaussian-corrected score composition (Eq. 4.16), the prior correction term is multiplied by $(1 - T)$, where T equals the number of transition windows over the trajectory; for $T_{\text{seg}} = 1000$ this amounts to roughly $T \approx 1000$ windows and a prior correction factor of order -10^3 relative to the standard prior gradient. Because the standard prior gradient at any point $\theta < \mu_{\text{prior}} = 1.0$ points *toward* the prior mean (positive direction), multiplying by $(1 - T)$ reverses its sign: the effective gradient on the diffusion trajectory now points *away* from the prior mean, pushing samples toward the boundaries of the prior’s design range. The reason the lower boundary at $\mu \approx 0.5$ is reached preferentially rather than the upper boundary at $\mu \approx 1.5$ is that, during reverse diffusion, samples are explicitly clipped to the normalised prior bounds (Section 4.3.5) to prevent divergence; combined with the asymmetry introduced by the per-window data signal (which on most test cases marginally favours one direction over the other), this clipping creates an absorbing wall at the boundary the trajectory first encounters. The composition of $T - 1 \approx 999$ weak local scores against this strong, sign-flipped prior correction overwhelms the local data signal, and the clipped boundary becomes a stable

attractor for the reverse diffusion. This mechanism is consistent with prior-boundary attractors observed in score-based methods more generally when the per-window training signal is too weak to overcome the accumulated prior correction; eliminating it would likely require either (i) stronger per-window signal (longer windows, more informative observations) or (ii) a less aggressive prior-correction scaling than $(1 - T)$.

2. **Simformer empirical coverage through diffuseness.** Simformer consistently achieves the highest empirical coverage in this benchmark. However, Figure 5.4 shows that this coverage is achieved not by learning an informative posterior but by producing a distribution that remains close to the prior. A posterior that never moves far from the prior will often achieve high empirical coverage under prior-drawn test cases, but without reducing uncertainty relative to the prior. The empirical 90%-coverage near or above the nominal level reflects diffuseness rather than posterior correctness; a well-calibrated posterior must be both nominally covered *and* informative. In a practical inference scenario, a Simformer posterior would contribute very little to reducing uncertainty relative to the prior.
3. **Real-data evaluation without posterior ground truth.** On real vehicle recordings, the true friction coefficient μ is never known. All real-data conclusions in Section 5.3.2 are therefore based on predictive quality (PPC RMSE) rather than inferential accuracy. It is not possible to verify whether any method correctly identifies μ from real data, or whether the posteriors are well-calibrated on real observations. The simulative evaluation in Section 5.2 is the only setting where inferential correctness can be assessed; the real-data results are a necessary but incomplete complement.
4. **Sim-to-real gap under default parameters.** The real-data experiments use the default simulator parameters, not the PSO-calibrated model from Section 5.3.1. The PSO calibration achieves $\text{NRMSE} = 0.185$ on the 7 optimisation recordings (in-sample); the default model produces substantially higher prediction errors. The real-data PPC RMSE values (approximately 3 for the mixed-surface recording and 15–28 for full-braking recordings; Table 5.10) do not primarily reflect limitations of the SBI inference but rather the structural mismatch between the default simulator and the real vehicle. This gap cannot be closed by training more SBI data under the default model: the distributional shift is inherent to the parameter mismatch. A fairer evaluation would use the PSO-calibrated simulator for generating SBI training data; the current results are therefore a conservative estimate of the achievable real-data performance.
5. **Single simulator and fixed embedding architecture.** All experiments use the same two-track vehicle model with a fixed BiGRU-based embedding. Method rankings may shift for simulators with qualitatively different posterior landscapes (multi-modal, chaotic,

high-dimensional) or with alternative embedding architectures such as transformers or state-space models.

6. **Limited parameter set.** Only one, two, and three parameters are evaluated. Higher-dimensional inference problems, or configurations where the target parameters are more weakly identified (e.g., simultaneous estimation of all Pacejka coefficients), may reveal different relative strengths and expose scaling limitations not visible in the current benchmark.
7. **Statistical confidence at long trajectory lengths.** Configurations at $T_{\text{seg}} \geq 2000$ were run with only 2 seeds. The trajectory-length-dependent comparisons should be interpreted as directional evidence rather than statistically robust conclusions. Tables 5.2–5.5 in the body of this thesis report only the 5-seed $T_{\text{seg}} = 1000$ results. The trajectory-length comparisons in figures (Figure 5.7) and in the corresponding text use 2-seed measurements; standard deviations should not be interpreted as confidence intervals, and apparent within-method differences across T_{seg} of magnitude smaller than the inter-seed range should not be treated as established.
8. **Missing posterior parameter estimates for real data.** The evaluation pipeline does not save per-recording posterior mean and standard deviation for real data, so a direct numerical comparison between SBI posterior summaries and the PSO point estimates in Table 5.7 is not available. This should be remedied in a future implementation.
9. **PSO friction estimates do not preserve expected surface ordering.** The PSO per-trajectory μ values in Table 5.7 show basalt at $\mu = 1.60$ (the upper search bound), above asphalt and concrete (1.16–1.42). As discussed in Section 5.3.1, this ordering inversion results from global Pacejka reshaping rather than a physically interpretable friction level. The SBI real-data plausibility check based on surface type is therefore limited: the inferred posterior for basalt cannot be directly compared against the PSO reference value of $\mu = 1.60$, which is an artefact of the model compensation rather than the true friction coefficient.
10. **Embedding capacity not ablated.** All four methods condition on observation summaries produced by similarly-sized embedding networks: the BiGRU encoder used by NPE and NPSE produces a 64-dimensional summary, the FNPSE local GRU produces a 64-dimensional context vector, and Simformer’s transformer operates on 40-dimensional token embeddings. For 9-channel trajectories of up to 3 000 steps, this embedding dimension is conservative and was selected based on memory and runtime trade-offs rather than on a capacity ablation. The conclusion that NPE achieves the best simulative posterior accuracy in this benchmark therefore holds at this embedding capacity; whether the relative rankings persist under substantially larger embedding dimensions, hierarchical

encoders, or state-space models such as S4 or Mamba (Section 6.3.4) remains an open question.

5.4.4 Connection to Chapter 6

The implications for practical vehicle parameter estimation, the limitations of this study, and directions for future research are discussed in Chapter 6.

6 Conclusion and Outlook

This chapter summarises the main findings of the thesis, discusses the limitations of the current work, and outlines directions for future research.

6.1 Summary of Contributions

This thesis presented a controlled comparative study of four amortised SBI methods for static parameter estimation in a vehicle dynamics setting. The methods-NPE, NPSE, FNPSE, and Simformer-were evaluated within a common experimental pipeline using a two-track vehicle model with 9 observation channels and up to 3 inferred parameters (tire-road friction μ , aerodynamic drag c_d , and vehicle mass m). The benchmark comprised 432 experiments across 4 methods, 4 parameter configurations, 3 trajectory lengths, 3 simulation budgets, and multiple random seeds.

The evaluation protocol went beyond pointwise parameter error and included posterior quality metrics, empirical coverage diagnostics, prior-separation C2ST, posterior predictive checks, and runtime measurements. The five research questions posed in Chapter 1 are addressed as follows.

RQ1: Method comparison under matched budgets. Within this benchmark, NPE with normalising flows consistently achieves the lowest parameter-recovery W_2 (defined here as the RMSE between posterior samples and the generating parameter; see Section 5.1.2) across all parameter configurations on simulated data, and is the only method that scales accuracy with simulation budget across all three parameter dimensionalities. NPSE achieves comparable accuracy on the (μ) and (μ, c_d) configurations but degrades on configurations involving mass. FNPSE in the present implementation does not achieve competitive simulative posterior quality, primarily because its two-step local windows cannot integrate trajectory-level information. Simformer achieves the highest empirical 90%-coverage but produces diffuse, near-prior posteriors with the highest W_2 .

These rankings reverse partly on real measurement data (RQ5, below): under the default-simulator sim-to-real gap, FNPSE and NPSE produce lower posterior-predictive RMSE than

NPE, and Simformer’s wider posteriors better cover the unmodelled real-data variability. The simulative and real-data rankings are therefore not consistent, and the practical recommendation depends on which evaluation regime is more representative of the deployment scenario. For applications where the simulator is well-calibrated to the real system, the simulative rankings should dominate the recommendation; where a substantial sim-to-real gap is unavoidable, the real-data behaviour becomes more informative.

RQ2: Effect of trajectory length. Trajectory length affected the methods differently. NPSE showed the strongest degradation at long trajectories ($T_{\text{seg}} = 2000$), with calibration collapsing under the added temporal complexity. Simformer was the most robust to trajectory length variation due to its temporal subsampling mechanism. NPE showed moderate improvement with longer trajectories for multi-parameter estimation. FNPSE improved with longer trajectories because more local windows become available, but remained non-competitive in absolute terms.

RQ3: Calibration and posterior predictive consistency. Score-based methods did not uniformly improve calibration relative to the NPE baseline in this benchmark. NPSE was systematically overconfident, while FNPSE produced poorly calibrated posteriors. Simformer was the only method that consistently achieved nominal or above-nominal coverage, likely due to its joint generative training objective, although the tail-shifted test design (Section 5.1.2) confounds the calibration interpretation of this result. The results highlight a calibration–accuracy trade-off: no method in this benchmark achieved both high accuracy and nominal calibration simultaneously, though the generality of this observation beyond the present application remains to be tested.

RQ4: Runtime and efficiency trade-offs. NPE was the fastest method in both training (< 25 minutes at 60 M budget) and sampling. The forward-pass component of NPE sampling is under 0.01 s per sample on GPU; full posterior wall-clock times (including embedding and Python overhead) are reported per checkpoint in Table 5.9 and are typically under 0.2 s per posterior. NPSE had comparable training times but slower sampling (1–5 s). FNPSE was 3–10 $\times$ slower in training due to internal trajectory generation. Simformer’s sampling time (163–175 s per posterior as measured in Table 5.9) was the main practical bottleneck, making it unsuitable for high-throughput applications.

RQ5: Robustness on real measurement data. PSO calibration confirmed that the two-track model is structurally capable of reproducing the gross features of the measured dynamics (in-sample NRMSE = 0.185), albeit with parameter values that are not individually physically

meaningful (Section 5.3.1). Applying the trained SBI networks to seven brake-test recordings under the *default* (uncalibrated) simulator produced rankings that invert the simulative comparison: FNPSE achieved the lowest mean PPC RMSE for μ -only inference across the seven recordings (mean 17.7, Table 5.10), while NPE-the simulative leader-achieved a mean of 18.8. The most probable interpretation of this inversion is that NPE’s narrow simulative posteriors do not adapt to the sim-to-real gap, while methods producing broader posteriors (FNPSE’s mode-collapsed but locally-uncertain output, Simformer’s near-prior diffuseness) inadvertently absorb the simulator-induced residuals. This does not vindicate FNPSE or Simformer as parameter-inference methods-neither correctly identifies friction in simulation-but it does indicate that *posterior-predictive* performance and *posterior-correctness* performance can decouple under model misspecification, and that the simulative rankings reported in this thesis should not be transferred uncritically to deployment scenarios with a substantial sim-to-real gap. A re-evaluation under the PSO-calibrated simulator would be needed to determine whether this decoupling persists when the simulator is closer to the real system. This is identified as future work in Section 6.3.

A practical synthesis across RQs 1 and 5. No single method dominates in this benchmark when both simulative and real-data behaviour are considered. NPE is the recommended default for *simulator-based parameter recovery* where the simulator is trusted and online sampling speed matters. NPSE is a reasonable second choice when score-based modelling is preferred and calibration concerns can be tolerated. FNPSE in its current form is not recommended as a primary inference method, but its predictive performance under sim-to-real gap on μ -only inference suggests that hybrid strategies-for example, using NPE as the inference method with FNPSE-style posterior diffusion for predictive ensembling-may merit investigation. Simformer’s runtime and near-prior behaviour make it impractical as a primary inference method on this problem in its present configuration. These recommendations are conditional on the present benchmark scope and may shift under different simulators, parameter ranges, or training schedules.

6.2 Scope, Limitations, and Choice of Focus

The findings of this thesis reflect a deliberate choice of focus. The original task description identified two complementary directions: (a) a controlled comparative study of recent amortised SBI methods on a realistic vehicle-dynamics inference problem, and (b) an investigation of a hidden-Markov-model extension via score-based data assimilation. The work presented here pursues direction (a) in depth, and treats direction (b) as a documented future-work outline

(Section 6.3.1, Appendix A.1). The reasoning for this choice is the following.

Implementing and benchmarking the four methods at the level of fidelity required for a fair comparison consumed more of the available budget than initially anticipated. This was not solely a matter of writing four pipelines: the methods come from two different ML frameworks (PyTorch and JAX/Haiku), use different conventions for normalisation and noise scheduling, are sensitive to embedding-network and training-schedule choices in ways that the originating papers do not always specify, and require careful instrumentation to produce comparable diagnostics. The 432 experiments reported here represent a substantial integration effort whose value is independent of any methodological extension: even if the data-assimilation extension had been implemented, it would have been benchmarked against the same four methods, on the same simulator, with the same metrics. The comparative infrastructure built in this thesis is therefore a prerequisite for the extension, not an alternative to it.

The benchmark itself produced findings that reframe what the data-assimilation extension would need to achieve. In particular, FNPSE in its original full-state-Markov formulation does not, in the present implementation, reach competitive posterior accuracy on this engineering problem (Section 5.2.1, Section 5.4.3). This is consequential for the planned extension: the partial-observation scenario targeted by the data-assimilation extension is *strictly harder* than the full-observation case, so building a hidden-Markov FNPSE variant on top of an FNPSE backbone whose full-observation behaviour is already brittle would risk compounding two sources of error. The benchmark therefore identifies a methodological prerequisite-strengthening FNPSE’s full-observation performance, for example via longer windows or a more stable composition rule—that should precede the data-assimilation extension. This is itself a contribution: it converts a vague “extend to hidden states” directive into a concrete, evidence-based work order.

With this framing, the remaining limitations of the thesis are as follows.

Simulation-only ground truth. Quantitative posterior evaluation relies entirely on simulated data, where the generating parameter is known. For real measurement data, the true posterior is unknown, and the evaluation is restricted to plausibility checks and posterior-predictive consistency. The real-data results therefore cannot definitively establish which method produces the most accurate posterior in practice; they can only establish which methods produce predictively-consistent posteriors under a sim-to-real gap.

No sequential or online inference. All methods are applied in a single-shot amortised fashion: the full trajectory is provided at once and the posterior is returned in a single inference step. The thesis does not explore sequential SBI strategies in which the posterior is refined as more data arrive, which would be relevant for online vehicle state estimation.

Hidden-state extension as future work. The hidden-state and data-assimilation extension identified in the task description is documented conceptually (Section 6.3.1, Appendix A.1) and motivated by the benchmark findings, but is not implemented as a benchmarked method in this thesis. The trajectory-length and simulation-budget sweeps in the benchmark grid serve as the primary ablation dimensions for assessing data-efficiency and observation-window sensitivity in the meantime.

The benchmark-specific limitations (single simulator, fixed embedding architecture, limited parameter range, uneven seed counts at $T_{\text{seg}} \geq 2000$, embedding capacity not ablated, and default-parameter simulator for the real-data inference) were enumerated in Section 5.4.3 and are not repeated here.

6.3 Outlook and Future Directions

Several directions for extending this work emerge from the findings and limitations discussed above.

6.3.1 Score-Based Data Assimilation for Hidden Markov Models

A natural and promising extension, identified in the task description of this thesis, is the integration of data assimilation strategies within the NPSE framework. In many real-world dynamical systems, the full internal state is not directly observable: only a subset of the state variables appears in the measurement channels, while others remain latent. The current pipeline assumes that all selected observation channels are available (the observation vector is 9-dimensional while the full simulator state is 10-dimensional, with global position and yaw angle excluded by design). In practice, further sensor dropouts, missing channels, or fundamentally unobservable state components could arise.

Glöckler et al. [GTFM24] identify this extension explicitly (Section 5.2 of their paper) as a direction for compositional SBI in time-series settings. The key idea is to combine the Markov-factorised score estimation of FNPSE with a data assimilation step that marginalises over unobserved state variables at each time step. Rather than requiring the full state trajectory for score computation, a data assimilation method can provide filtered state estimates that account for partial observability.

Rozet and Louppe [RL23] propose score-based data assimilation, which formulates the filtering problem as a sequence of conditional score-based diffusion steps. Their approach is compatible with the score-based posterior estimation framework used in NPSE and FNPSE: the same

neural score network that estimates $\nabla_{\theta} \log p(\theta \mid \mathbf{y}_{0:T})$ can, in principle, be augmented with a data assimilation layer that handles missing observations within the diffusion sampling process.

Concretely, this extension would involve three components: (i) a state-space formulation in which only a subset of the simulator outputs $\mathbf{y}_t$ is observed, with the remaining components treated as latent; (ii) a score-based filtering step that estimates the marginal posterior $p(\mathbf{x}_t \mid \mathbf{y}_{0:t})$ at each time step using the diffusion framework; and (iii) integration of these filtered state estimates into the factorised posterior score computation. This would allow the inference pipeline to handle partially observed trajectories without requiring architectural changes to the underlying score network.

Due to time constraints, this extension could not be fully implemented and evaluated within the scope of the present thesis. However, the codebase and experimental pipeline developed here—including the modular simulator, the FNPSE implementation, and the evaluation framework—provide a foundation on which such an extension can be built. A more detailed formulation and feasibility analysis are provided in Appendix A.1.

6.3.2 Improved Calibration for Flow-Based Methods

The persistent calibration–accuracy trade-off observed across all configurations suggests that post-hoc calibration techniques could be valuable. Recent work on conformal prediction for SBI [HDR⁺22] and ensembling strategies could be applied to the NPE posteriors to improve coverage without sacrificing posterior precision. Alternatively, hybrid approaches that combine the accuracy of normalising flows with the calibration properties of diffusion-based sampling merit investigation.

6.3.3 Higher-Dimensional Parameter Spaces

Extending the inference target from 3 to 10–20 parameters (e.g., including individual tire model coefficients, suspension parameters, or per-axle friction) would test the scalability limits of each method. The PSO calibration in Section 5.3.1 already optimises 23 parameters simultaneously, demonstrating that the simulator supports higher-dimensional inference targets. Whether the SBI methods can recover meaningful posteriors in such settings remains an open question.

6.3.4 Alternative Embedding Architectures

The current pipeline supports BiGRU, causal CNN, and transformer-based embeddings, but only the BiGRU was used in the main benchmark. A systematic comparison of embedding

architectures-particularly state-space models such as S4 or Mamba, which are designed for long-sequence modelling-could improve the trajectory compression and thereby benefit all global methods (NPE, NPSE, Simformer).

6.3.5 Online and Sequential Inference

Extending the pipeline to sequential settings, where the posterior is updated as new trajectory segments arrive, would be relevant for real-time vehicle state estimation. Amortised SBI is naturally suited for this because the trained estimator can be re-evaluated on each new observation window. Combining this with the Markov structure exploited by FNPSE could yield an efficient online inference system.

Appendix

Appendix Contents

A.1	Data Assimilation Outlook	88
A.1.1	Motivation: From Full-State to Partial Observations	88
A.1.2	Score-Based Data Assimilation	89
A.1.3	Connection to FNPSE and Joint Inference	90
A.1.4	Application to Vehicle Dynamics	90
A.1.5	Expected Challenges	91

A.1 Data Assimilation Outlook

This appendix provides a more detailed formulation of the score-based data assimilation extension discussed in Section 6.3.1. The extension was explicitly identified in the thesis task description as a promising research direction and is discussed by Glöckler et al. [GTFM24] in Section 5.2 of their paper. Due to time constraints, this extension could not be implemented and evaluated within the present thesis; the following sections document the mathematical formulation, the algorithmic concept, and the expected challenges.

A.1.1 Motivation: From Full-State to Partial Observations

Consider a state-space model with a Markovian transition and an observation function:

$$\mathbf{x}_{t+1} \sim p(\mathbf{x}_{t+1} \mid \mathbf{x}_t, \boldsymbol{\theta}), \quad (\text{A.1})$$

$$\mathbf{y}_t \sim p(\mathbf{y}_t \mid \mathbf{x}_t), \quad (\text{A.2})$$

where $\mathbf{x}_t \in \mathbb{R}^{d_x}$ is the full latent state, $\mathbf{y}_t \in \mathbb{R}^{d_y}$ is the observation with $d_y \leq d_x$, and $\boldsymbol{\theta}$ denotes the static parameters. In the fully observed case ($d_y = d_x$ and $\mathbf{y}_t = \mathbf{x}_t + \boldsymbol{\epsilon}$ with small noise), the FNPSE score factorisation from Chapter 2 can be applied directly. When $d_y < d_x$, the latent states must be marginalised out, and the posterior becomes

$$p(\boldsymbol{\theta} \mid \mathbf{y}_{0:T}) = \int p(\boldsymbol{\theta} \mid \mathbf{x}_{0:T}) p(\mathbf{x}_{0:T} \mid \mathbf{y}_{0:T}) d\mathbf{x}_{0:T}. \quad (\text{A.3})$$

This integral is intractable for nonlinear systems because the posterior over state trajectories $p(\mathbf{x}_{0:T} \mid \mathbf{y}_{0:T})$ does not have a closed-form solution. Data assimilation provides a way to

approximate this quantity.

A.1.2 Score-Based Data Assimilation

Rozet and Louppe [RL23] propose a framework that decouples the learning of the state dynamics from the incorporation of observations. The key insight is that the score of a trajectory posterior can be decomposed as

$$\nabla_{\mathbf{x}_{0:T}} \log p(\mathbf{x}_{0:T} \mid \mathbf{y}_{0:T}) = \underbrace{\nabla_{\mathbf{x}_{0:T}} \log p(\mathbf{x}_{0:T})}_{\text{prior score}} + \underbrace{\nabla_{\mathbf{x}_{0:T}} \log p(\mathbf{y}_{0:T} \mid \mathbf{x}_{0:T})}_{\text{likelihood score}}, \quad (\text{A.4})$$

where the prior score captures the temporal dynamics and the likelihood score incorporates the observations. This decomposition mirrors Bayes' rule at the score level.

The prior score $\nabla_{\mathbf{x}_{0:T}} \log p(\mathbf{x}_{0:T})$ is learned offline using a score-based generative model trained on simulated state trajectories. Due to the Markov property, this score decomposes into local contributions:

$$\nabla_{\mathbf{x}_{0:T}} \log p(\mathbf{x}_{0:T}) = \nabla_{\mathbf{x}_0} \log p(\mathbf{x}_0) + \sum_{t=0}^{T-1} \nabla_{\mathbf{x}_{t:t+1}} \log p(\mathbf{x}_{t+1} \mid \mathbf{x}_t). \quad (\text{A.5})$$

This is analogous to the FNPSE score factorisation, but applied to the state trajectory rather than the parameter posterior. A local score network can be trained on one-step transitions, avoiding the need to process full trajectories during training.

If the observation model $p(\mathbf{y}_t \mid \mathbf{x}_t)$ is known (e.g., Gaussian noise on a known projection), the likelihood score can be computed analytically:

$$\nabla_{\mathbf{x}_{0:T}} \log p(\mathbf{y}_{0:T} \mid \mathbf{x}_{0:T}) = \sum_{t=0}^T \nabla_{\mathbf{x}_t} \log p(\mathbf{y}_t \mid \mathbf{x}_t). \quad (\text{A.6})$$

At inference time, the learned prior score and the analytical likelihood score are combined to guide a reverse diffusion process that generates state trajectories consistent with both the dynamics and the observations. Crucially, the observation model is introduced *only at inference time*, meaning the score network does not need to be retrained for different observation configurations. This enables zero-shot generalisation to varying sensor setups, missing channels, or different noise levels.

A.1.3 Connection to FNPSE and Joint Inference

The connection between score-based data assimilation and the FNPSE framework used in this thesis can be formulated as follows. In FNPSE, the posterior score over static parameters θ is factorised over transitions:

$$\nabla_{\theta} \log p(\theta \mid \mathbf{x}_{0:T}) \approx \sum_{t=0}^{T-1} \nabla_{\theta} \log \tilde{p}(\theta \mid \mathbf{x}_t, \mathbf{x}_{t+1}) - (T-1) \nabla_{\theta} \log p(\theta). \quad (\text{A.7})$$

When the states $\mathbf{x}_{0:T}$ are not directly observed, one could combine these two ideas into a joint inference scheme:

1. **State filtering.** Use the score-based data assimilation framework to obtain filtered state estimates $\hat{\mathbf{x}}_{0:T}$ from the partial observations $\mathbf{y}_{0:T}$, conditioned on a current parameter estimate θ_a .
2. **Parameter score estimation.** Evaluate the FNPSE local score network on the filtered state trajectory $\hat{\mathbf{x}}_{0:T}$ to obtain the parameter posterior score.
3. **Joint diffusion.** Iterate steps 1 and 2 within a joint reverse diffusion process that simultaneously refines both the state trajectory and the parameter posterior.

Glöckler et al. [GTFM24] suggest this combined approach in their discussion (Section 5.2), noting that the Markov-factorised score structure is naturally compatible with filtering-based state estimation. Recent work on joint parameter–state inference using transformer-based and NPSE-based architectures further supports the feasibility of this direction, though without the explicit data assimilation formulation described here.

A.1.4 Application to Vehicle Dynamics

In the vehicle dynamics setting of this thesis, a partial-observation scenario could arise in several ways:

- **Missing sensor channels.** If only a subset of the 9 measurement channels is available (e.g., only wheel speeds and yaw rate, without body accelerations), the inference must marginalise over the unobserved state components.
- **Internal state variables.** The two-track model contains internal state variables (e.g., individual tire slip states, suspension deflections) that are not included in the observation vector. While the current setup treats only the 9 measured channels as the state, a more detailed model could include additional latent components.

- **Sensor dropout.** In practice, individual sensors may fail intermittently, creating time-varying observation masks. Score-based data assimilation handles this naturally because the likelihood score can be adjusted at each time step to reflect which observations are available.

The vehicle simulator used in this thesis is Markovian by construction (RK4 integration of ODEs), so the transition structure required by both FNPSE and score-based data assimilation is available. The observation model for the measurement channels is approximately linear with Gaussian noise, meaning the likelihood score in Eq. (A.6) can be computed analytically.

A.1.5 Expected Challenges

Several practical challenges would need to be addressed in an implementation:

1. **Joint state–parameter diffusion.** Running a joint reverse diffusion over both the state trajectory $\mathbf{x}_{0:T}$ and the parameters $\boldsymbol{\theta}$ increases the dimensionality of the sampling problem significantly. For a trajectory of length $T = 1000$ with $d_x = 9$ state dimensions and $d_\theta = 3$ parameters, the joint diffusion operates in $\mathbb{R}^{9003}$. Efficient parameterization and careful noise scheduling are required.
2. **Approximation error accumulation.** Both the FNPSE score composition and the data assimilation likelihood guidance involve approximations. In the FNPSE composition, the Gaussian correction terms (covariance-weighted score aggregation) are already approximate; adding a second layer of approximation from the likelihood guidance could compound errors.
3. **Computational cost.** Each step of the reverse diffusion requires evaluating the local score network on all T transition pairs and computing the likelihood scores for all T observation steps. For long trajectories, this per-step cost is substantial and must be multiplied by the number of diffusion steps (typically 500 for VE-SDE sampling).
4. **Training data requirements.** The score network must be trained on state transitions that cover the state-space region relevant for the parameter range of interest. If the partial observability introduces large uncertainty in the filtered states, the score network must generalise to state regions it was not exposed to during training.

Despite these challenges, the theoretical compatibility between the Markov-factorised score estimation used in FNPSE and the score-based data assimilation framework of Rozet and Louppe makes this a well-motivated direction for future work. The modular codebase developed in this thesis—with separate components for simulation, score network training, and diffusion sampling—provides a starting point for such an implementation.

Bibliography

- [ABK⁺25] ARRUDA, Jonas ; BRACHER, Niels ; KÖTHE, Ullrich ; HASENAUER, Jan ; RADEV, Stefan T.: *Diffusion Models in Simulation-Based Inference: A Tutorial Review*. <http://dx.doi.org/10.48550/arXiv.2512.20685>. Version: Dezember 2025. – arXiv:2512.20685 [stat] version: 1
- [BDG⁺25] BOELTS, Jan ; DEISTLER, Michael ; GLOECKLER, Manuel ; TEJERO-CANTERO, Álvaro ; LUECKMANN, Jan-Matthis ; MOSS, Guy ; STEINBACH, Peter ; MOREAU, Thomas ; MURATORE, Fabio ; LINHART, Julia ; DURKAN, Conor ; VETTER, Julius ; MILLER, Benjamin K. ; HEROLD, Maternus ; ZIAEEMEHR, Abolfazl ; PALS, Matthijs ; GRUNER, Theo ; BISCHOFF, Sebastian ; KROUGLOVA, Nastya ; GAO, Richard ; LAPPALAINEN, Janne K. ; MUCSÁNYI, Bálint ; PEI, Felix ; SCHULZ, Auguste ; STEFANIDI, Zinovia ; RODRIGUES, Pedro ; SCHRÖDER, Cornelius ; ZAID, Faried A. ; BECK, Jonas ; KAPOOR, Jaivardhan ; GREENBERG, David S. ; GONÇALVES, Pedro J. ; MACKE, Jakob H.: sbi reloaded: a toolkit for simulation-based inference workflows. In: *JOSS* 10 (2025), April, Nr. 108, 7754. <http://dx.doi.org/10.21105/joss.07754>. – DOI 10.21105/joss.07754. – ISSN 2475–9066
- [BFH⁺18] BRADBURY, James ; FROSTIG, Roy ; HAWKINS, Peter ; JOHNSON, Matthew J. ; LEARY, Chris ; MACLAURIN, Dougal ; NECULA, George ; PASZKE, Adam ; VANDERPLAS, Jake ; WANDERMAN-MILNE, Skye ; ZHANG, Qiao: *JAX: Composable transformations of Python+NumPy programs*. <http://github.com/jax-ml/jax>. Version: 2018
- [CBL20] CRANMER, Kyle ; BREHMER, Johann ; LOUPPE, Gilles: The frontier of simulation-based inference. In: *Proc. Natl. Acad. Sci. U.S.A.* 117 (2020), Dezember, Nr. 48, 30055–30062. <http://dx.doi.org/10.1073/pnas.1912789117>. – DOI 10.1073/pnas.1912789117. – ISSN 0027–8424, 1091–6490. – arXiv:1911.01429 [stat]
- [CMG⁺14] CHO, Kyunghyun ; MERRIËNBOER, Bart van ; GULCEHRE, Caglar ; BAH-DANAU, Dzmitry ; BOUGARES, Fethi ; SCHWENK, Holger ; BENGIO, Yoshua: Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. In: MOSCHITTI, Alessandro (Hrsg.) ; PANG, Bo (Hrsg.) ;

- DAELEMANS, Walter (Hrsg.): *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Doha, Qatar : Association for Computational Linguistics, Oktober 2014, 1724–1734
- [GDW⁺24] GLOECKLER, Manuel ; DEISTLER, Michael ; WEILBACH, Christian D. ; WOOD, Frank ; MACKE, Jakob H.: All-in-one simulation-based inference. In: *Proceedings of the 41st International Conference on Machine Learning*, PMLR, Juli 2024. – ISSN 2640–3498, 15735–15766
- [GNM19] GREENBERG, David ; NONNENMACHER, Marcel ; MACKE, Jakob: Automatic Posterior Transformation for Likelihood-Free Inference. In: *Proceedings of the 36th International Conference on Machine Learning*, PMLR, Mai 2019. – ISSN 2640–3498, 2404–2414
- [GPM23] GEFFNER, Tomas ; PAPAMAKARIOS, George ; MNIH, Andriy: Compositional Score Modeling for Simulation-Based Inference. In: *Proceedings of the 40th International Conference on Machine Learning*, PMLR, Juli 2023. – ISSN 2640–3498, 11098–11116
- [GTFM24] GLOECKLER, Manuel ; TOYOTA, Shoji ; FUKUMIZU, Kenji ; MACKE, Jakob H.: Compositional simulation-based inference for time series, 2024
- [HCNB20] HENNIGAN, Tom ; CAI, Trevor ; NORMAN, Tamara ; BABUSCHKIN, Igor: *Haiku: Sonnet for JAX*. <http://github.com/deepmind/dm-haiku>. Version: 2020
- [HDR⁺22] HERMANS, Joeri ; DELAUNOY, Arnaud ; ROZET, François ; WEHENKEL, Antoine ; BEGY, Volodimir ; LOUPPE, Gilles: *A Trust Crisis In Simulation-Based Inference? Your Posterior Approximations Can Be Unfaithful*. <http://dx.doi.org/10.48550/arXiv.2110.06581>. Version: Dezember 2022. – arXiv:2110.06581 [stat]
- [HJA20] HO, Jonathan ; JAIN, Ajay ; ABBEEL, Pieter: *Denoising Diffusion Probabilistic Models*. <http://dx.doi.org/10.48550/arXiv.2006.11239>. Version: Dezember 2020. – arXiv:2006.11239 [cs]
- [Hyv05] HYVÄRINEN, Aapo: Estimation of Non-Normalized Statistical Models by Score Matching. In: *Journal of Machine Learning Research* 6 (2005), Nr. 24, 695–709. <http://jmlr.org/papers/v6/hyvarinen05a.html>. – ISSN 1533–7928
- [KALA22] KARRAS, Tero ; AITTALA, Miika ; LAINE, Samuli ; AILA, Timo: Elucidating the design space of diffusion-based generative models. In: *Proceedings of the 36th*

- International Conference on Neural Information Processing Systems*. Red Hook, NY, USA : Curran Associates Inc., 2022 (NIPS '22). – ISBN 9781713871088
- [KB14] KINGMA, Diederik P. ; BA, Jimmy: Adam: A Method for Stochastic Optimization. In: *CoRR* abs/1412.6980 (2014). <https://api.semanticscholar.org/CorpusID:6628106>
- [KE95] KENNEDY, J. ; EBERHART, R.: Particle swarm optimization. In: *Proceedings of ICNN'95 - International Conference on Neural Networks* Bd. 4, 1995, S. 1942–1948 vol.4
- [LBG⁺21] LUECKMANN, Jan-Matthis ; BOELTS, Jan ; GREENBERG, David ; GONCALVES, Pedro ; MACKE, Jakob: Benchmarking Simulation-Based Inference. In: *Proceedings of The 24th International Conference on Artificial Intelligence and Statistics*, PMLR, März 2021. – ISSN 2640–3498, 343–351
- [LCG⁺25] LINHART, Julia ; CARDOSO, Gabriel ; GRAMFORT, Alexandre ; CORFF, Sylvain L. ; RODRIGUES, Pedro L. C.: Diffusion posterior sampling for simulation-based inference in tall data settings. In: *Transactions on Machine Learning Research* (2025), November. <https://openreview.net/forum?id=cdhfoS6Gyo>. – ISSN 2835–8856
- [Pac12] PACEJKA, Hans B.: *Tire and vehicle dynamics*. 3rd ed. Oxford, UK : Butterworth-Heinemann, 2012. – ISBN 978–0–08–097017–2
- [PM16] PAPAMAKARIOS, George ; MURRAY, Iain: Fast ε -free Inference of Simulation Models with Bayesian Conditional Density Estimation. In: *Advances in Neural Information Processing Systems* Bd. 29, Curran Associates, Inc., 2016
- [PNR⁺21] PAPAMAKARIOS, George ; NALISNICK, Eric ; REZENDE, Danilo J. ; MOHAMED, Shakir ; LAKSHMINARAYANAN, Balaji: Normalizing Flows for Probabilistic Modeling and Inference. In: *Journal of Machine Learning Research* 22 (2021), Nr. 57, 1–64. <http://jmlr.org/papers/v22/19-1028.html>. – ISSN 1533–7928
- [PPM17] PAPAMAKARIOS, George ; PAVLAKOU, Theo ; MURRAY, Iain: Masked Autoregressive Flow for Density Estimation. In: *Advances in Neural Information Processing Systems* Bd. 30, Curran Associates, Inc., 2017
- [RL23] ROZET, François ; LOUPPE, Gilles: Score-based Data Assimilation. In: *Thirty-seventh Conference on Neural Information Processing Systems*, 2023
- [RM16] REZENDE, Danilo J. ; MOHAMED, Shakir: *Variational Inference with Normalizing Flows*. <http://dx.doi.org/10.48550/arXiv.1505.05770>.

Version: Juni 2016. – arXiv:1505.05770 [stat]

- [SSDK⁺21] SONG, Yang ; SOHL-DICKSTEIN, Jascha ; KINGMA, Diederik P. ; KUMAR, Abhishek ; ERMON, Stefano ; POOLE, Ben: *Score-Based Generative Modeling through Stochastic Differential Equations*. <http://dx.doi.org/10.48550/arXiv.2011.13456>. Version: Februar 2021. – arXiv:2011.13456 [cs]
- [SSLB24] SHARROCK, Louis ; SIMONS, Jack ; LIU, Song ; BEAUMONT, Mark: Sequential neural score estimation: likelihood-free inference with conditional score based diffusion models. In: *Proceedings of the 41st International Conference on Machine Learning* Bd. 235. Vienna, Austria : JMLR.org, Juli 2024 (ICML'24), S. 44565–44602
- [TCBD⁺20] TEJERO-CANTERO, Alvaro ; BOELTS, Jan ; DEISTLER, Michael ; LUECKMANN, Jan-Matthis ; DURKAN, Conor ; GONÇALVES, Pedro ; GREENBERG, David ; MACKE, Jakob: sbi: A toolkit for simulation-based inference. In: *JOSS* 5 (2020), August, Nr. 52, 2505. <http://dx.doi.org/10.21105/joss.02505>. – DOI 10.21105/joss.02505. – ISSN 2475–9066
- [ZMSDH25] ZAMMIT-MANGION, Andrew ; SAINSBURY-DALE, Matthew ; HUSER, Raphaël: Neural Methods for Amortized Inference. In: *Annual Review of Statistics and Its Application* 12 (2025), März, Nr. 1, 311–335. <http://dx.doi.org/10.1146/annurev-statistics-112723-034123>. – DOI 10.1146/annurev-statistics-112723-034123. – ISSN 2326–8298, 2326–831X